



Ho Chi Minh City University of Technology and Education
Faculty of Electrical & Electronics Engineering
Department of Computer & Communications Engineering



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

BÁO CÁO CUỐI KÌ

THIẾT KẾ MÔ HÌNH IOT AQUANOVA

GVHD: ThS. Trịnh Quốc Thanh

SVTT: Nhóm 2

Môn: Kiến trúc và giao thức IoT



Ho Chi Minh City University of Technology and Education
Faculty of Electrical & Electronics Engineering
Department of Computer & Communications Engineering



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

DANH SÁCH THÀNH VIÊN

Hoàng Ngọc Dung
Đoàn Minh Duy Bình
Trần Hữu Dương

Mssv: 23139006
Mssv: 23139005
Mssv: 23139009



Nội dung báo cáo



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

Chương 1: Giới thiệu tổng quan về đề tài

Chương 2: Cơ sở lý thuyết

Chương 3: Thiết kế và thi công mô hình

Chương 4: Thực nghiệm, kiểm chứng và đánh giá

Chương 5: Kết luận và hướng phát triển





Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI



Chương 1: Giới thiệu tổng quan về đề tài



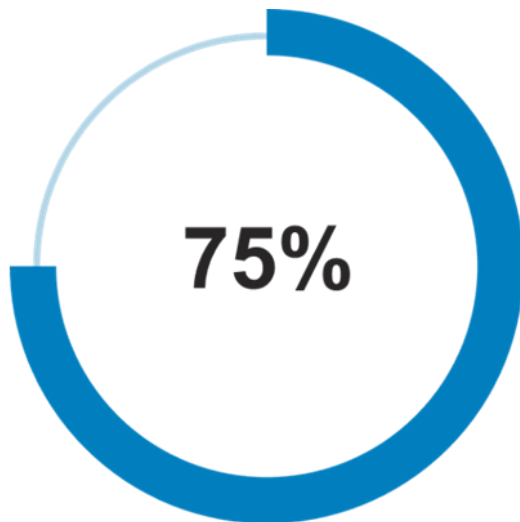
Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

1. Tình hình nghiên cứu
2. Đặt vấn đề
3. Mục tiêu của đề tài
4. Nội dung nghiên cứu
5. Đối tượng, phạm vi nghiên cứu
6. Phương pháp nghiên cứu



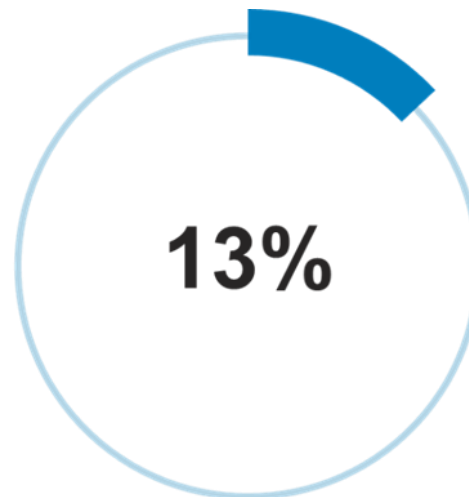
1. Tình hình nghiên cứu

Tỉ lệ cách thức chăm sóc cá cảnh của người nuôi:



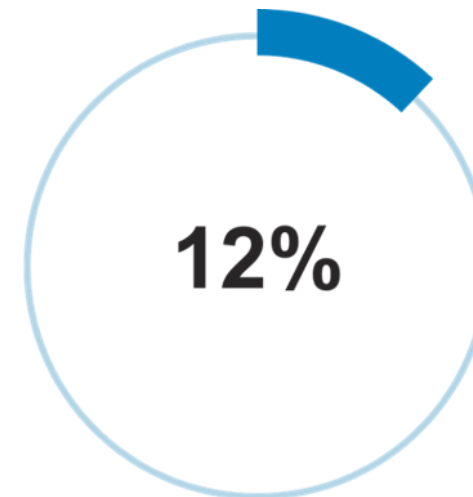
Tự Chăm Sóc

Người nuôi cá cảnh tự chăm sóc cá của mình.



Người Thân Chăm Sóc

Giao cho người thân chăm sóc.



Cả Gia Đình

Cả gia đình cùng chăm sóc.

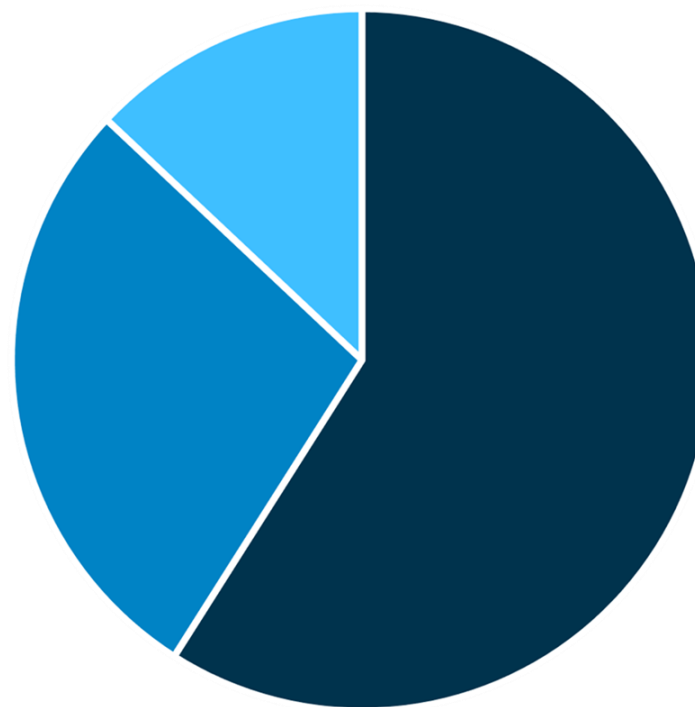


1. Tình hình nghiên cứu



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

Thời gian người nuôi dành cho cá cảnh:



■ Dưới 30 phút/ngày

■ 30 - 60 phút/ngày

■ Hơn 60 phút/ngày

2. Đặt vấn đề

- Nhu cầu: cần hệ thống chăm sóc cá cảnh tự động, giám sát và cảnh báo từ xa do người nuôi bận rộn.
- Tiềm năng: ứng dụng IoT (ESP32, MQTT) giúp theo dõi môi trường hồ cá theo thời gian thực.
- Thách thức: cần giải quyết vấn đề ổn định Wi-Fi, bảo mật dữ liệu và khả năng mở rộng hệ thống.





3. Mục tiêu của đề tài



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

Đề tài hướng đến việc thiết kế và triển khai một mô hình giám sát và chăm sóc cá cảnh tự động AquaNova:

Thiết kế chức năng hẹn giờ và điều khiển cho ăn tự động có đèn sáng/tắt

Sử dụng vi điều khiển STM32 làm bộ xử lý trung tâm

Thiết kế module cảm biến đo độ đục, nhiệt độ và đo mức thức ăn tích hợp cơ chế cảnh báo khi độ đục và ghi nhận lịch sử tiêu thụ.

Hiệu chỉnh các thông số đồng thời phân tích số liệu

Tích hợp ESP32 cho truyền dữ liệu và giám sát từ xa

Xây dựng website để hiển thị dashboard



4. Nội dung nghiên cứu



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

- Nghiên cứu lý thuyết
- Thiết kế phần cứng nhúng
- Thiết kế phần mềm (Backend & Frontend)
- Tích hợp và kiểm thử
- Ứng dụng và mở rộng



5. Đối tượng, phạm vi nghiên cứu

Đối Tượng Nghiên Cứu: Các Loài Cá Cảnh Nước Ngọt:



Cá Molly

Ăn tạp thiên về thực vật,
0.5g/ngày, 1-3 lần/ngày.



Cá Neon tetra

Cá nhỏ bơi đàn, 1g/ngày,
1-2 lần/ngày.



Cá Vàng

Ăn nhiều, 5g/ngày, 1
lần/ngày.



Cá Lau Kính

Ăn đáy, ăn đêm, cần tảo
và rau củ.



5. Đối tượng và phạm vi nghiên cứu



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

Phạm vi nghiên cứu & độ chính xác:

Kiến trúc điều khiển

STM32 (điều khiển thời gian thực), ESP32 (kết nối Wi-Fi, IoT).

Môi trường thực nghiệm

Hồ cá tại nhà kính $50 \times 27 \times 30$ cm (tối thiểu 30 lít và tối đa 40 lít) với thức ăn khô.

Giả định về dòng chảy và áp lực nước

Độ chính xác đo

- Nhiệt độ: sai số $\leq \pm 0,5$ °C.
- Độ đục: độ lặp lại $\leq 5\%$.
- Định lượng cho ăn: sai số $\leq \pm 10\%$.
- Độ tin cậy: uptime $\geq 90\%$ /tuần, độ trễ MQTT ≤ 1 s.



6. Phương pháp nghiên cứu



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

Thu thập số liệu

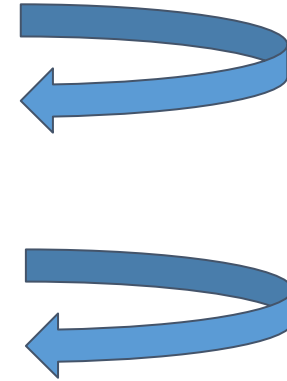
Sử dụng phương pháp quan sát và tiến hành phỏng vấn khảo sát

Thực nghiệm

Thiết kế, triển khai và kiểm thử mô hình tự động.

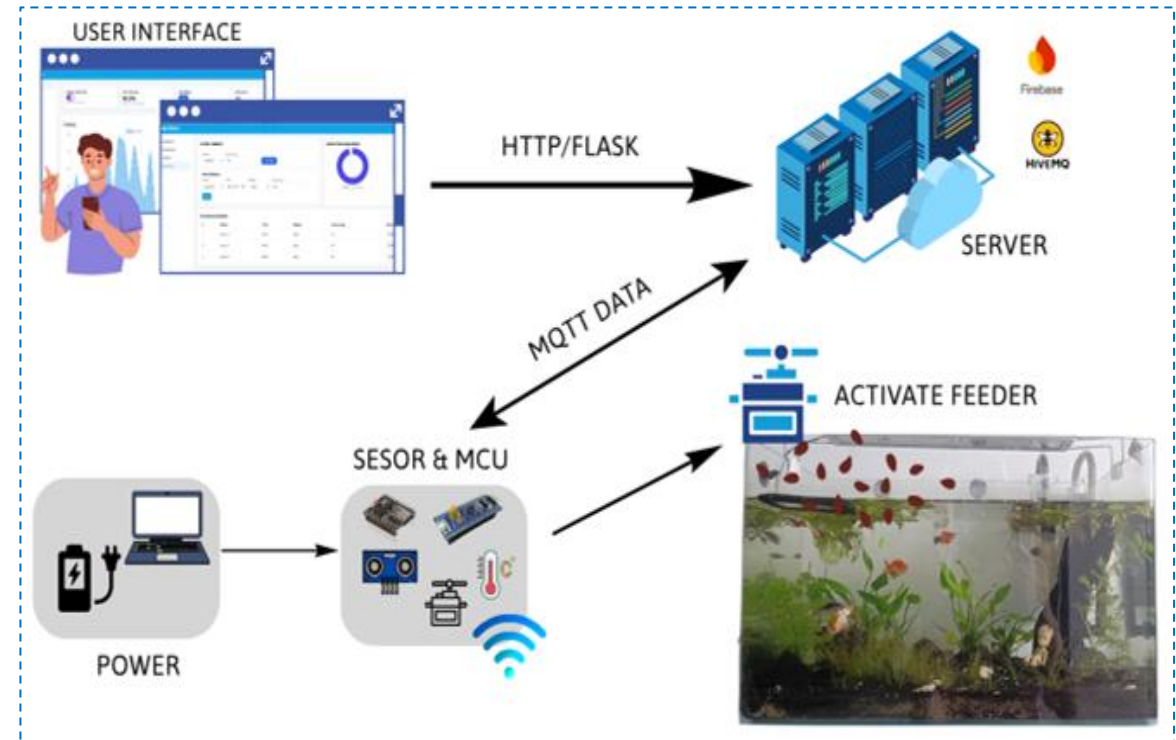
Phân tích & Tổng kết

Đánh giá hiệu quả, ưu nhược điểm và đề xuất giải pháp cải tiến.



Chương 3: Thiết kế và thi công mô hình

- 3.1. Yêu cầu về tính năng của mô hình
- 3.2. Yêu cầu về kỹ thuật
- 3.3. Mô hình tổng quan hệ thống
- 3.4. Thiết kế kiến trúc của mô hình
- 3.5. Thiết kế cơ khí
- 3.6. Thiết kế nhúng
- 3.7. Thiết kế website
- 3.8. Hiệu chuẩn và kiểm thử





Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT



2.1. Mô hình giám sát chất lượng nước và cung cấp thức ăn tự động



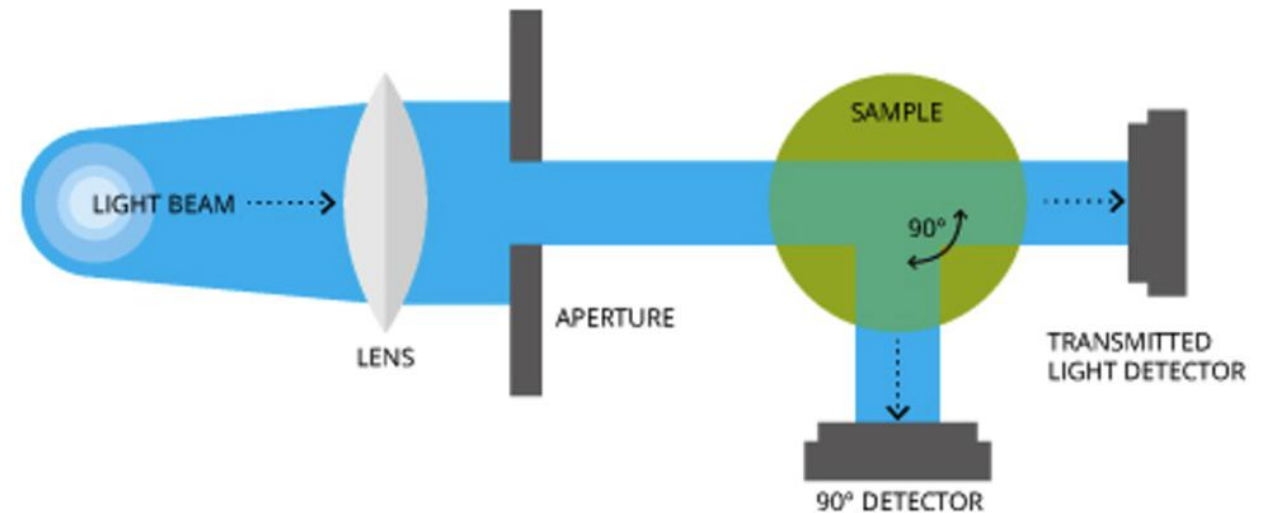
Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

- Giám sát môi trường:
 - Theo dõi liên tục các thông số: Nhiệt độ, Độ đục của nước, lượng thức ăn
 - Phát cảnh báo khi thông số vượt ngưỡng cho phép.
- Tự động hóa chăm sóc:
 - Điều khiển bộ cho ăn tự động và bật tắt đèn qua relay.
 - Cho phép cài đặt lịch cho ăn hoặc kích hoạt từ xa qua ứng dụng.



2.1.2. Nguyên lý đo độ đục của nước

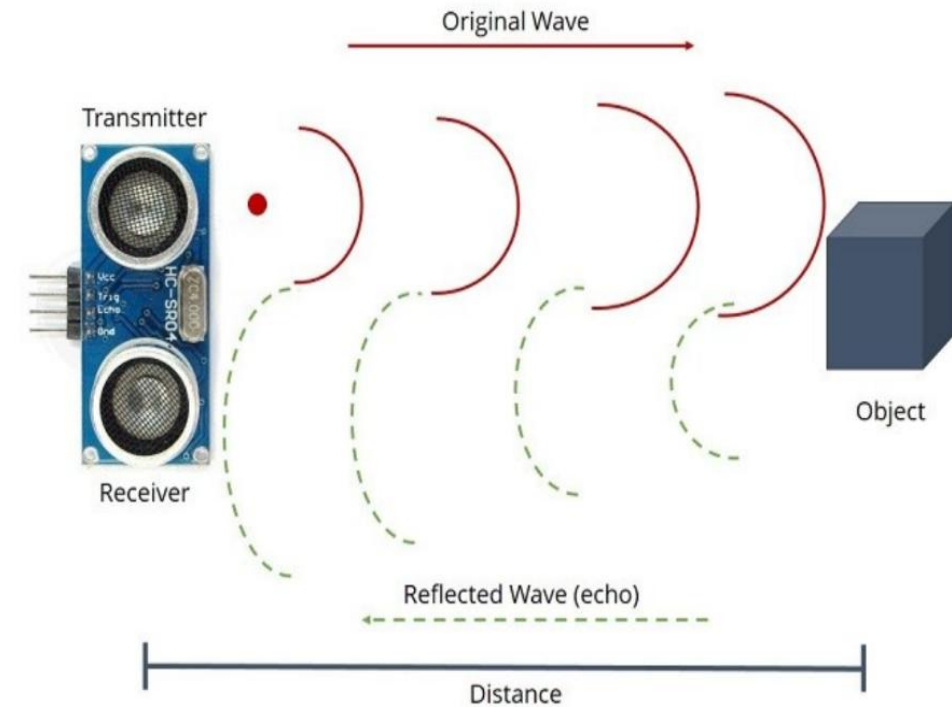
- Định nghĩa: Là thước đo độ "vẩn đục" hay "trong" của nước, gây ra bởi các hạt lơ lửng (như tảo, bụi bẩn, khoáng chất, vi khuẩn...).
- Phân biệt: khác với TDS (Tổng chất rắn hòa tan). Độ đục đo lường khả năng truyền sáng (tán xạ/hấp thụ), không phải khối lượng chất hòa tan.
- Đơn vị đo: NTU, FNU, FTU, FAU (được xem là tương đương).



2.1.2. Hiện tượng phản xạ sóng âm trong cảm biến siêu âm

Nguyên lý cốt lõi: Dựa trên hiện tượng phản xạ sóng âm (Echo). Cách thức hoạt động :

- + Phát (Trigger): Bộ phát tạo một chùm sóng siêu âm (thường 40 kHz) truyền đi trong không khí.
- + Phản xạ: Sóng gặp vật cản và bị dội ngược trở lại.
- + Thu: Bộ thu ghi nhận tín hiệu sóng phản xạ.



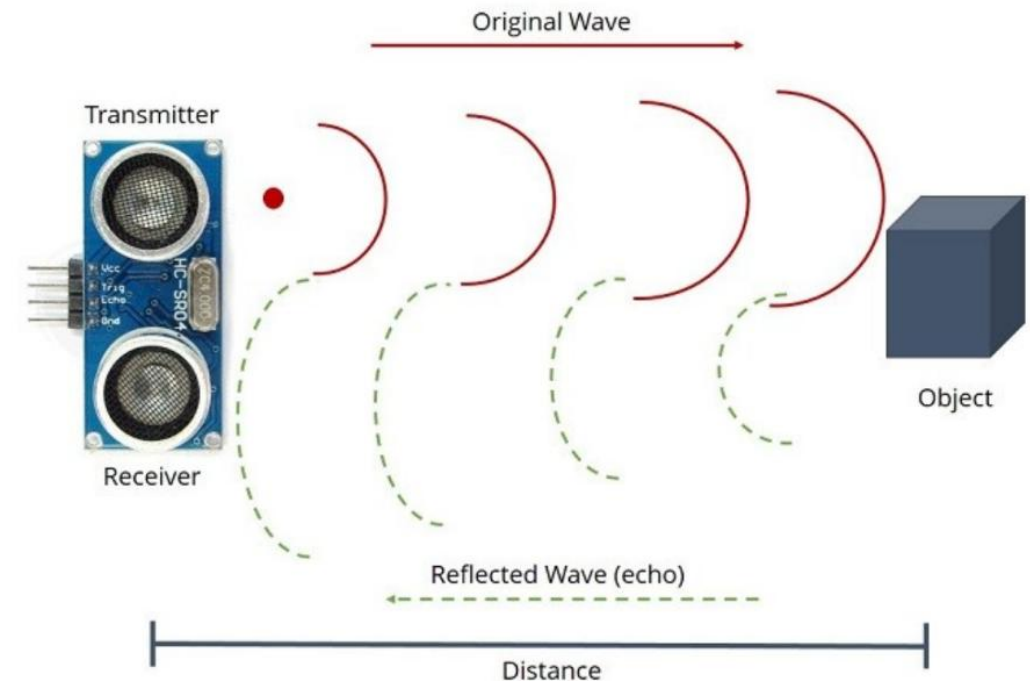
2.1.3. Hiện tượng phản xạ sóng âm trong cảm biến siêu âm

Tính toán khoảng cách:

- + Hệ thống đo tổng thời gian từ lúc phát đến lúc nhận về (t).
- + Vận tốc âm thanh trong không khí (v) là đã biết (khoảng 343 m/s ở 25°C).
- + Công thức: Khoảng cách = (Vận tốc * Thời gian) / 2

VĐK chỉ cần đo độ rộng xung này để tính khoảng cách:

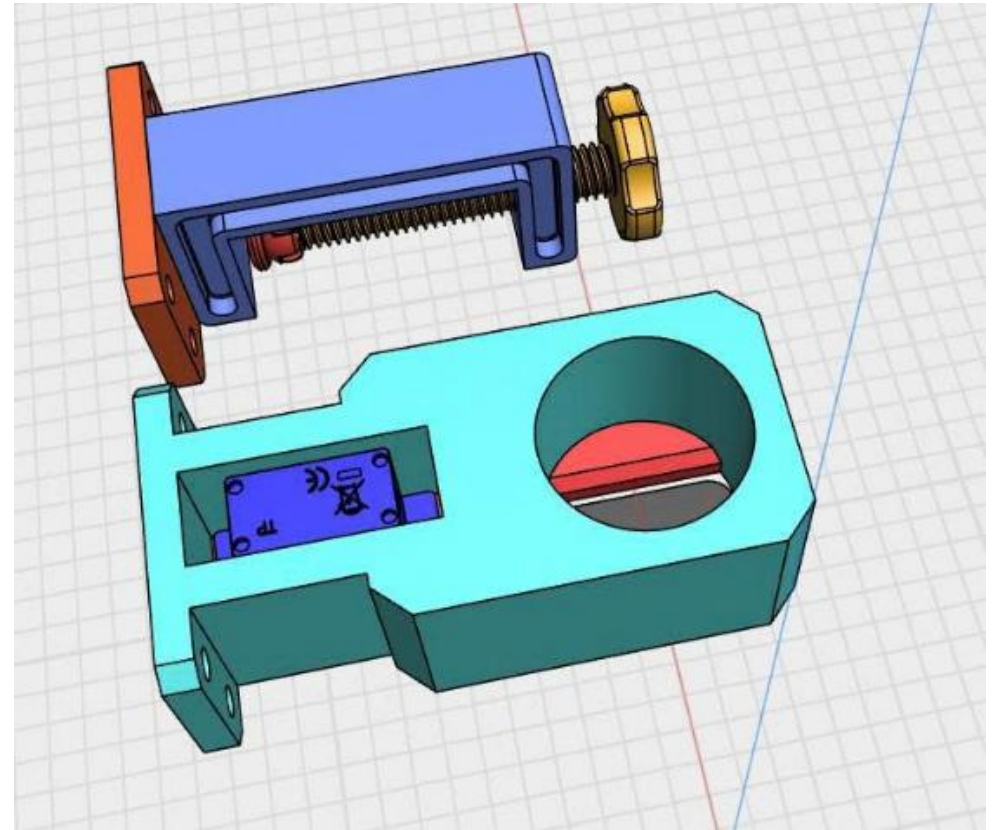
$$\text{Khoảng cách (m)} = T_{\text{echo}} (\text{giây}) * 343 / 2$$



2.1.4. Hình thức cung cấp thức ăn

Cấu tạo chính:

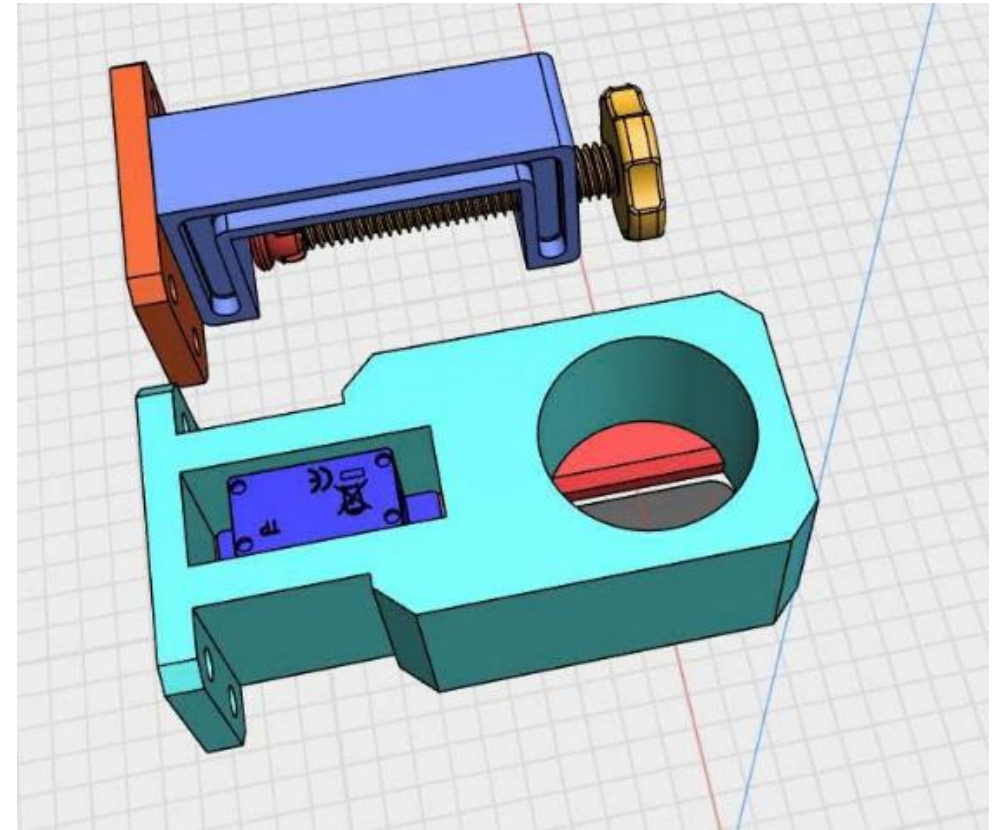
- + Phễu chứa (Hopper): Một chai nhựa thông thường được đảo ngược.
- + Cơ cấu chấp hành (Actuator): 1 động cơ Servo.
- + Van/Cửa xả: Cánh tay servo (servo horn) hoặc tấm gạt, hoạt động như một "cánh cửa" ngay miệng chai.



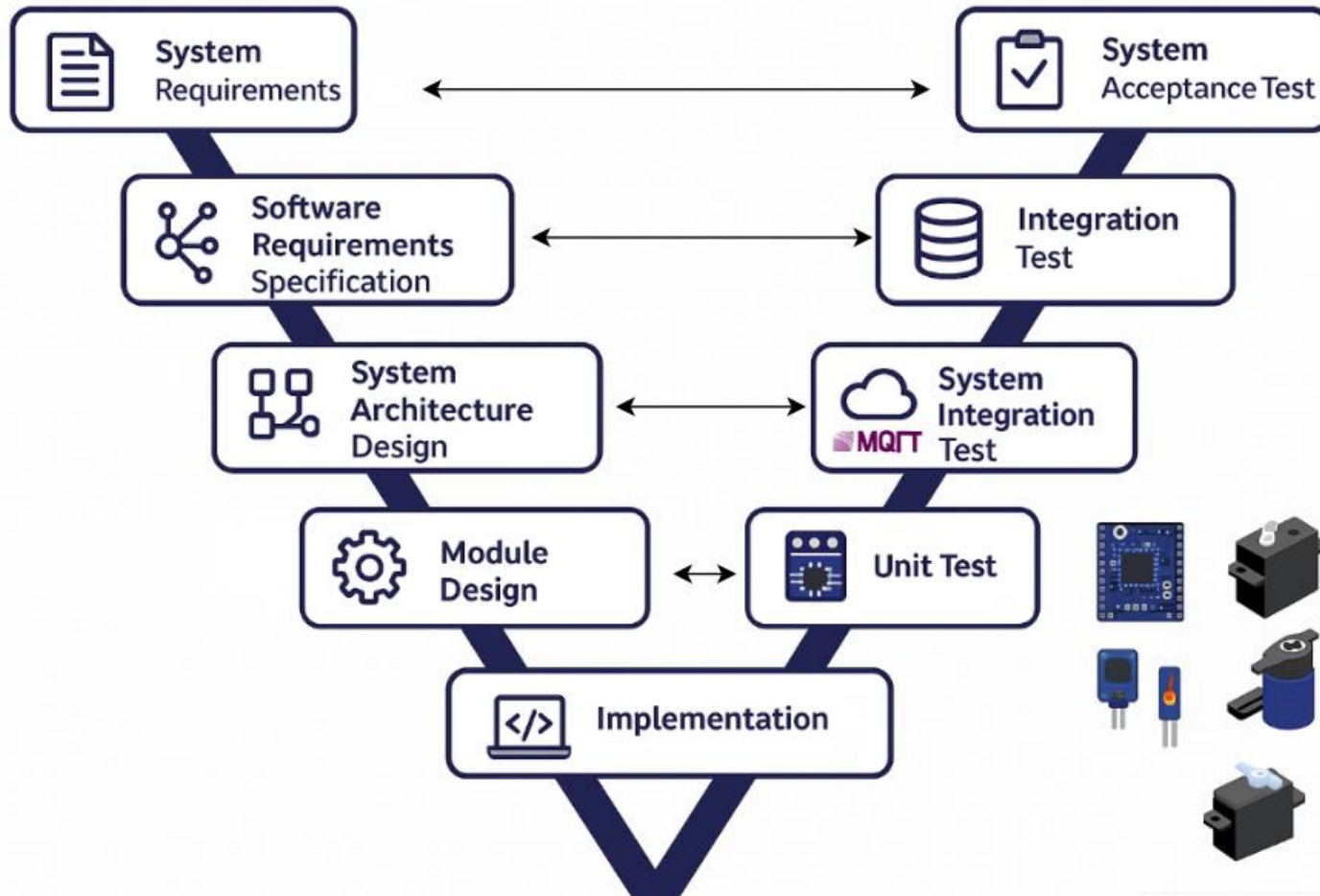
2.1.4. Hình thức cung cấp thức ăn

Nguyên lý hoạt động:

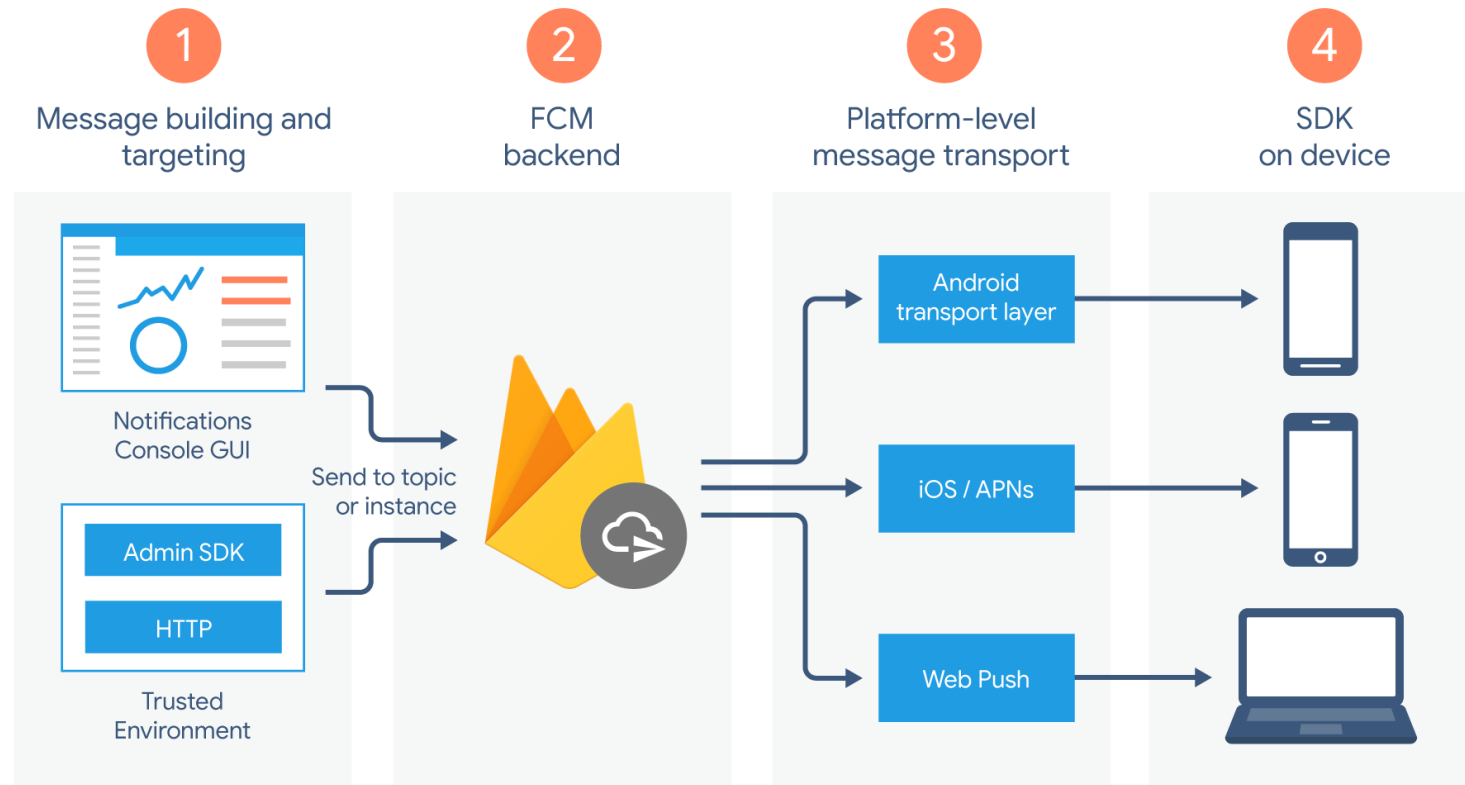
- Trạng thái chờ: Servo giữ cánh tay gạt ở vị trí ĐÓNG, ngăn thức ăn rơi ra.
- Kích hoạt (có tín hiệu): Servo quay, di chuyển tấm gạt → tạo khoảng hở (vị trí MỞ).
- Phân phối: Thức ăn tự động rơi tự do qua khoảng hở nhờ trọng lực.
- Kết thúc: Servo quay trở lại vị trí ĐÓNG, đóng kín.



2.1.5. Mô hình chữ V



2.2.1. Firebase



Sơ đồ quy trình hoạt động của Firebase Cloud Messaging

2.2.2. Cloud Firestore



Cloud
Firestore



Cơ chế đồng bộ dữ liệu thời gian thực của Cloud Firestore



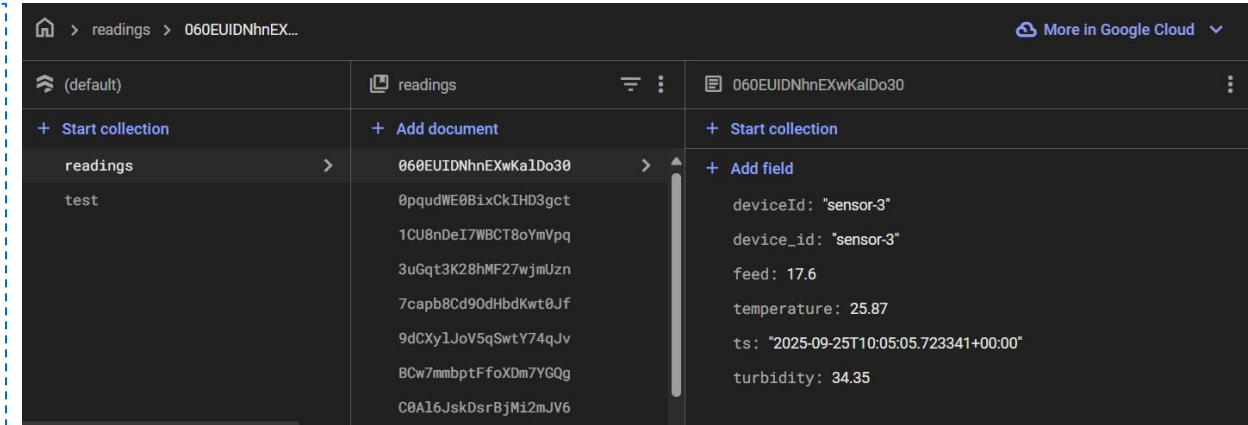
2.2. Cơ sở dữ liệu.



2.2.2. Cloud Firestore – cơ sở dữ liệu của Firebase

Tổ chức theo Collections và Documents

- Đồng bộ thời gian thực: Tự động cập nhật khi dữ liệu thay đổi.
- Khả năng mở rộng: Hỗ trợ xử lý khối lượng lớn dữ liệu và người dùng.
- Truy vấn mạnh mẽ: Hỗ trợ lọc, sắp xếp, phân trang.
- Bảo mật & phân quyền: Firebase Security Rules ở mức tài liệu/tập hợp.
- Độ tin cậy cao: Multi-region replication, chống mất mát dữ liệu.



Ví dụ Payload: JSON

```
{  
  "device_id": "sensor-1",  
  "ts": "2025-09-30T10:01:05Z",  
  "temperature": 30.5,  
  "turbidity": 48.5,  
  "feed": 51.6  
}
```

2.3. UI (User interface) & UX (User experience)

- HTML tạo cấu trúc và bố cục nội dung.
- CSS định dạng và thiết kế giao diện trực quan, màu sắc, phông chữ.
- JavaScript giúp trang web trở nên tương tác, xử lý dữ liệu động và cập nhật thời gian thực từ Flask Server.





2.3. UI (User interface) và UX (User experience)

2.3.1. HTML (HyperText Markup Language)

Định nghĩa: HTML (HyperText Markup Language) là ngôn ngữ căn bản dùng để xây dựng cấu trúc và định dạng nội dung trên trang web.

Nguyên tắc hoạt động: Sử dụng các thẻ (tags) để xác định và sắp xếp các phần tử.

Cấu trúc cơ bản:

<html>: Bọc toàn bộ tài liệu.

<head>: Chứa thông tin meta, tiêu đề (<title>).

<body>: Chứa toàn bộ nội dung hiển thị (văn bản, ảnh, liên kết...).

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>Tiêu đề trang</title>
</head>
<body>
  <h1>Đây là tiêu đề chính</h1>
  <p>Đây là một đoạn văn bản trong
HTML.</p>
  
  <a href="https://www.example.com">Đây là
một liên kết</a>
</body>
</html>
```



2.3. UI (User interface) và UX (User experience)



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.3.2. CSS (Cascading Style Sheets)

Định nghĩa: CSS (Cascading Style Sheets) là ngôn ngữ định kiểu dùng để mô tả cách trình bày và hiển thị của tài liệu HTML. Nếu HTML là "khung xương" (cấu trúc), thì CSS là "lớp da" (giao diện, thẩm mỹ).

Kiểm soát: Màu sắc, phong chữ, kích thước, khoảng cách (margin, padding), bố cục (layout) và hiệu ứng.

Bộ chọn (Selectors): Dùng để "chọn" phần tử HTML muốn định kiểu.

Thuộc tính & Giá trị (Properties & Values): Gán kiểu dáng cho phần tử đã chọn.

```
<p style="color: red;">Văn bản màu đỏ</p>
<style>
  h1 {
    color: green
    font-size: 24px;
  }
  p { color: blue; }
</style>
<link rel="stylesheet" href="style.css">
```



2.3. UI (User interface) và UX (User experience)



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.3.3. JavaScript (JS)

JavaScript (JS): Tương tác & Giao tiếp Server

Vai trò chính: Là ngôn ngữ lập trình giúp trang web trở nên tương tác và động. Thay đổi nội dung HTML (ví dụ: cập nhật số liệu cảm biến). Xử lý sự kiện người dùng (ví dụ: nhấn nút "Cho ăn").

Các phương thức HTTP chính (trong dự án):

+ GET: Dùng để lấy dữ liệu từ server (ví dụ: lấy dữ liệu cảm biến, danh sách lịch cho ăn).

+ POST: Dùng để gửi dữ liệu mới hoặc lệnh điều khiển lên server (ví dụ: thêm lịch mới, yêu cầu cho ăn ngay).

```
// Gửi dữ liệu đến server Flask qua HTTP
POST
fetch("/control/feed-now", {
  method: "POST",
  headers: { "Content-Type":
"application/json" },
  body: JSON.stringify({
    device_id: "esp32-feeder-1",
    amount: 30
  })
})
.then(response => response.json())
.then(data => console.log("Server
response:", data))
.catch(error => console.error("Error:",
error));
```




2.3. UI (User interface) và UX (User experience)



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.3.4. HTTP (Hypertext Transfer Protocol)

Định nghĩa: Là giao thức truyền tải siêu văn bản, đóng vai trò "cầu nối" cho việc trao đổi dữ liệu giữa Client (trình duyệt web, ứng dụng) và Server (máy chủ).

Mô hình hoạt động: Hoạt động theo cơ chế Yêu cầu – Phản hồi (Request – Response).

- + Client gửi một HTTP Request (Yêu cầu) đến Server.
- + Server xử lý và gửi lại một HTTP Response (Phản hồi) chứa dữ liệu hoặc thông tin trạng thái.

Ứng dụng trong AquaNova:

- + Là phương tiện giao tiếp chính giữa Giao diện Web (Front-end) và Máy chủ Flask (Back-end).
- + Luồng hoạt động:
 - Người dùng thao tác (ví dụ: nhấn nút "Cho ăn").
 - JavaScript gửi yêu cầu HTTP (POST/GET) đến API của Flask.
 - Flask xử lý, đồng bộ dữ liệu với Firebase Cloud Firestore.
 - Flask trả Response về giao diện web để cập nhật trạng thái.



2.3. UI (User interface) và UX (User experience)



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.3.4. HTTP (Hypertext Transfer Protocol)

Định nghĩa: Là giao thức truyền tải siêu văn bản, đóng vai trò "cầu nối" cho việc trao đổi dữ liệu giữa Client (trình duyệt web, ứng dụng) và Server (máy chủ).

Mô hình hoạt động: Hoạt động theo cơ chế Yêu cầu – Phản hồi (Request – Response).

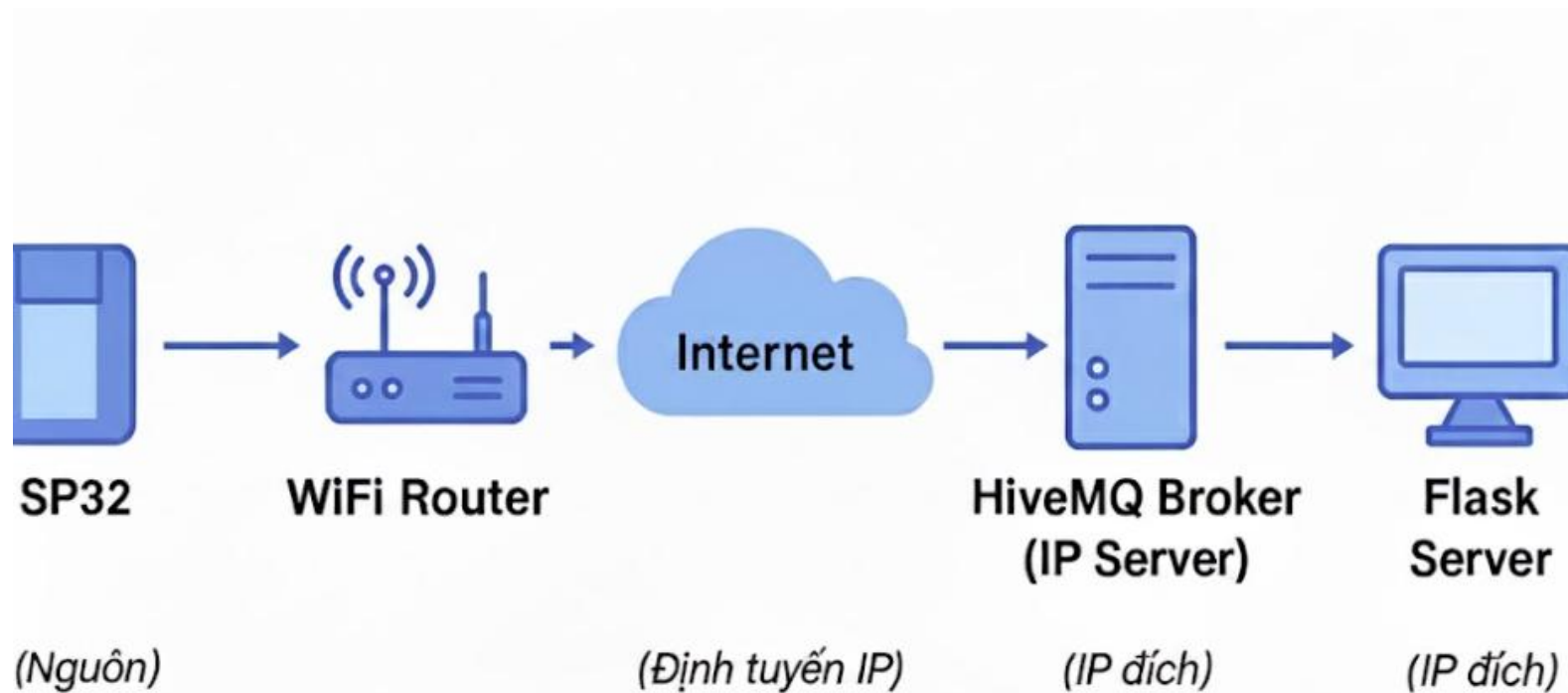
- + Client gửi một HTTP Request (Yêu cầu) đến Server.
- + Server xử lý và gửi lại một HTTP Response (Phản hồi) chứa dữ liệu hoặc thông tin trạng thái.

Ứng dụng trong AquaNova:

- + Là phương tiện giao tiếp chính giữa Giao diện Web (Front-end) và Máy chủ Flask (Back-end).
- + Luồng hoạt động:
 - Người dùng thao tác (ví dụ: nhấn nút "Cho ăn").
 - JavaScript gửi yêu cầu HTTP (POST/GET/DELETE) đến API của Flask.
 - Flask xử lý, đồng bộ dữ liệu với Firebase Cloud Firestore.
 - Flask trả Response về cho giao diện web để cập nhật trạng thái.

2.3. UI (User interface) và UX (User experience)

2.3.5. IP Address



Sơ đồ truyền gói tin qua giao thức IP trong mô hình AquaNova



2.3. UI (User interface) và UX (User experience)



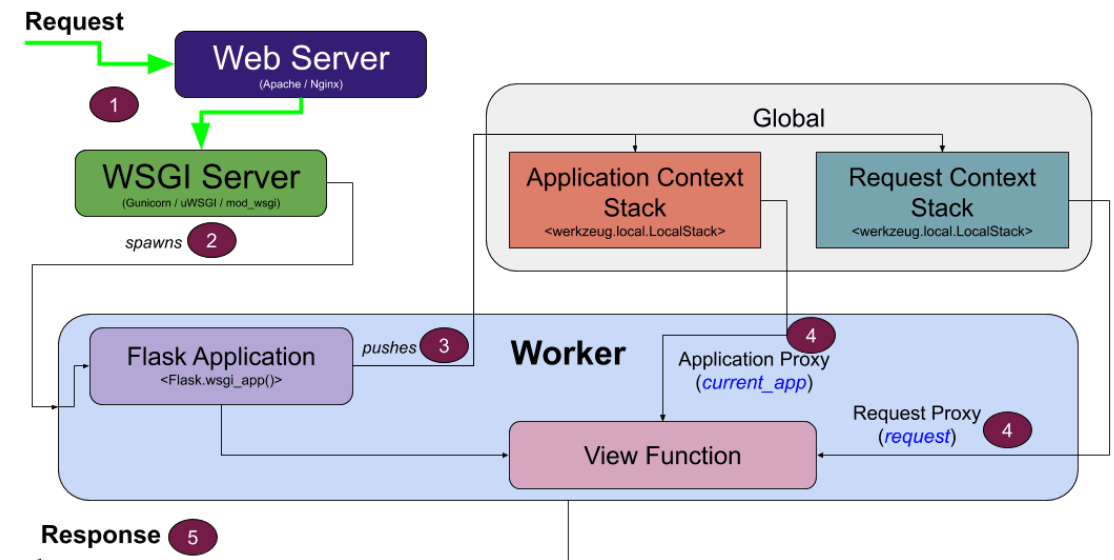
Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

2.3.6. Giao thức tầng giao vận TCP (Transmission Control Protocol)

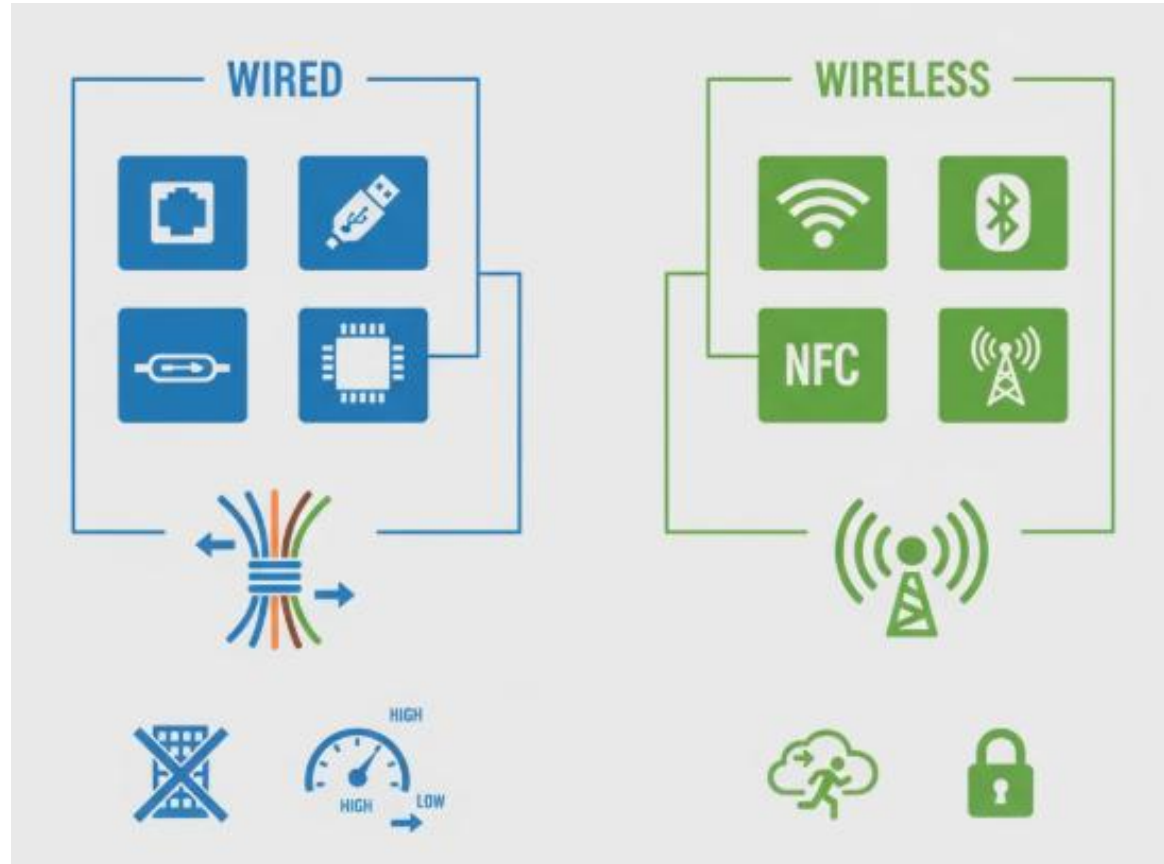
Giao thức	Tầng OSI	Vai trò trong hệ thống AquaNova	Ghi chú
IP	Tầng Mạng	Định tuyến gói tin từ ESP32 đến Broker và Flask	Không đảm bảo độ tin cậy
TCP	Tầng Giao vận	Đảm bảo truyền dữ liệu an toàn, có xác nhận, không mất gói	Dùng cho MQTT qua TLS

2.3.7. Flask (micro web framework for Python)

- Trong đề tài này, Flask đóng vai trò là Web Server:
 - Nhận dữ liệu từ MQTT và lưu trữ vào cơ sở dữ liệu (Firebase Firestore).
 - Cung cấp API để dashboard và ứng dụng web truy xuất dữ liệu cảm biến.
 - Xử lý lệnh điều khiển (bật/tắt, cho ăn, lên lịch) và gửi ngược lại cho thiết bị thông qua MQTT Broker.



2.4. Các chuẩn truyền thông



Minh họa sơ lược chuẩn truyền thông có dây và không dây



2.4. Các chuẩn truyền thông



2.5.1. Các chuẩn truyền thông không dây

- IEEE 802.11, hay còn gọi là WiFi, là chuẩn truyền thông không dây phổ biến trong mạng LAN, cho phép các thiết bị như máy tính, điện thoại, IoT kết nối Internet.
- Đặc điểm: tốc độ truyền cao, phạm vi phủ sóng rộng, hỗ trợ bảo mật (WEP, WPA, WPA2) và tương thích ngược với các phiên bản cũ





2.4. Các chuẩn truyền thông



2.5.1. Các chuẩn truyền thông không dây

1999: 802.11a

Tần số 5GHz, tốc độ 54Mbps, sử dụng
OFDM.

1

2

1999: 802.11b

Tần số 2.4GHz, tốc độ 11Mbps, sử dụng DSSS.

3

2003: 802.11g

Tần số 2.4GHz, tốc độ 54Mbps, sử dụng OFDM.

4

2009: 802.11n

Tần số 2.4/5GHz, tốc độ đến 600Mbps, tích hợp MIMO.

5

2013: 802.11ac

Tần số 5GHz, tốc độ hàng trăm–ngàn
Mbps, MIMO + OFDM.

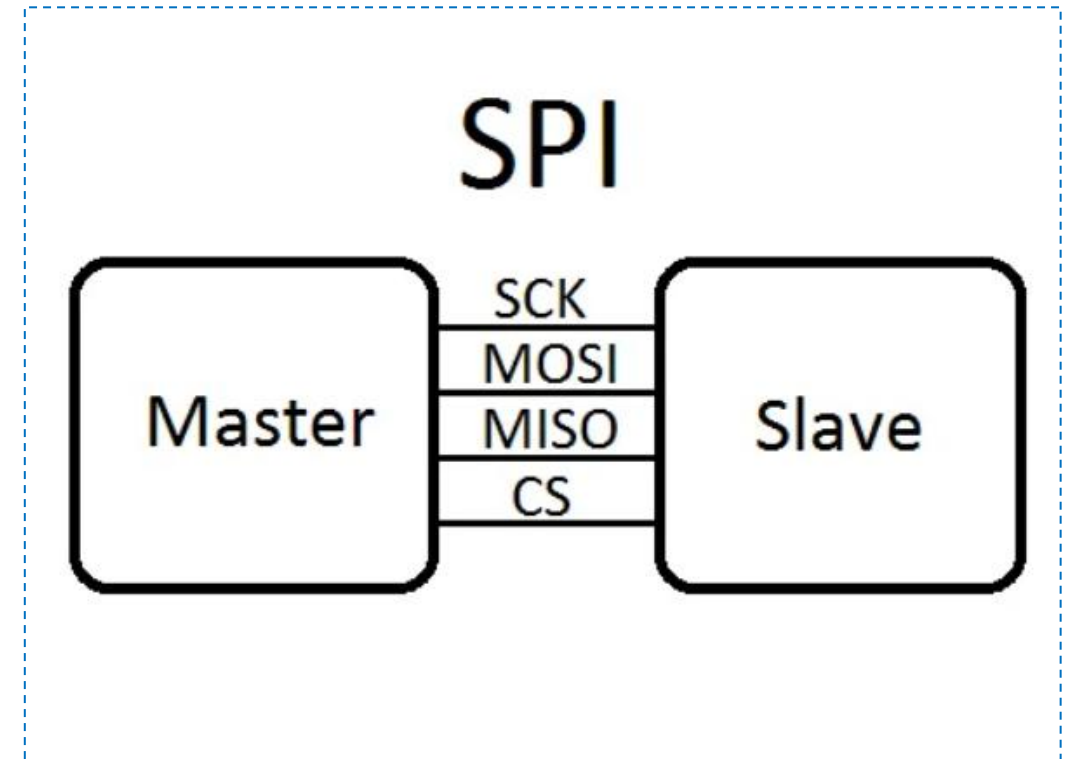
2.4. Các chuẩn truyền thông

2.5.2. Các chuẩn truyền thông có dây

SPI (Serial Peripheral Interface) là giao thức truyền đồng bộ do Motorola phát triển, hoạt động theo mô hình Master–Slave.

Sử dụng các tín hiệu: MOSI, MISO, SCK, SS.

- Tốc độ cao và truyền full-duplex.
- Cấu hình linh hoạt, dễ mở rộng.
- Tương thích tốt giữa nhiều thiết bị.

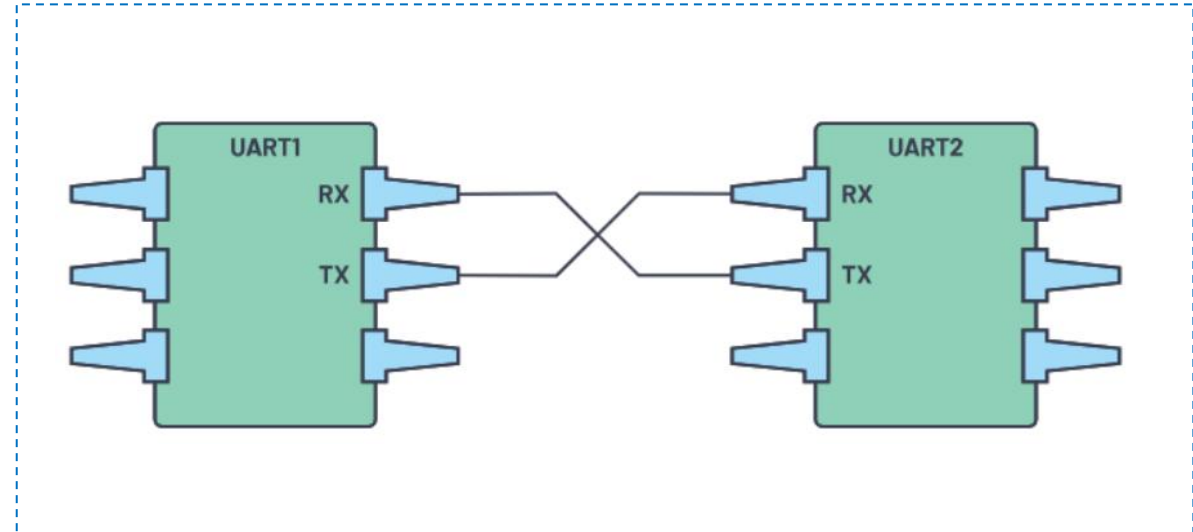


2.4. Các chuẩn truyền thông

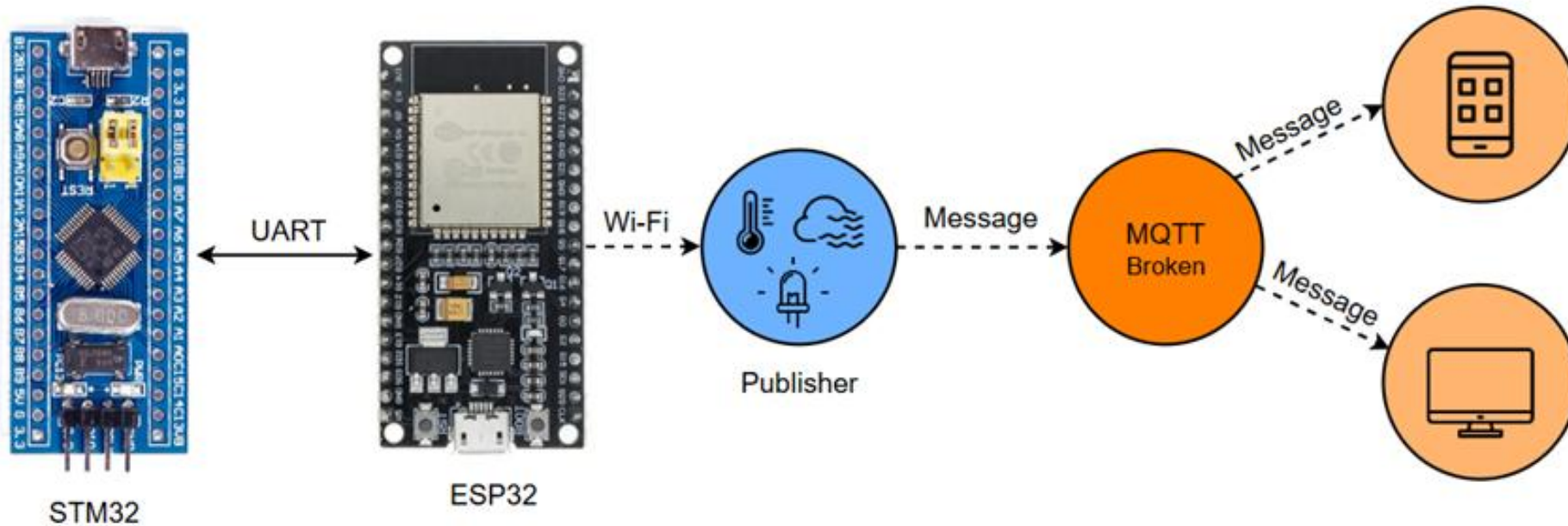
2.5.2. Các chuẩn truyền thông có dây

UART (Universal Asynchronous Receiver-Transmitter) là giao thức truyền bất đồng bộ, không cần xung đồng hồ chung.

- Sử dụng 2 tín hiệu chính: TX (truyền) và RX (nhận).
- Tốc độ được xác định bằng baud rate (9600, 115200,...).
- Đơn giản, phổ biến và dễ tích hợp trong các hệ thống nhúng.
- Đảm bảo tương thích ngược.



2.5. Giao thức MQTT trong hệ thống IoT



MQTT được chọn vì nhẹ, linh hoạt, đảm bảo độ tin cậy, dễ mở rộng và dễ tích hợp với cloud, rất phù hợp cho hệ thống giám sát môi trường nước và cho cá ăn tự động.



2.5. Gói tin MQTT

2.5.1. MQTT Connection:

- Thực tế dùng ESP8266 kết nối với STM32 lấy dữ liệu từ cảm biến.
- Thí nghiệm, các thiết bị được giả lập (MCU giả lập sensor)

Mục tiêu:

- Sinh ngẫu nhiên dữ liệu cảm biến: nhiệt độ, độ ẩm, mức thức ăn và client_id
- Đóng gói dữ liệu thành JSON.
- Publish lên MQTT Broker (HiveMQ Cloud).

Cấu hình kết nối:

```
MQTT_HOST=d2b2655505394c9b8e88c06c8d30d912.s1.eu.hivemq.cloud
```

```
MQTT_PORT=8883 (TLS)
```

```
MQTT_USER=aquanova_user
```

```
MQTT_PASS=12345678Aqua
```

```
MQTT_TOPIC=aquanova/devices/+/telemetry
```



HiveMQ Cloud

MQTT hoạt động dựa trên ba thành phần chính:

- Publisher: Thiết bị hoặc ứng dụng gửi dữ liệu.
- Subscriber: Thiết bị hoặc ứng dụng nhận dữ liệu.
- Broker (Máy chủ/Server): Thành phần trung gian, nhận thông điệp từ Publisher và chuyển tiếp đến các Subscriber đã đăng ký (subscribe) một Topic cụ thể. Broker giúp tách biệt bên gửi và bên nhận về thời gian và không gian.



2.5. Gói tin MQTT



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.5.2. MQTT Connection:

✓ The WebClient is connected

Connection Settings

Connect to your HiveMQ Cloud Cluster with your credentials. Do not worry you can quickly connect with autogenerated credentials.

Username * Password *

Disconnect

Topic Subscriptions 1

Unsubscribe from all topics

Subscribe to topics to receive messages from the HiveMQ cluster. You can also set the Quality of Service (QoS) for each topic. The higher the QoS, the more reliable the message delivery is. You can always subscribe to the (#) wildcard to receive all messages. Please note that the messages from internal probe topics are not displayed here.

TOPIC	QOS	ACTIONS
#	QoS: 0	

Messages 10

Filter icon Share icon Clear all messages

0

Topic: home/garden/fountain QoS: 0

OFF

1

Topic: aquanova/devices/sensor-1/telemetry QoS: 0

```
{"device_id": "sensor-1", "ts": "2025-10-01T07:20:22.825034+00:00", "temperature": 30.51, "turbidity": 31.48, "feed": 7.0}
```

2

Topic: aquanova/devices/sensor-2/telemetry QoS: 0

```
{"device_id": "sensor-2", "ts": "2025-10-01T07:20:22.970471+00:00", "temperature": 24.68, "turbidity": 39.05, "feed": 70.0}
```

3

Topic: aquanova/devices/sensor-3/telemetry QoS: 0

```
{"device_id": "sensor-3", "ts": "2025-10-01T07:20:23.292036+00:00", "temperature": 26.33, "turbidity": 41.58, "feed": 59.8}
```

4

Topic: aquanova/devices/sensor-1/telemetry QoS: 0

```
{"device_id": "sensor-1", "ts": "2025-10-01T07:20:27.827408+00:00", "temperature": 30.28, "turbidity": 71.29, "feed": 14.1}
```

5

Topic: aquanova/devices/sensor-2/telemetry QoS: 0

```
{"device_id": "sensor-2", "ts": "2025-10-01T07:20:27.973321+00:00", "temperature": 25.24, "turbidity": 78.0, "feed": 48.8}
```

6

Topic: aquanova/devices/sensor-3/telemetry QoS: 0

```
{"device_id": "sensor-3", "ts": "2025-10-01T07:20:28.294382+00:00", "temperature": 28.56, "turbidity": 14.33, "feed": 97.4}
```

7

Topic: aquanova/devices/sensor-1/telemetry QoS: 0

Giao diện HiveMQ Cloud Web Client các sensor (sensor-1, sensor-2, sensor-3) đã publish dữ liệu thành công.



2.5. Gói tin MQTT



2.5.3. MQTT Connection:

Gói tin	Mô tả	Cấu trúc (Header + Payload)
CONNECT	Client gửi đến Broker để yêu cầu thiết lập kết nối.	Fixed Header + Variable Header (Tên và phiên bản Protocol, cờ kết nối, Keep Alive) + Payload (Client ID, Username/Password, Last Will & Testament)
CONNACK	Broker phản hồi cho gói CONNECT, xác nhận kết nối thành công hay không	Fixed Header + Variable Header (cờ xác nhận, lý do)
DISCONNECT	Client gửi đến Broker để ngắt kết nối một cách an toàn.	Chỉ có Fixed Header
PINGREQ PINGRESP	Client gửi PINGREQ để giữ kết nối hoạt động khi không có dữ liệu trao đổi; Broker phản hồi bằng PINGRESP.	Chỉ có Fixed Header

2.5.4. MQTT Connection:

Luồng gói tin MQTT

- **CONNECT:** Client gửi đến Broker để yêu cầu thiết lập kết nối.
- **CONNACK:** Broker phản hồi cho gói CONNECT, xác nhận kết nối thành công hay thất bại.

```
[sensor-1] log: Sending CONNECT (u1, p1, wr0, wq0, wf0, c1, k60) client_id=b'sim-sensor-1'
[sensor-1] log: Received CONNACK (0, 0)
[sensor-1] Connected with result code Success
```

- u1, p1: username & password
- c1 : cleanSession = 1 | wq0: mức QoS
- k60: keepAlive = 60 giây
- client_id= 'sim-sensor-1': định danh client
- wf0: khai báo Last Will
- wr0: cờ retain Last Will message.

Sau đó broker trả về **CONNACK (0,0)** → thành công.

Mức Chất lượng Dịch vụ (Quality of Service - QoS)

QoS 0: At most once (Tối đa một lần)

- Mess được gửi một lần, không có xác nhận (ACK).

=> Truyền tải nhanh nhất nhưng có thể bị mất.

- Cập nhật liên tục nhiệt độ, độ đục mỗi giây/giờ

QoS 1: At least once (Tối thiểu một lần)

- Mess được đảm bảo đến đích ít nhất một lần. Bên nhận sẽ gửi gói PUBACK xác nhận.

=> Mess có thể bị trùng lặp (nếu bên gửi không nhận được PUBACK và gửi lại).

- Trạng thái cho ăn và lượng thức ăn

QoS 2: Exactly once (Duy nhất một lần)

- Mess được đảm bảo đến đích đúng một lần. Sử dụng giao thức bắt tay bốn bước (handshake)

=> Đáng tin cậy nhất nhưng tốn nhiều thời gian nhất.

- Tín hiệu, điều khiển đặt lịch cho ăn



2.5. Gói tin MQTT



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.5.5. MQTT Publisher

Luồng gói tin MQTT

- PUBLISH: Thiết bị gửi dữ liệu cảm biến (telemetry) lên topic định sẵn.

```
[sensor-1] log: Sending PUBLISH (d0, q1, r0, m1),  
      'b'aquanova/devices/sensor-1/telemetry'', ...  
(123 bytes)  
[sensor-1] -> publishing {'device_id': 'sensor-1',  
      'ts': '2025-09-30T10:01:05.944232+00:00',  
      'temperature': 30.53, 'turbidity': 48.55,  
      'feed': 51.6}
```

d0: dupFlag = 0 | r0: retain = 0

q1: QoS 1 (phải có PUBACK xác nhận)

m1: messageId = 1

Topic: aquanova/devices/sensor-1/telemetry

Payload: JSON chứa thông số sensor

- PUBACK: Broker xác nhận đã nhận dữ liệu (QoS=1).
Broker trả PUBACK (mid=1) → nhận gói tin thành công

```
2025-09-30T17:14:44.996069 [sensor-1] log: Received  
PUBACK (Mid: 1)  
2025-09-30T17:14:44.996898 [sensor-1] Published  
mid=1, reason=Success
```

- DISCONNECT: Client chủ động ngắt kết nối.

```
2025-09-30T17:15:34.852617 [sensor-1] log: Sending  
DISCONNECT  
2025-09-30T17:15:34.855632 [sensor-1] log: ...  
Disconnected, reason=0
```



2.5. Gói tin MQTT



2.5.5. MQTT Publisher:

Gói tin	Mô tả	Cấu trúc (Header + Payload)
PUBLISH	Client (hoặc Broker) gửi thông điệp ứng dụng đến Topic.	Fixed Header (QoS, DUP, RETAIN) + Variable Header (Tên Topic, Packet Identifier) + Payload (Dữ liệu thực tế)
PUBACK	Phản hồi cho PUBLISH QoS 1, xác nhận đã nhận thông điệp.	Fixed Header + Variable Header (Packet Identifier, Mã lý do)
PUBREC, PUBREL, PUBCOMP	Các gói tin bắt tay (handshake) để đảm bảo giao tiếp QoS 2 (Exactly once):	
	- PUBREC (Received): Broker xác nhận đã nhận PUBLISH.	
	- PUBREL (Release): Client xác nhận đã sẵn sàng hoàn tất.	
	- PUBCOMP (Complete): Broker xác nhận đã hoàn tất giao dịch.	



2.5. Gói tin MQTT



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

2.5.5. MQTT Publisher

Luồng gói tin MQTT

- PUBLISH: Thiết bị gửi dữ liệu cảm biến (telemetry) lên topic định sẵn.

```
[sensor-1] log: Sending PUBLISH (d0, q1, r0, m1),  
      'b'aquanova/devices/sensor-1/telemetry'', ...  
(123 bytes)  
[sensor-1] -> publishing {'device_id': 'sensor-1',  
      'ts': '2025-09-30T10:01:05.944232+00:00',  
      'temperature': 30.53, 'turbidity': 48.55,  
      'feed': 51.6}
```

d0: dupFlag = 0 | r0: retain = 0

q1: QoS 1 (phải có PUBACK xác nhận)

m1: messageId = 1

Topic: aquanova/devices/sensor-1/telemetry

Payload: JSON chứa thông số sensor

- PUBACK: Broker xác nhận đã nhận dữ liệu (QoS=1).
Broker trả PUBACK (mid=1) → nhận gói tin thành công

```
2025-09-30T17:14:44.996069 [sensor-1] log: Received  
PUBACK (Mid: 1)  
2025-09-30T17:14:44.996898 [sensor-1] Published  
mid=1, reason=Success
```

- DISCONNECT: Client chủ động ngắt kết nối.

```
2025-09-30T17:15:34.852617 [sensor-1] log: Sending  
DISCONNECT  
2025-09-30T17:15:34.855632 [sensor-1] log: ...  
Disconnected, reason=0
```




2.5. Gói tin MQTT

2.5.6. MQTT Subscriber

- SUBSCRIBE → client gửi yêu cầu đăng ký topic.

```
2025-10-01T17:03:37.123756 [MQTT] LISTENING
d2b265...hivemq.cloud:8883
topic=aquanova/devices/+/telemetry
2025-10-01T17:03:37.389492 [MQTT] CONNECTED with
code=Success
2025-10-01T17:03:37.390013 [MQTT] SUBSCRIBE sent
topic=aquanova/devices/+/telemetry mid=1 result=0
```

Subscriber gửi gói CONNECT và broker phản hồi CONNACK với code Success, xác nhận kết nối MQTT

- topic: aquanova/devices/+/telemetry
- mid=1: ID duy nhất của gói đăng ký
- result=0: gửi thành công về phía client library

- SUBACK → broker xác nhận quyền đăng ký với QoS

```
2025-10-01T17:03:37.601655 [MQTT] SUBACK received
mid=1, granted_qos=[ReasonCode(Suback, 'Granted QoS
1')] ]
```

Luồng kết nối MQTT

- CONNECT → client gửi yêu cầu kết nối.
- CONNACK → broker xác nhận kết nối thành công
- SUBSCRIBE → client gửi yêu cầu đăng ký topic.
- SUBACK → broker xác nhận quyền đăng ký với QoS



2.5. Gói tin MQTT



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.5.6. MQTT Subscriber

Thành phần	Subscribe topic	Ý nghĩa
Server/WebApp	aquanova/telemetry	Nhận dữ liệu từ tất cả sensor
MCU Feeder	aquanova/feeder/	Nhận lệnh cho ăn
MCU Light	aquanova/light	Nhận lệnh bật/tắt đèn
App	aquanova/devices/+/status, .../alert	Nhận trạng thái & cảnh báo
AI Engine (future)	aquanova/devices/+/telemetry	Phân tích dữ liệu môi trường, đưa ra hành động

Bảng tổng hợp Subscriber dự kiến có trong hệ thống



2.5. Gói tin MQTT



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

2.5.6. MQTT Subscriber

Gói tin	Mô tả	Cấu trúc (Header + Payload)
SUBSCRIBE	Client gửi đến Broker để đăng ký nhận thông điệp từ một hoặc nhiều Topic.	Fixed Header + Variable Header (Packet Identifier) + Payload (Danh sách Topic và mức QoS yêu cầu cho từng Topic)
SUBACK	Broker phản hồi cho gói SUBSCRIBE, xác nhận các đăng ký và mức QoS tối đa được cấp phép.	Fixed Header + Variable Header (Packet Identifier) + Payload (Danh sách Mã lý do/Mức QoS được cấp phép)
UNSUBSCRIBE	Client gửi để hủy đăng ký khỏi một hoặc nhiều Topic.	Fixed Header + Variable Header (Packet Identifier) + Payload (Danh sách Topic)
UNSUBACK	Broker phản hồi cho gói UNSUBSCRIBE, xác nhận việc hủy đăng ký.	Fixed Header + Variable Header (Packet Identifier, Mã lý do)



2.6. Thiết bị phần cứng



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

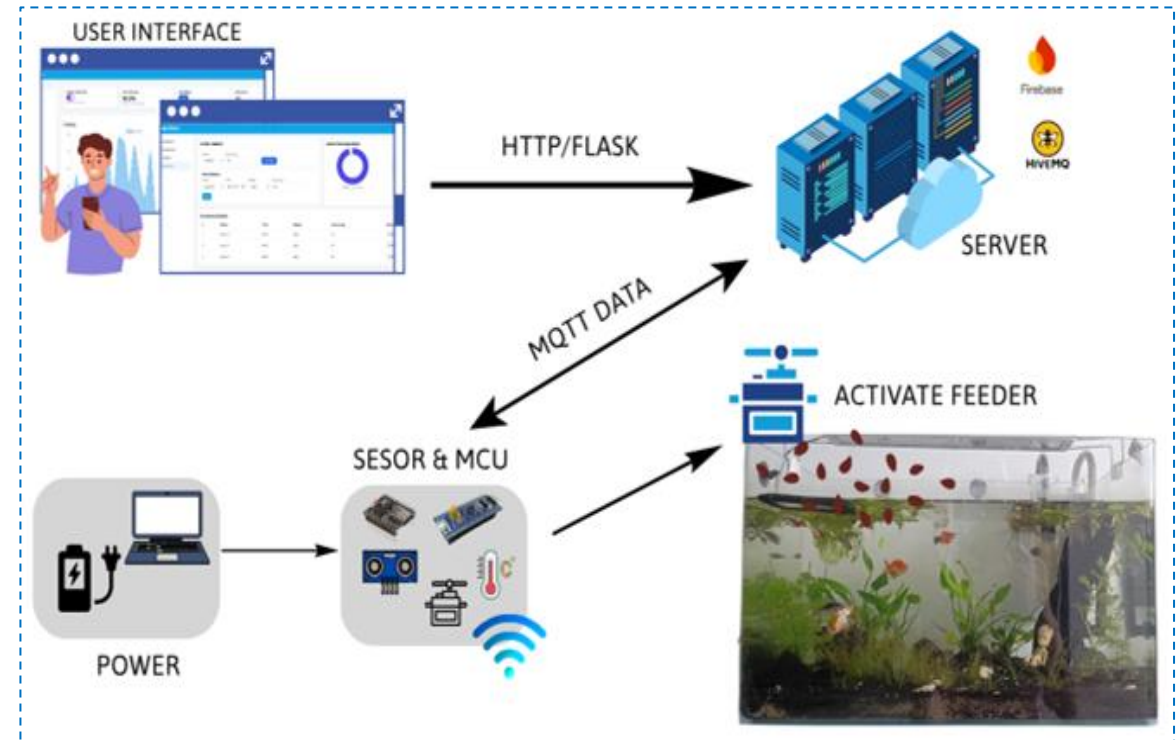
Tên linh kiện	Chức năng chính
STM32F103C8T6	Vi điều khiển, bộ não xử lý trung tâm của mạch.
ESP23-C3 mini	Module Wi-Fi, giúp kết nối Internet để gửi/nhận dữ liệu.
Cảm biến độ đục	Đo độ trong hay đục của chất lỏng (thường là nước).
Cảm biến siêu âm	Đo khoảng cách đến vật cản bằng sóng siêu âm.
Cảm biến nhiệt độ	Đo nhiệt độ của môi trường (chất lỏng).
Led	Phát ra ánh sáng, dùng để hiển thị trạng thái hoặc trang trí.
Động cơ Servo	Động cơ quay và giữ chính xác một góc theo lệnh.
Relay (Rơ-le)	Công tắc điện từ, dùng tín hiệu nhỏ để bật/tắt thiết bị điện lớn.



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

CHƯƠNG 3: THIẾT KẾ VÀ THI CÔNG MÔ HÌNH

- 3.1. Yêu cầu về tính năng của mô hình
- 3.2. Yêu cầu về kỹ thuật
- 3.3. Mô hình tổng quan hệ thống
- 3.4. Thiết kế kiến trúc của mô hình
- 3.5. Thiết kế cơ khí
- 3.6. Thiết kế nhúng
- 3.7. Thiết kế website
- 3.8. Hiệu chuẩn và kiểm thử





3.1. Yêu cầu về tính năng của mô hình



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

Chức năng hệ thống	Mô tả yêu cầu	Linh kiện sử dụng
Đo nhiệt độ nước	Cảm biến chính xác, chống nước, sai số nhỏ $\pm 0.5\text{ }^{\circ}\text{C}$	Cảm biến DS18B20
Đo độ đục của nước	Đo độ trong/đục để đánh giá chất lượng nước	Turbidity sensor (MJKDZ module)
Đo lượng thức ăn	Xác định lượng thức ăn	HC-SR04 Ultrasonic sensor
Xử lý và hiển thị dữ liệu	Đọc giá trị cảm biến, hiển thị LCD, cảnh báo khi vượt ngưỡng	STM32F103C8, LCD I2C 16×2, Buzzer



3.1. Yêu cầu về tính năng của mô hình



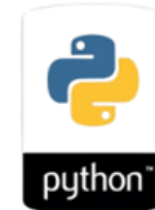
Chức năng hệ thống	Mô tả yêu cầu	Linh kiện sử dụng
Kết nối Internet và truyền dữ liệu	Gửi dữ liệu cảm biến lên server qua MQTT, nhận lệnh điều khiển	ESP32-C3 Mini, Wi-Fi module tích hợp
Đồng bộ thời gian	Lưu và hiển thị giờ thực, hỗ trợ hẹn lịch cho ăn	RTC DS3232 module
Điều khiển cơ cấu cho ăn	Quay servo định lượng thức ăn theo lịch hoặc lệnh tay	Servo motor SG90
Cấp nguồn cho hệ thống	Cung cấp điện ổn định 5 V–3.3 V, có thể dùng pin hoặc adapter	Nguồn pin Li-ion + Module sạc/ổn áp



3.2. Yêu cầu về kỹ thuật



Visual Studio Code



Flask



Inventor



Phần mềm sử dụng để xây
dựng AquaNova



3.2. Yêu cầu về kỹ thuật

3.2.1. Chức năng kỹ thuật

- Đọc dữ liệu từ cảm biến
- Gửi yêu cầu cho ăn, bật tắt đèn tự động
- Hẹn giờ, đặt lịch cho ăn; xóa lịch cho ăn.
- Cho biết phần trăm lượng thức ăn còn lại thông qua cảm biến siêu âm.
- Hiển thị thông tin nhiệt độ, độ đục, đặt lịch,.. qua màn hình LCD.
- Cảnh báo khi độ đục nước vượt ngưỡng cho phép.



3.2. Yêu cầu về kỹ thuật



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

3.2.1. Thông số kỹ thuật

Thông số kỹ thuật	Mô tả
Vi điều khiển	ESP32
Nguồn hoạt động	5V DC
Cảm biến	DS18B20 (nhiệt độ), cảm biến độ đục, cảm biến siêu âm HC-SR04
Màn hình hiển thị	LCD 16x2
Kết nối	Wi-Fi (MQTT/Web Server)



3.2. Yêu cầu về kỹ thuật

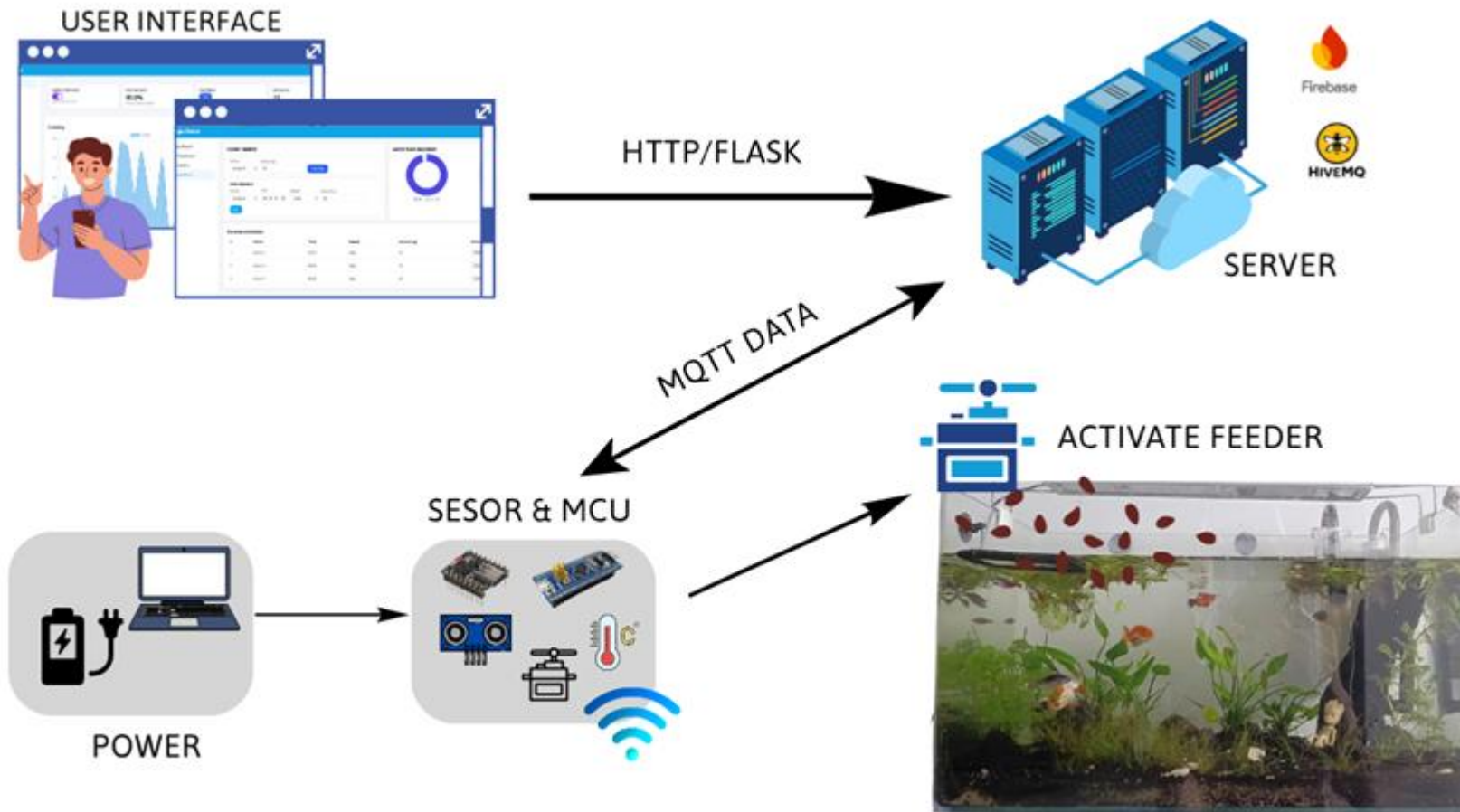


Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

3.2.1. Thông số kỹ thuật

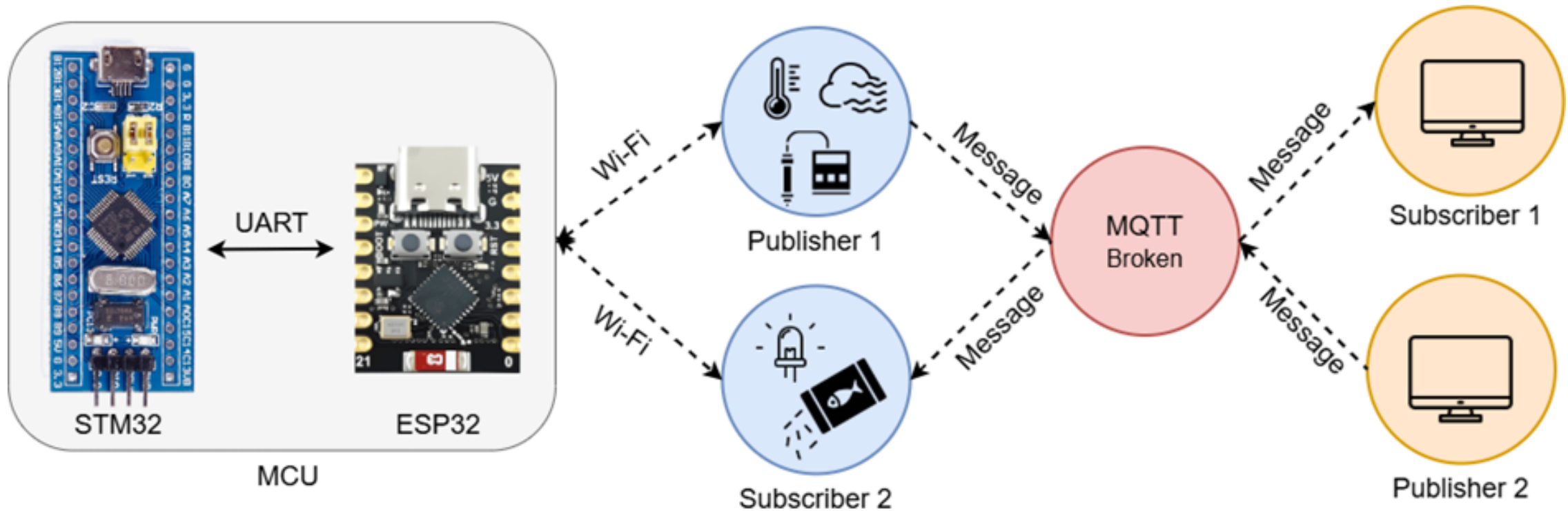
Thông số kỹ thuật	Mô tả
Tần suất cập nhật dữ liệu	30 giây/lần
Nguồn cấp cho động cơ/relay	5V–12V DC
Kích thước mô hình	10cm × 15cm
Thời gian khởi động ổn định	10 giây

3.3. Mô hình tổng quan hệ thống



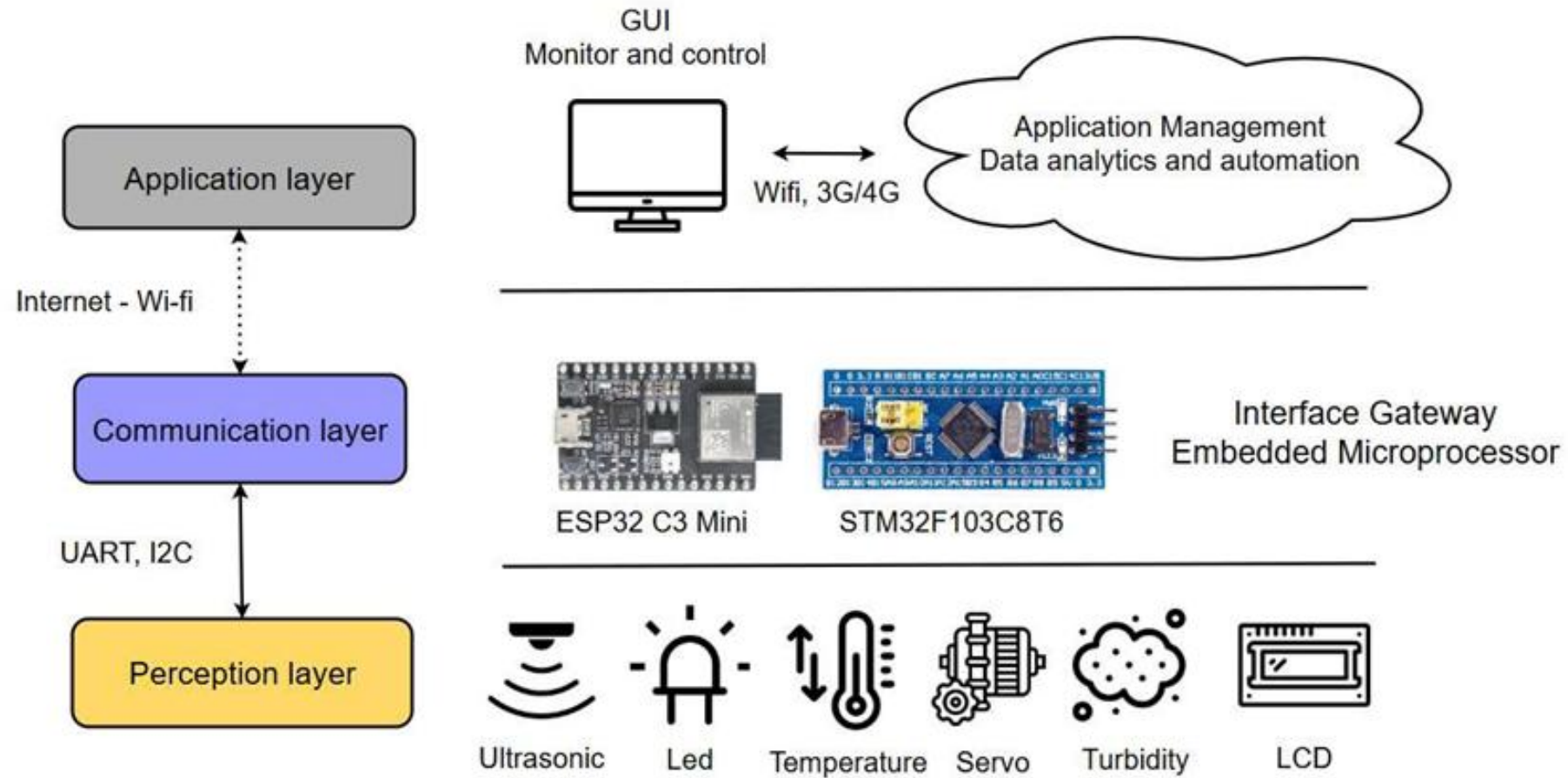
Hình 17: Tổng quan mô hình AquaNova

3.3. Mô hình tổng quan hệ thống



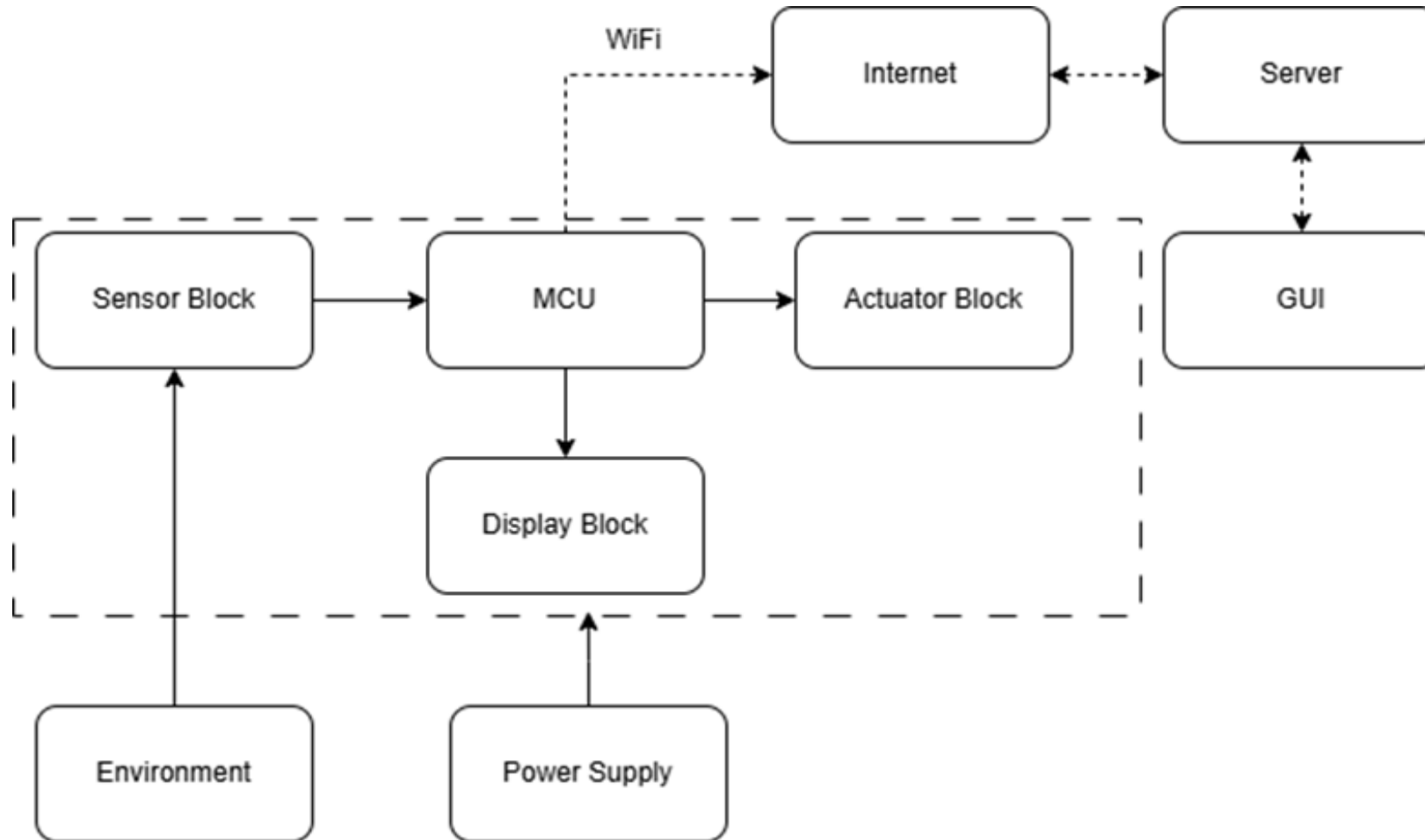
Hình 18: Sơ đồ sử dụng MQTT vào mô hình AquaNova

3.3. Mô hình tổng quan hệ thống



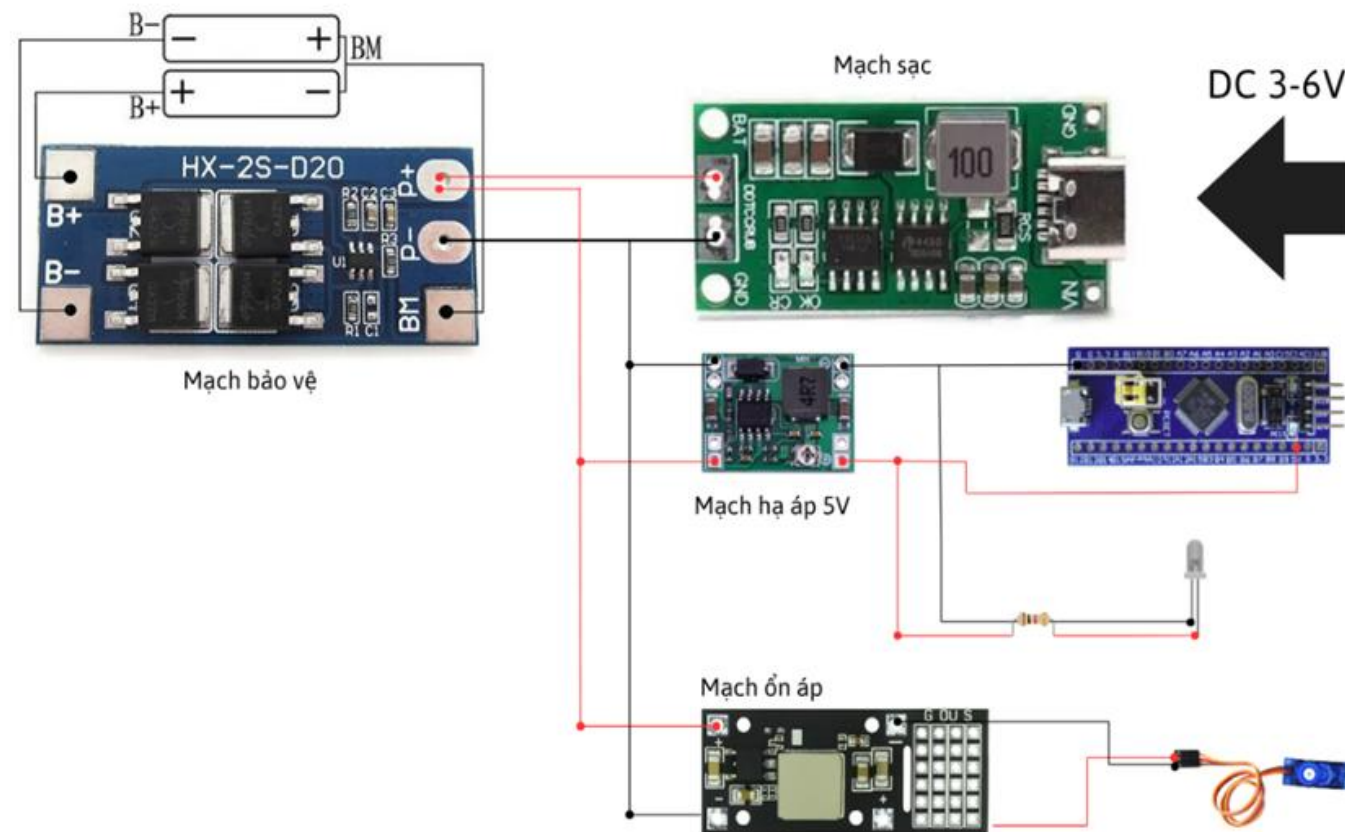
Hình 19: Kiến trúc tổng quan của hệ thống IoT 3 lớp của mô hình AquaNova

3.4. Thiết kế kiến trúc của mô hình



Hình 20: Sơ đồ khối của mô hình AquaNova

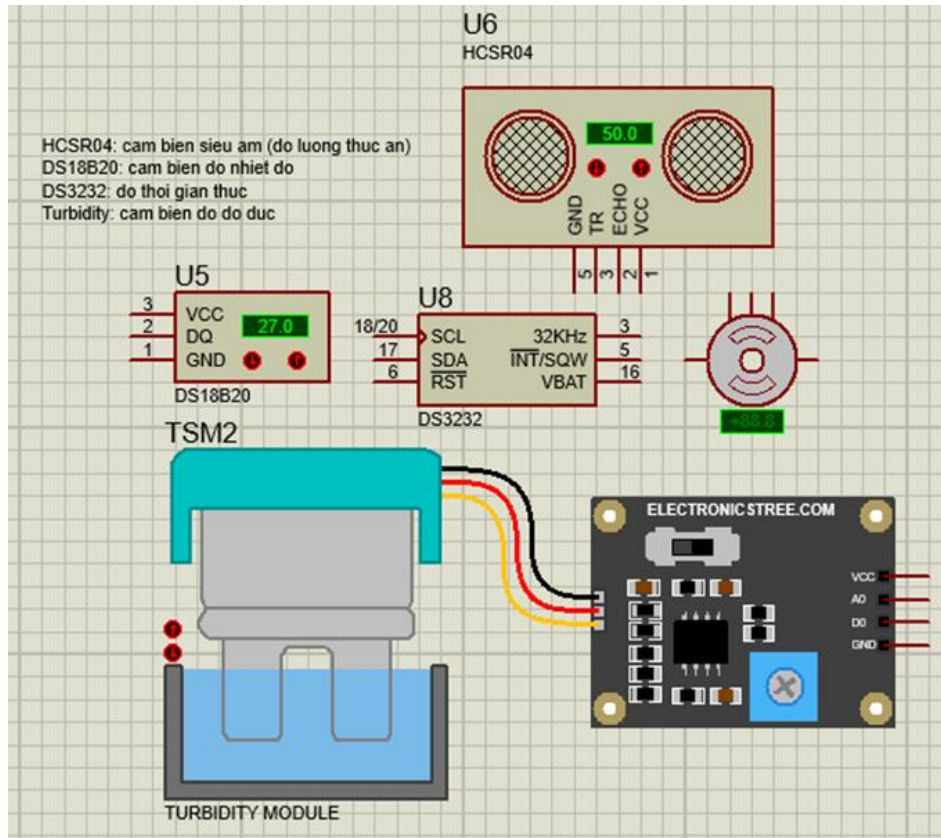
3.4.1. Khối nguồn



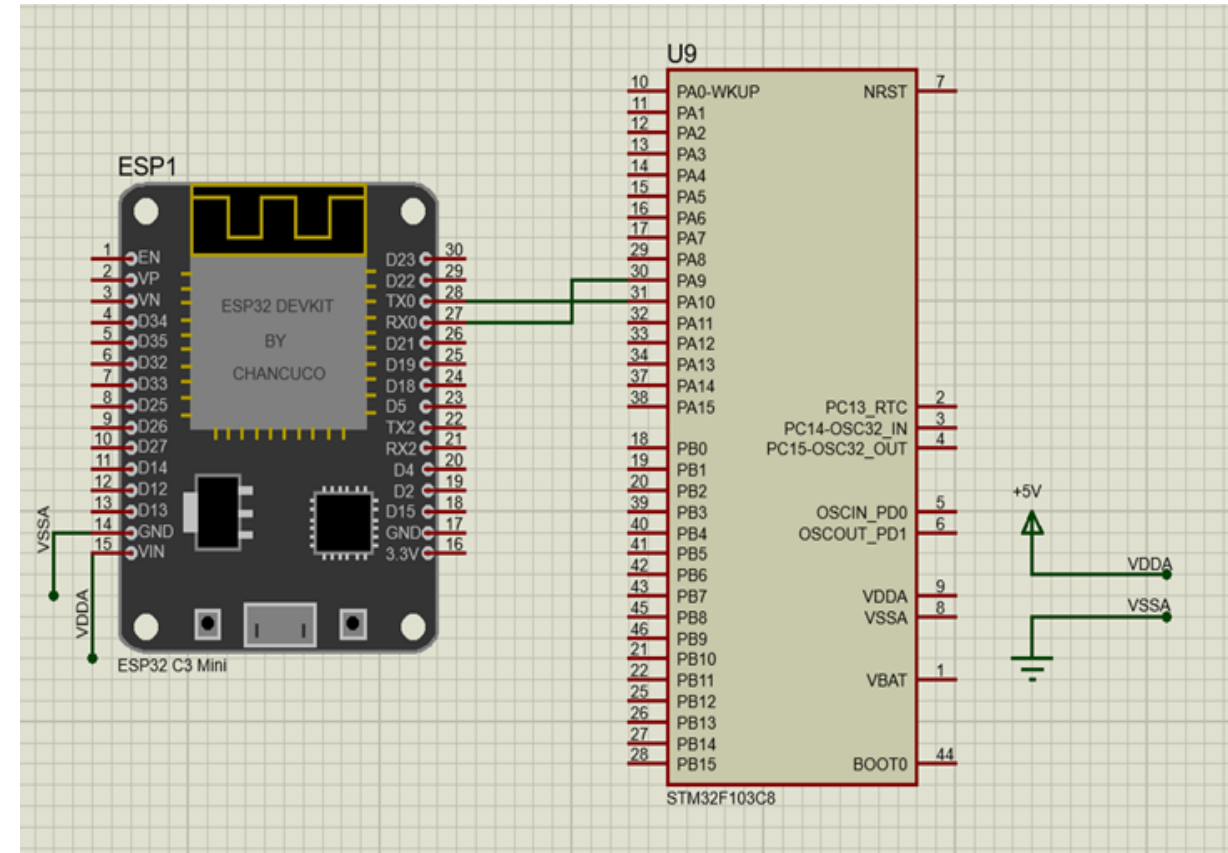
Khối cấp nguồn của mô hình AquaNova

3.4. Thiết kế kiến trúc của mô hình

3.4.2. Khối cảm biến



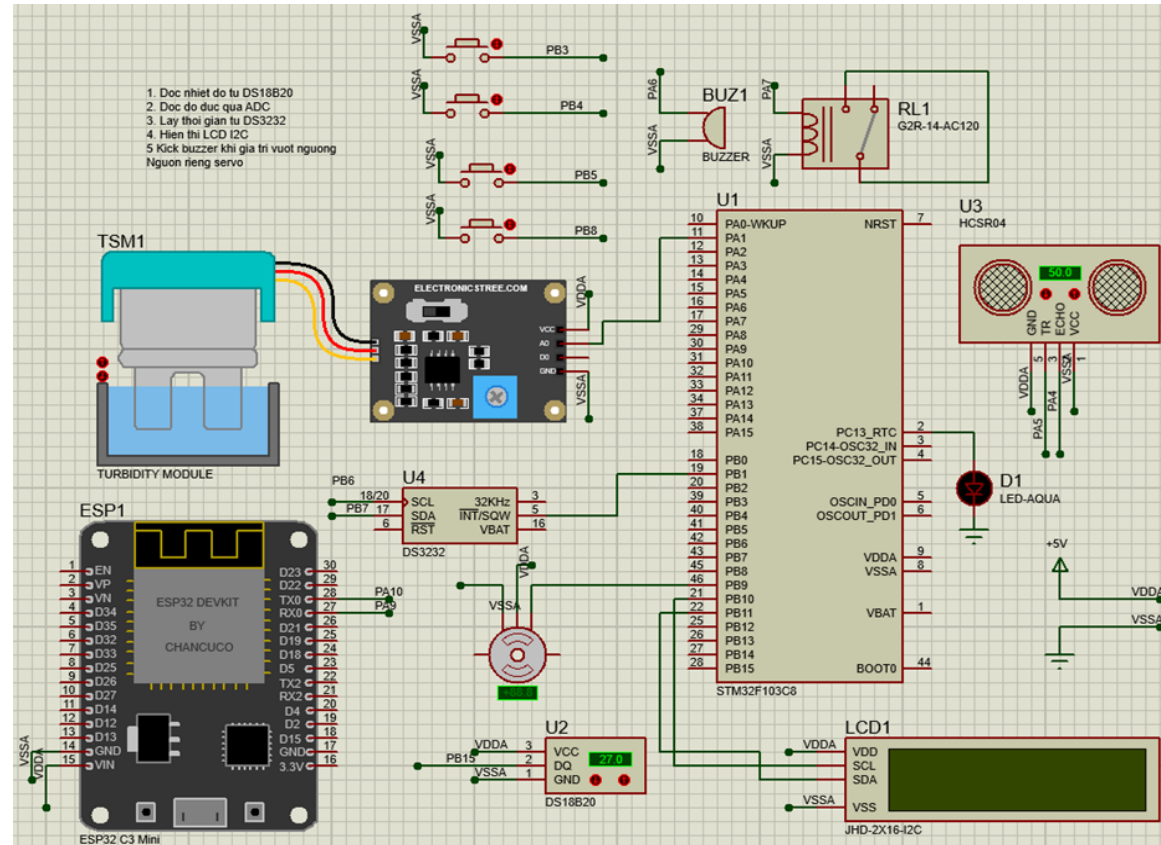
Sơ đồ tổng quan các cảm biến trong mô hình



Sơ đồ UART giữa vi điều khiển STM32 và ESP32 trong mô hình AquaNova

3.4. Thiết kế kiến trúc của mô hình

3.4.4. Kết nối toàn bộ hệ thống



Sơ đồ nguyên lý mô hình IoT AquaNova

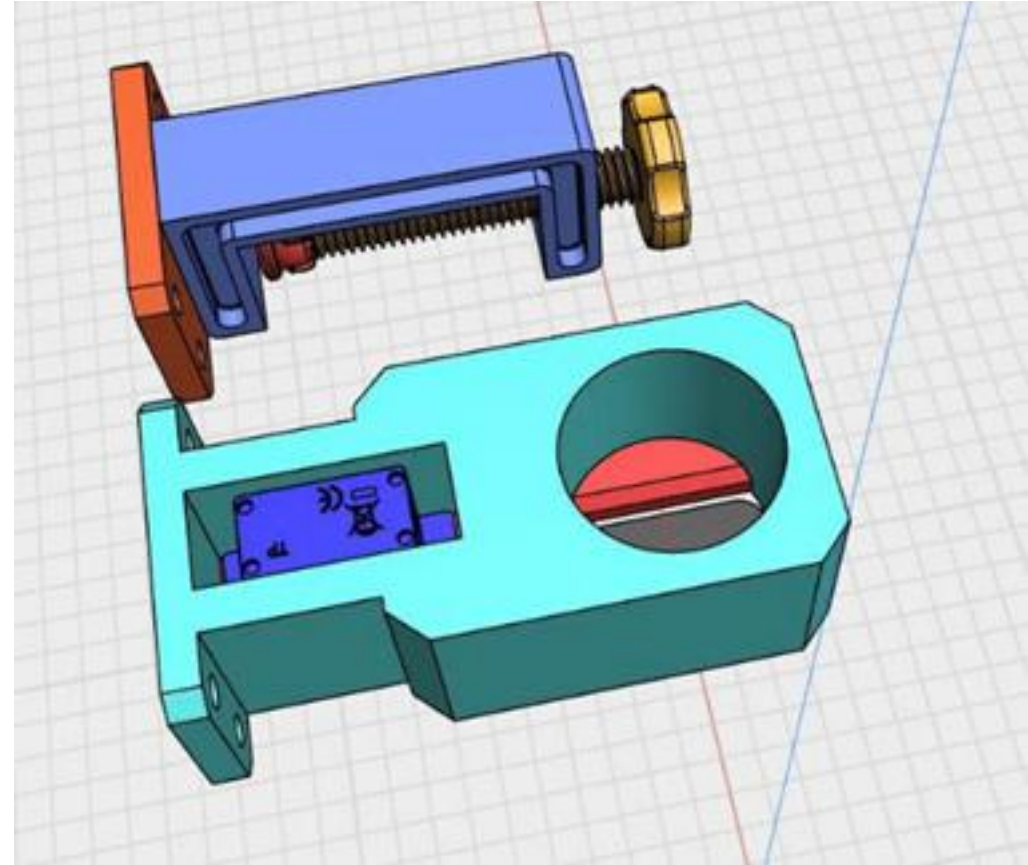
3.5.1. Cài đặt phần mềm



Inventor

3.5.2. Chi tiết thiết kế

- Phễu chứa (Hopper)
- Bộ đỡ (Giá đỡ in 3D)
- Tấm gạt (Van chặn)



3.5. Thiết kế cơ khí

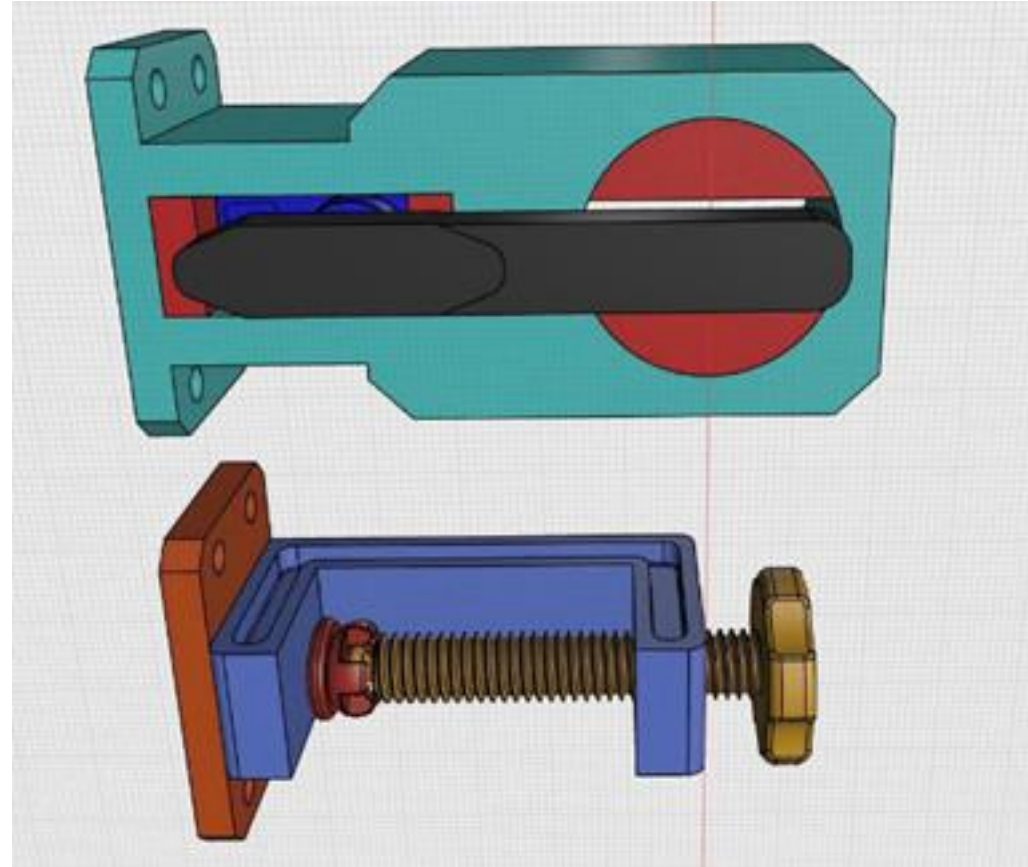
3.5.1. Cài đặt phần mềm



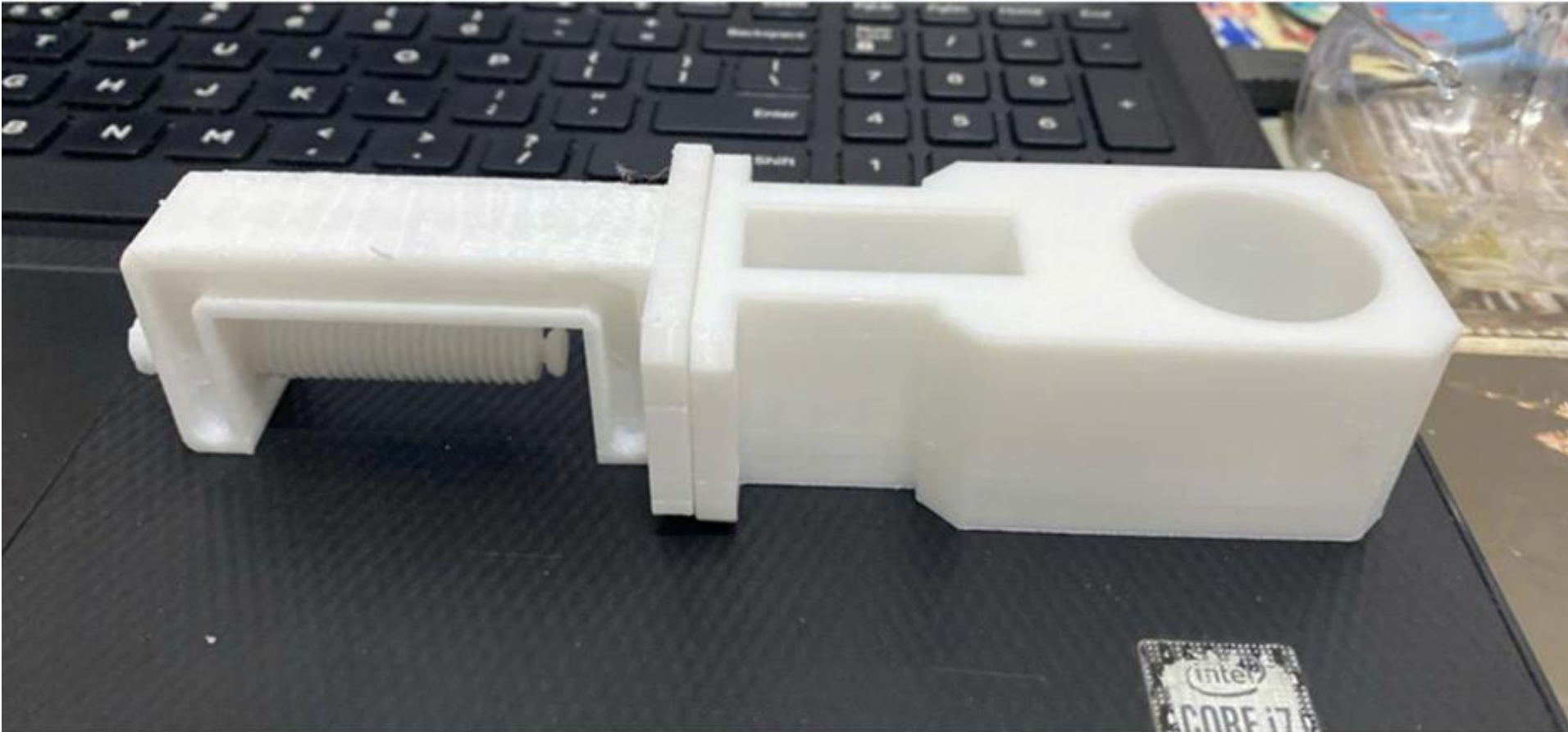
Inventor

3.5.2. Chi tiết thiết kế

- Phễu chứa (Hopper)
- Bộ đỡ (Giá đỡ in 3D)
- Tấm gạt (Van chặn)



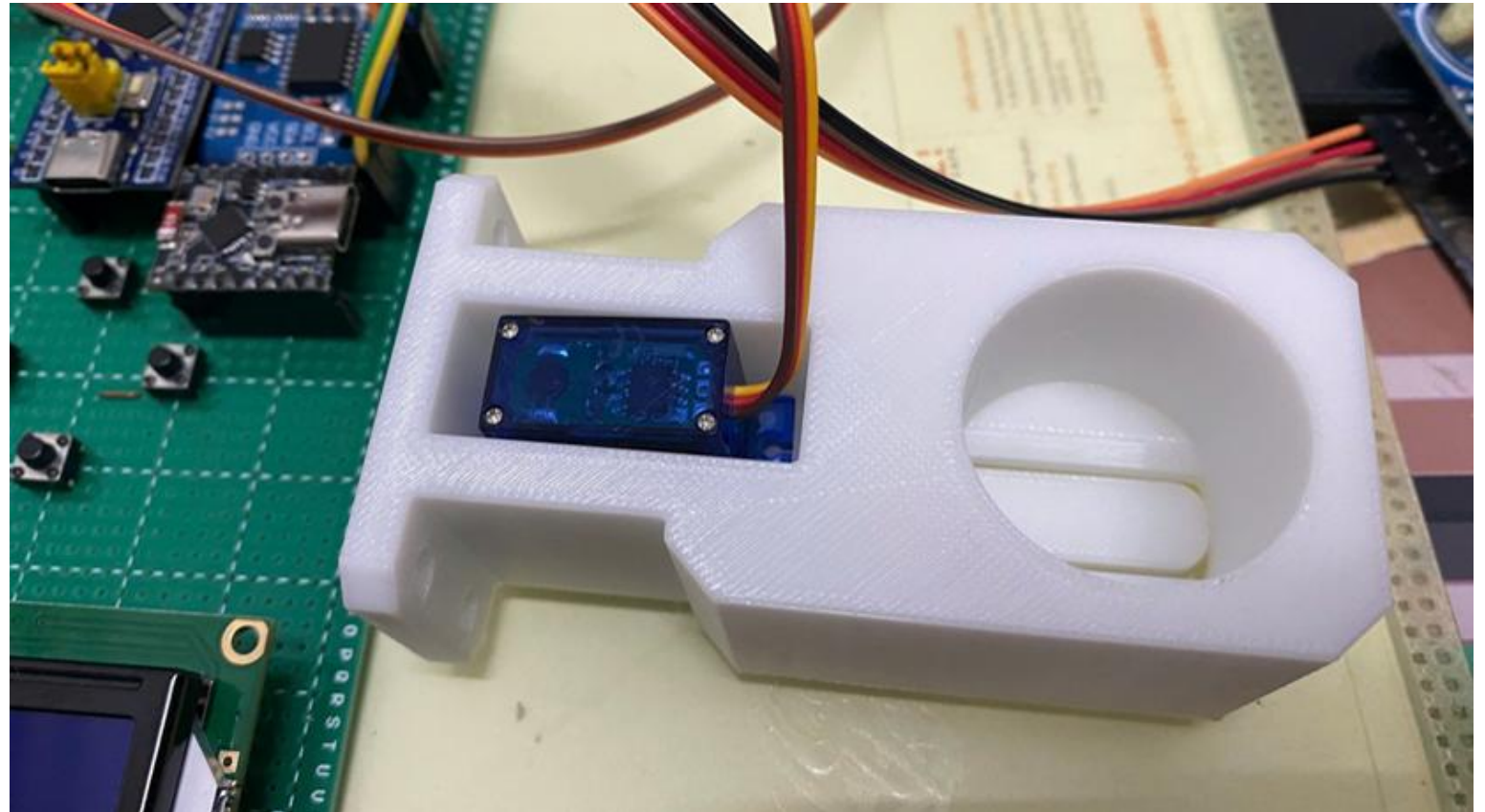
3.5.2. Chi tiết thiết kế



Hình 29: Giá đỡ phân giữ thức ăn và servo

3.5.3. Lắp ráp sản phẩm

Lắp servo vào bộ đồ
Gắn tấm gạt
Cố định chai chứa
Kiểm tra cơ cấu

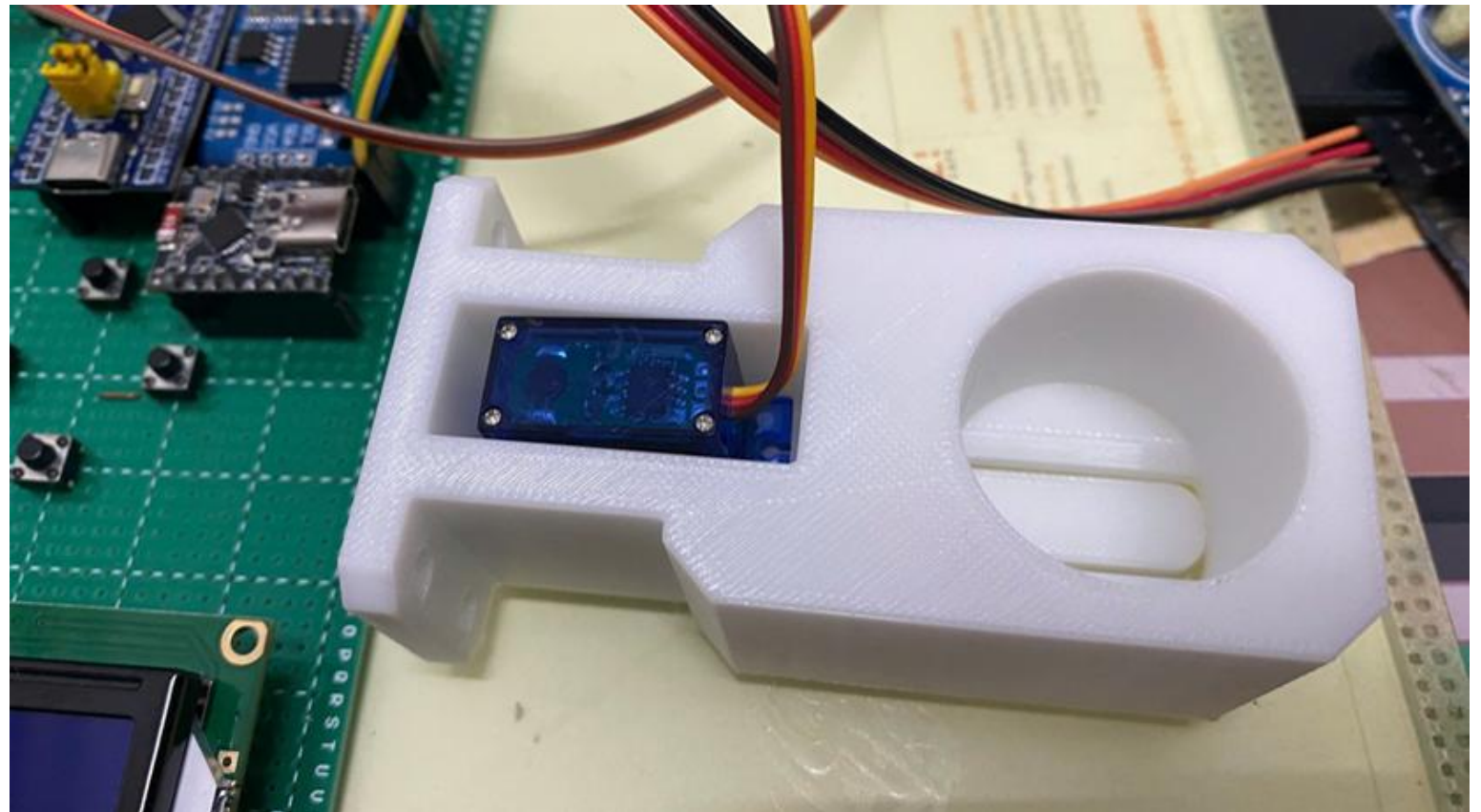


3.5. Thiết kế cơ khí

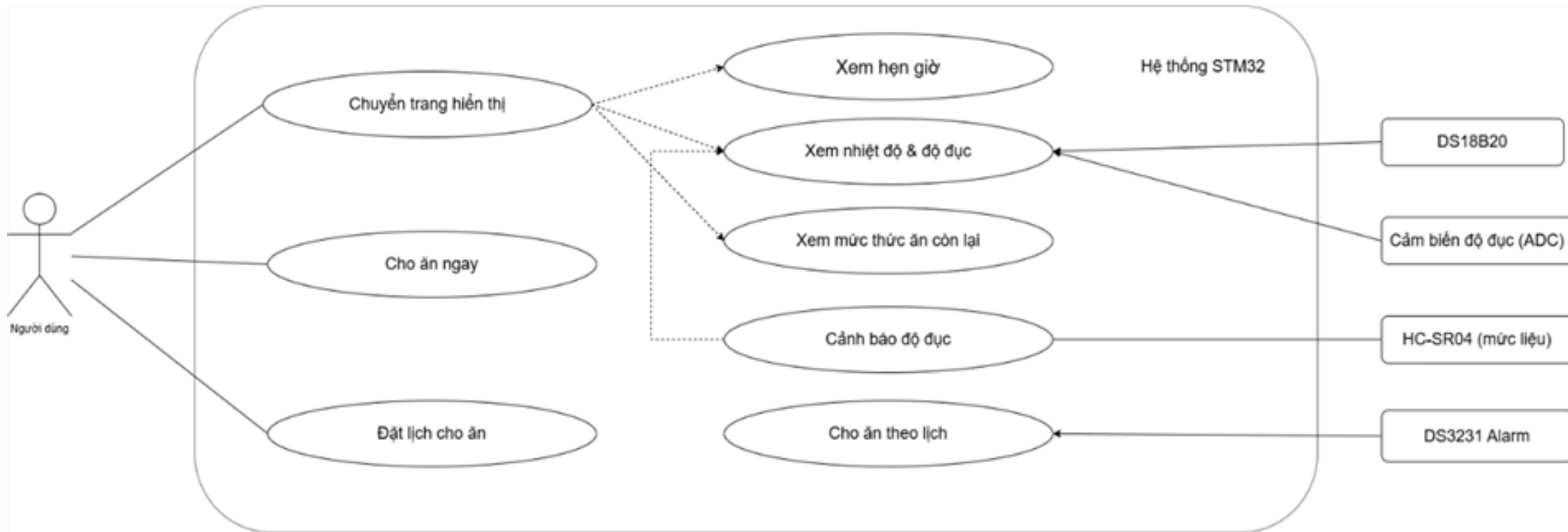
3.5.3. Lắp ráp sản phẩm



Hình ảnh cơ khí trong thực tế



3.6. Thiết kế nhúng



Use Case các tính năng của mô hình AquaNova



3.6. Thiết kế nhúng

3.6.1. Cài đặt phần mềm và cấu hình

1. Thiết lập cơ bản hệ thống

- Vào mục System Core → SYS, tại phần Debug chọn Serial Wire để kích hoạt chức năng nạp và gỡ lỗi thông qua cổng SWD.
- Trong mục Timebase Source, chọn SysTick để sử dụng bộ đếm hệ thống làm nguồn định thời chính cho các tác vụ trong chương trình.

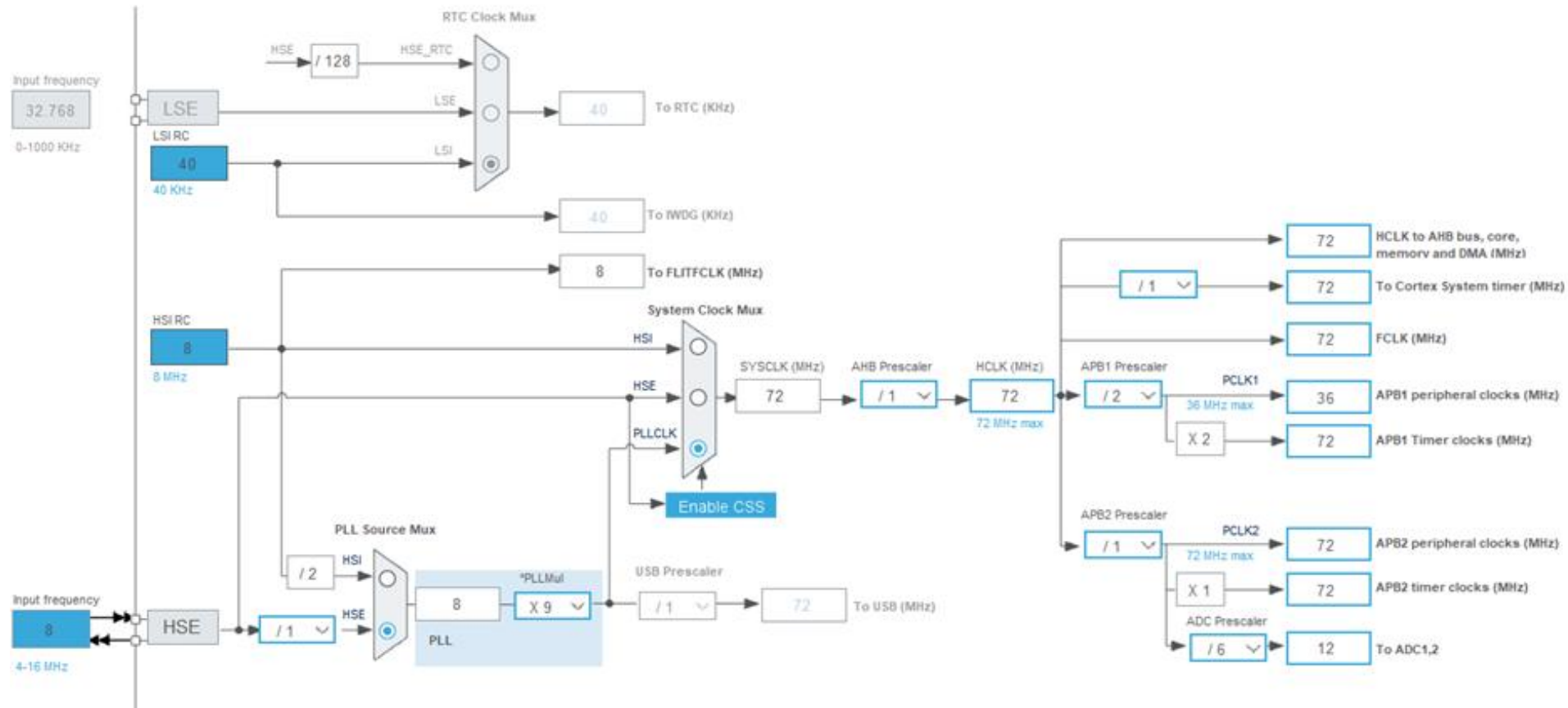


2. Cấu hình xung nhịp hệ thống (Clock Configuration)

- Trong phần RCC, mục High Speed Clock (HSE), chọn Crystal/Ceramic Resonator nhằm sử dụng thạch anh ngoài làm nguồn xung ổn định cho hệ thống.
- Vào Clock Configuration, trong ô HCLK (MHz) chỉnh thành 72.



3.6.1. Cài đặt phần mềm và cấu hình



Hình 37: Cấu hình Clock Configuration



3.6. Thiết kế nhúng

3.6.1. Cài đặt phần mềm và cấu hình

Pin Name	Signal on Pin	GPIO output I...	GPIO mode	GPIO Pull-up/...	Maximum out...	User Label	Modified
PB3	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_C...	✓
PB4	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_D...	✓
PB5	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_UP	✓
PB8	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON	✓
PB15	n/a	Low	Output Push ...	No pull-up an...	Low	DS18B20	✓
PC13-TAMP...	n/a	Low	Output Push ...	No pull-up an...	Low	LED	✓
PA4	n/a	n/a	Input mode	No pull-up an...	n/a	ECHO	✓
PA5	n/a	Low	Output Push ...	No pull-up an...	Low	TRIG_HCSR...	✓
PA6	n/a	Low	Output Push ...	No pull-up an...	Low	BUZZER	✓
PA7	n/a	Low	Output Push ...	No pull-up an...	Low	LIGHT	✓
PA15	n/a	n/a	External Interr...	Pull-up	n/a	BUTTON_OFF	✓
PB1	n/a	n/a	External Interr...	Pull-up	n/a	ALARM	✓

Hình 38: Cấu hình GPIO



3.6. Thiết kế nhúng

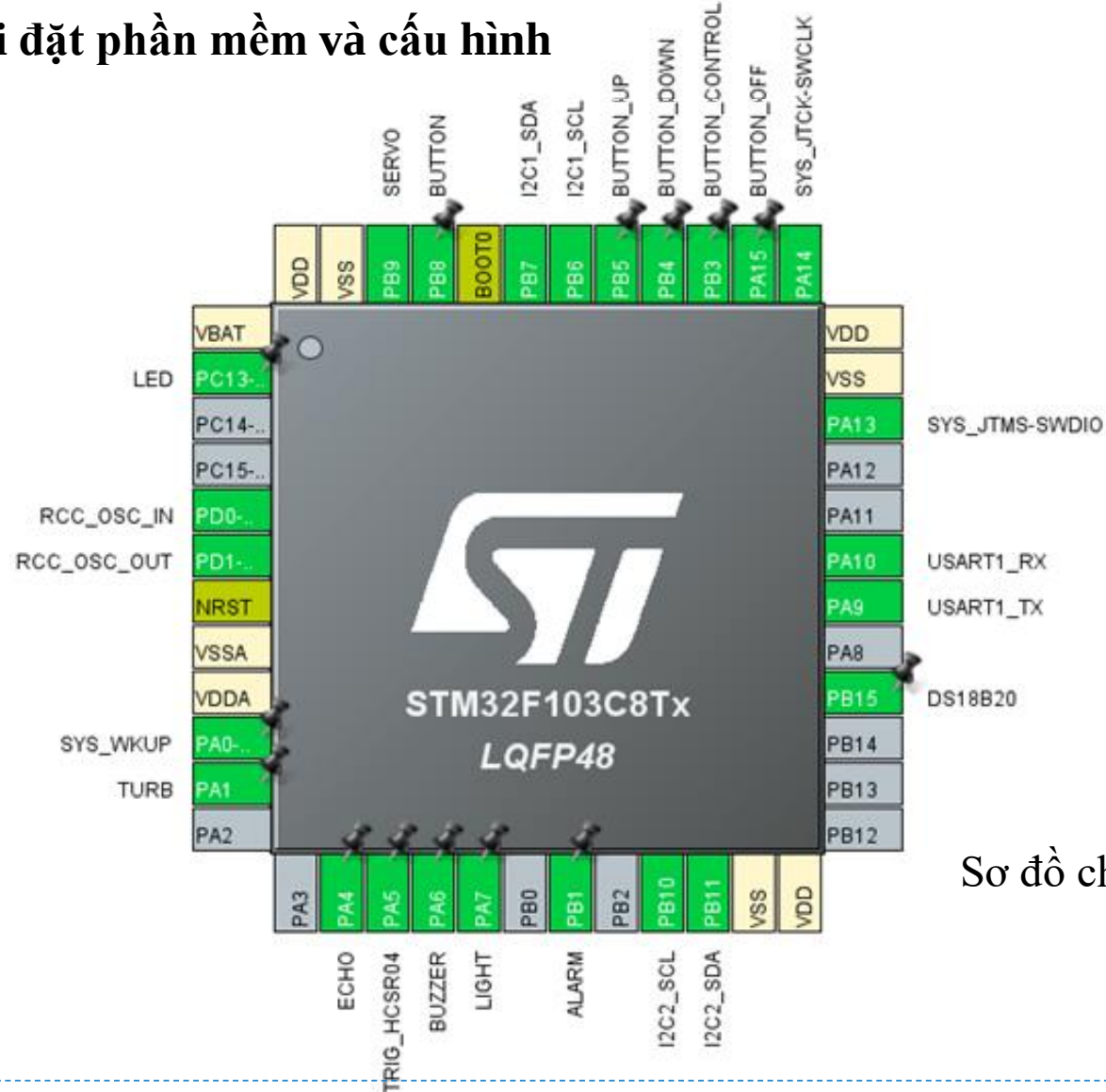
3.6.1. Cài đặt phần mềm và cấu hình

NVIC Code generation			
RCC global interrupt	☐	0	0
EXTI line1 interrupt	☒	0	0
EXTI line3 interrupt	☒	0	0
EXTI line4 interrupt	☒	0	0
ADC1 and ADC2 global interrupts	☐	0	0
EXTI line[9:5] interrupts	☒	0	0
TIM1 break interrupt	☐	0	0
TIM1 update interrupt	☐	0	0
TIM1 trigger and commutation interrupts	☐	0	0
TIM1 capture compare interrupt	☐	0	0
TIM2 global interrupt	☐	0	0
TIM4 global interrupt	☐	0	0
I2C1 event interrupt	☐	0	0
I2C1 error interrupt	☐	0	0
I2C2 event interrupt	☐	0	0
I2C2 error interrupt	☐	0	0
USART1 global interrupt	☒	1	0
EXTI line[15:10] interrupts	☒	0	0

Hình cấu hình ngắt

3.6. Thiết kế nhúng

3.6.1. Cài đặt phần mềm và cấu hình



Sơ đồ chân sau khi đã cấu hình trong STM32CubeMx



3.6. Thiết kế nhúng

3.6.2. STM32

a) Module đo độ đục

- Cảm biến: MJKDZ (Nguyên lý tán xạ ánh sáng).
- Nguyên tắc hoạt động:
 - + Nước trong: Ánh sáng ít tán xạ.
=> Điện áp đầu ra CAO.
 - + Nước đục: Ánh sáng tán xạ nhiều.
=> Điện áp đầu ra THẤP.
- Bộ xử lý: STM32 sử dụng bộ chuyển đổi ADC (Analog-to-Digital) để đọc giá trị điện áp này.

- Vấn đề: Tín hiệu analog từ cảm biến độ đục rất nhiễu (noisy), gây ra các đột biến giá trị.
- Giải pháp (Lọc nhiễu): Lấy trung bình cộng của nhiều mẫu đo.



3.6. Thiết kế nhúng

3.6.2. STM32

```
uint32_t Read_ADC_Avg(uint8_t samples) {  
    uint32_t sum = 0;  
    for(uint8_t i = 0; i < samples; i++) {  
        HAL_ADC_Start(&hadc1);  
        HAL_ADC_PollForConversion(&hadc1,  
10);  
        sum += HAL_ADC_GetValue(&hadc1);  
        HAL_ADC_Stop(&hadc1);  
    }  
    return sum / samples;  
}
```

- Hàm Read_ADC_Avg(200) được gọi.
- Hàm này thực hiện vòng lặp đọc giá trị ADC 200 lần liên tiếp, cộng dồn, sau đó chia trung bình.
- Kết quả avg_adc là một giá trị ổn định, đã được lọc nhiễu.

3.6.2. STM32

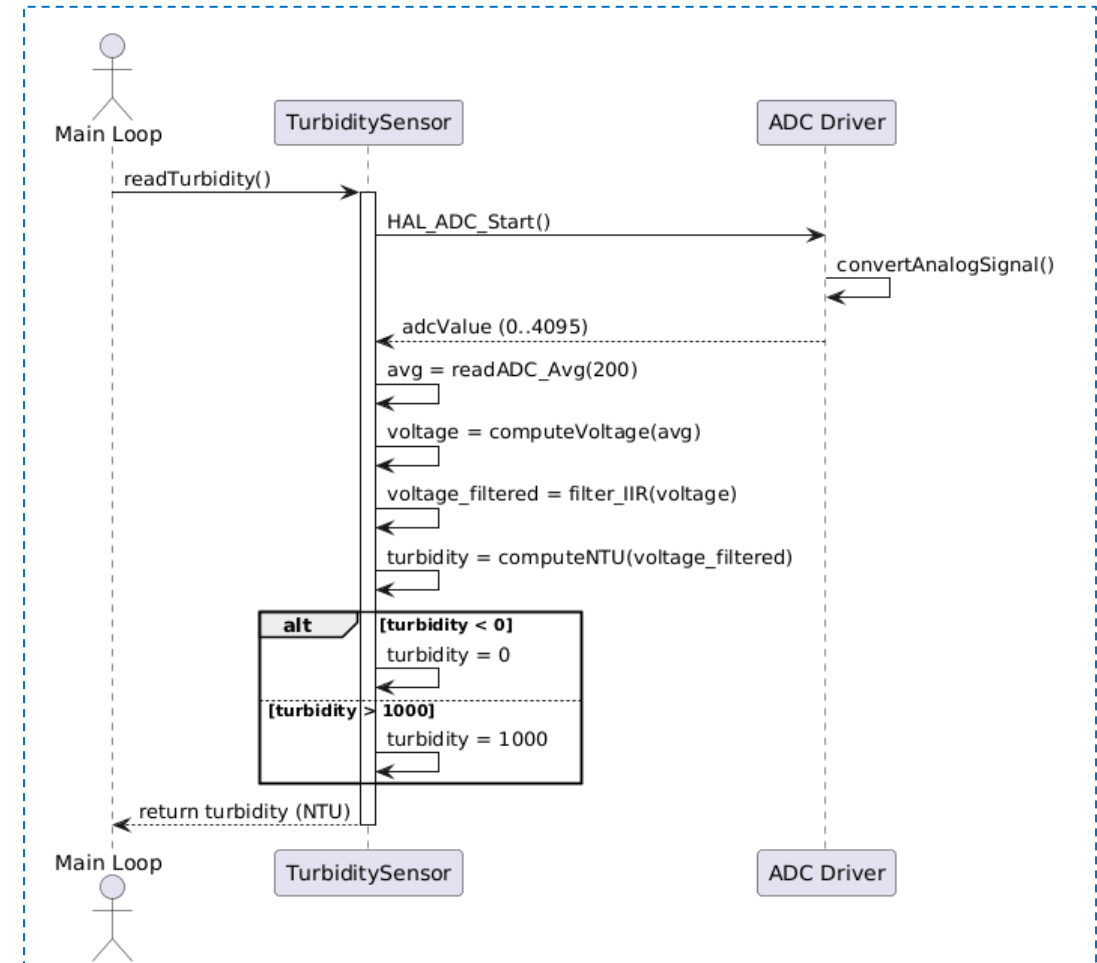
a) Module đo độ đục

- Điện áp đầu ra được STM32 đọc qua bộ chuyển đổi ADC.
- Giá trị trung bình được chuyển đổi sang điện áp thực tế:

$$V = \frac{ADC \times 3.3}{4095}$$

- quy đổi điện áp đo được sang giá trị độ đục (NTU):

$$\text{Turbidity} = -900.4 \times V^2 - 336.302 \times V + 2973.295$$





3.6. Thiết kế nhúng

3.6.2. STM32

a) Module đo độ đục

- Điện áp đầu ra được STM32 đọc qua bộ chuyển đổi ADC.
- Giá trị trung bình được chuyển đổi sang điện áp thực tế:

$$V = \frac{ADC \times 3.3}{4095}$$

- quy đổi điện áp đo được sang giá trị độ đục (NTU):

$$\text{Turbidity} = -900.4 \times V^2 - 336.302 \times V + 2973.295$$

```
void process_turbidity(void) {  
    uint32_t avg_adc =  
    Read_ADC_Avg(200);  
    float voltage_raw = (avg_adc *  
    3.3f) / 4095.0f;  
    float voltage = voltage_raw +  
    0.01f;           // hiệu chỉnh  
    nhiễu  
    turbidity = -900.4f * (voltage *  
    voltage) - 336.302f * voltage +  
    2973.295f;  
    if (turbidity < 0) {  
        turbidity = 0;  
    } else if (turbidity > 1000) {  
        turbidity = 1000;  
    }  
}
```



3.6. Thiết kế nhúng

3.6.2. STM32

b) Module đo nhiệt độ

Tầng	Hàm tiêu biểu	Chức năng chính
Low Level (Hardware)	DS18B20_SetPinOut/In(), DS18B20_ReadPin(), DS18B20_DelayUs()	Điều khiển trực tiếp chân GPIO, tạo xung delay chính xác ở cấp vi giây
Middle Level (Protocol)	DS18B20_Start(), DS18B20_Write(), DS18B20_Read()	Thực thi giao thức 1-Wire (khởi tạo, đọc/ghi bit, nhận phản hồi presence)
High Level (Application)	DS18B20_Init(), DS18B20_ReadTemp()	Hàm ứng dụng cao nhất – khởi tạo cảm biến và đọc giá trị nhiệt độ thực tế

3.6.2. STM32

b) Module đo nhiệt độ

Bước 1: Khởi tạo (Reset & Presence)

STM32 kéo bus DQ xuống thấp (Reset).

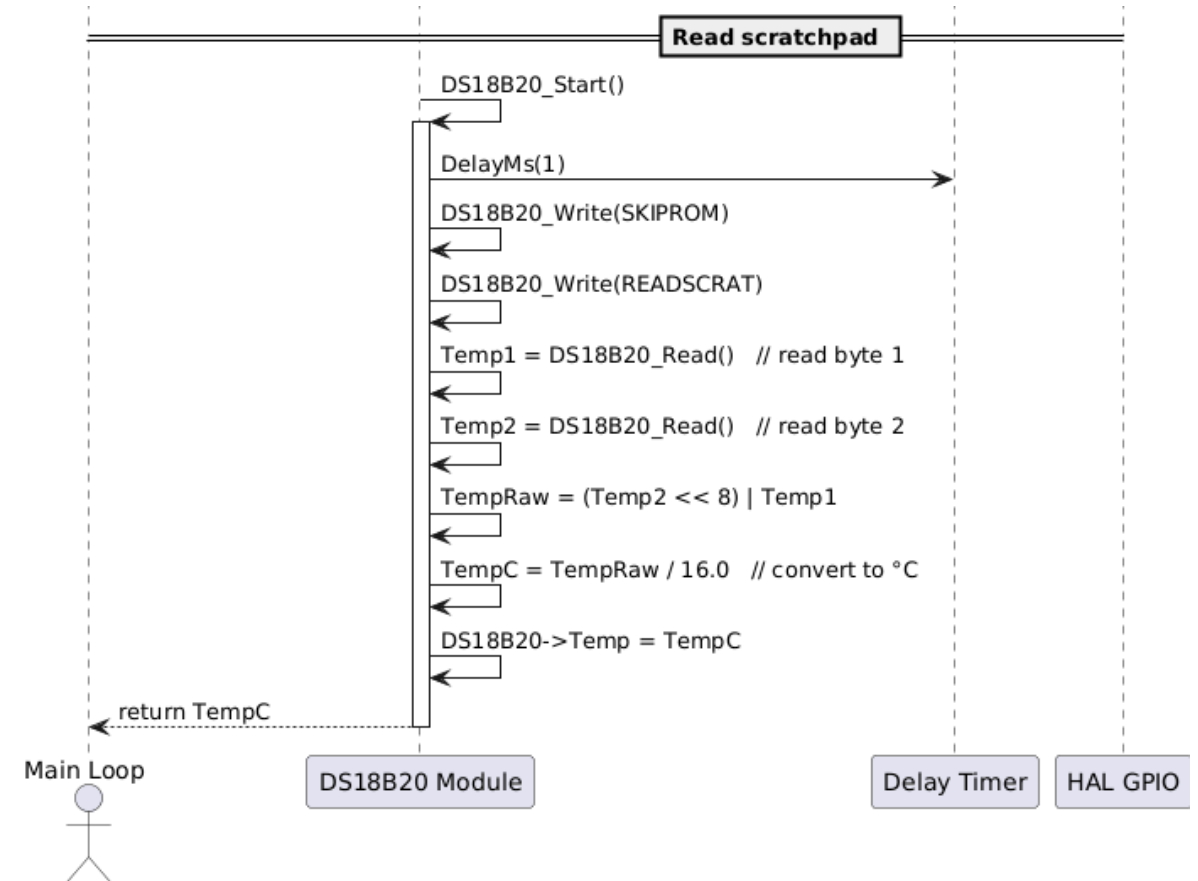
DS18B20 phản hồi bằng cách kéo bus xuống thấp (Presence Pulse) để báo "tôi có mặt".

Bước 2: Ra Lệnh Đo (Convert T)

STM32 gửi lệnh SKIP ROM (0xCC) (vì chỉ có 1 cảm biến).

STM32 gửi lệnh CONVERT T (0x44) (yêu cầu cảm biến bắt đầu đo).

Chờ khoảng 200ms để cảm biến đo xong.



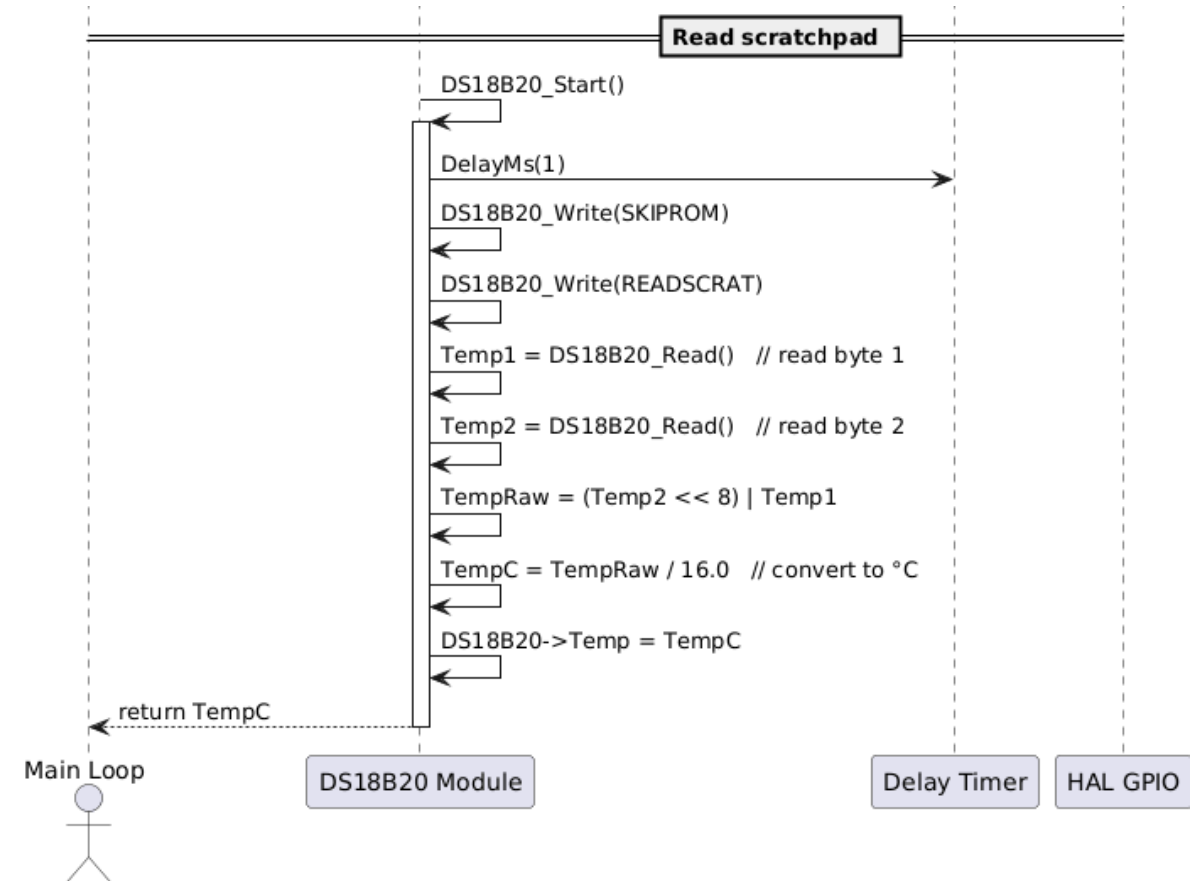
3.6.2. STM32

b) Module đo nhiệt độ

Bước 3: Đọc Dữ Liệu (Read Scratchpad)

Lặp lại Bước 1 (Reset & Presence). STM32 gửi lệnh SKIP ROM (0xCC).

STM32 gửi lệnh READ SCRATCHPAD (0xBE) (yêu cầu cảm biến gửi dữ liệu). STM32 đọc về 2 byte: Temp1 (LSB) và Temp2 (MSB).



3.6. Thiết kế nhúng

3.6.2. STM32

c) Module thời gian thực DS3231

- Nhiệm vụ: Thiết lập và quản lý thời gian cho ăn tự động.
- Thành phần chính:
 - + Người dùng (tương tác qua nút nhấn). Vi điều khiển (STM32) xử lý logic.
 - + Module thời gian thực (DS3231) lưu trữ và báo thức.
- Hai giai đoạn hoạt động chính:
 - + Thiết lập thời gian: Người dùng cài đặt giờ cho ăn.
 - + Kích hoạt cho ăn: Hệ thống tự động (hoặc thủ công) điều khiển servo.





3.6. Thiết kế nhúng

3.6.2. STM32

c) Module thời gian thực DS3231

- Khởi tạo hệ thống:

Chỉ chạy 1 lần khi khởi động (feeding_Init()).

Khởi tạo và kết nối với module DS3231 qua I2C.

Khởi động Timer (TIMx) ở chế độ PWM để chuẩn bị điều khiển servo.

Gọi hàm feeding_Servo_Angle(0) để đảm bảo servo ở vị trí đóng (0°) khi bắt đầu.

```
void feeding_Init(struct feeding_t
*feed) {
    _feed = feed;    // lay dia chi
    bien thong so
    DS3231_Init(_feed->hi2c);
    HAL_TIM_PWM_Start(_feed->htim,
    _feed->channel);
    feeding_Servo_Angle(0);
    // DS3231_SetTime(settime);
}
```



3.6. Thiết kế nhúng



3.6.2. STM32

c) Module thời gian thực DS3231

- Hàm `feeding_Handler()` được gọi liên tục trong vòng lặp `while(1)` để:
- Giám sát trạng thái cờ điều khiển (`IsFeed`, `IsSetTime`).
- Kiểm tra cờ `IsFeed` trong cấu trúc `_feed`.
- TH1: Nếu cờ khác 0 $\rightarrow$ tức là có yêu cầu cho cá ăn (do người dùng nhấn nút 1 chọn chế độ cho ăn thủ công hoặc đến giờ hẹn). Khi đó gọi hàm `feeding_Servo_Spin()` để điều khiển servo quay

```
void feeding_Servo_Spin() {  
    switch(_feed->IsFeed) {  
        case 1:  
            feeding_Servo_Angle(180);  
            _feed->IsFeed = 2;  
            break;  
        case 3:  
            feeding_Servo_Angle(0);  
            _feed->IsFeed = 0;  
            break;  
    }  
}
```



3.6. Thiết kế nhúng

3.6.2. STM32

c) Module thời gian thực DS3231

- Hàm `feeding_Handler()` được gọi liên tục trong vòng lặp `while(1)` để:
- Giám sát trạng thái cờ điều khiển (`IsFeed`, `IsSetTime`).
- Kiểm tra cờ `IsFeed` trong cấu trúc `_feed`.
- TH1: Nếu cờ khác 0 $\rightarrow$ tức là có yêu cầu cho cá ăn (do người dùng nhấn nút 1 chọn chế độ cho ăn thủ công hoặc đến giờ hẹn). Khi đó gọi hàm `feeding_Servo_Spin()` để điều khiển servo quay

```
static void
feeding_Servo_Angle(int16_t angle)
{
    if(angle > 180) angle = 180;
    if(angle < 0) angle = 0;

    int duty = -(angle * 50/9) +
1250;
    __HAL_TIM_SET_COMPARE(_feed-
>htim, _feed->channel, duty);
    _feed->angle = angle;
}
```



3.6. Thiết kế nhúng

3.6.2. STM32

```
void DS3231_SetAlarm(DS3231_Time_t time) { uint8_t set_alarm[4];

    set_alarm[0] = decToBcd(time.seconds) & 0x7F;
    set_alarm[1] = decToBcd(time.minutes) & 0x7F;
    set_alarm[2] = decToBcd(time.hour) & 0x7F;
    set_alarm[3] = 0x80;

    while(HAL_I2C_Mem_Write(_i2c, DS3231_ADDRESS, 0x07, 1, set_alarm, 4, 1000) != HAL_OK);

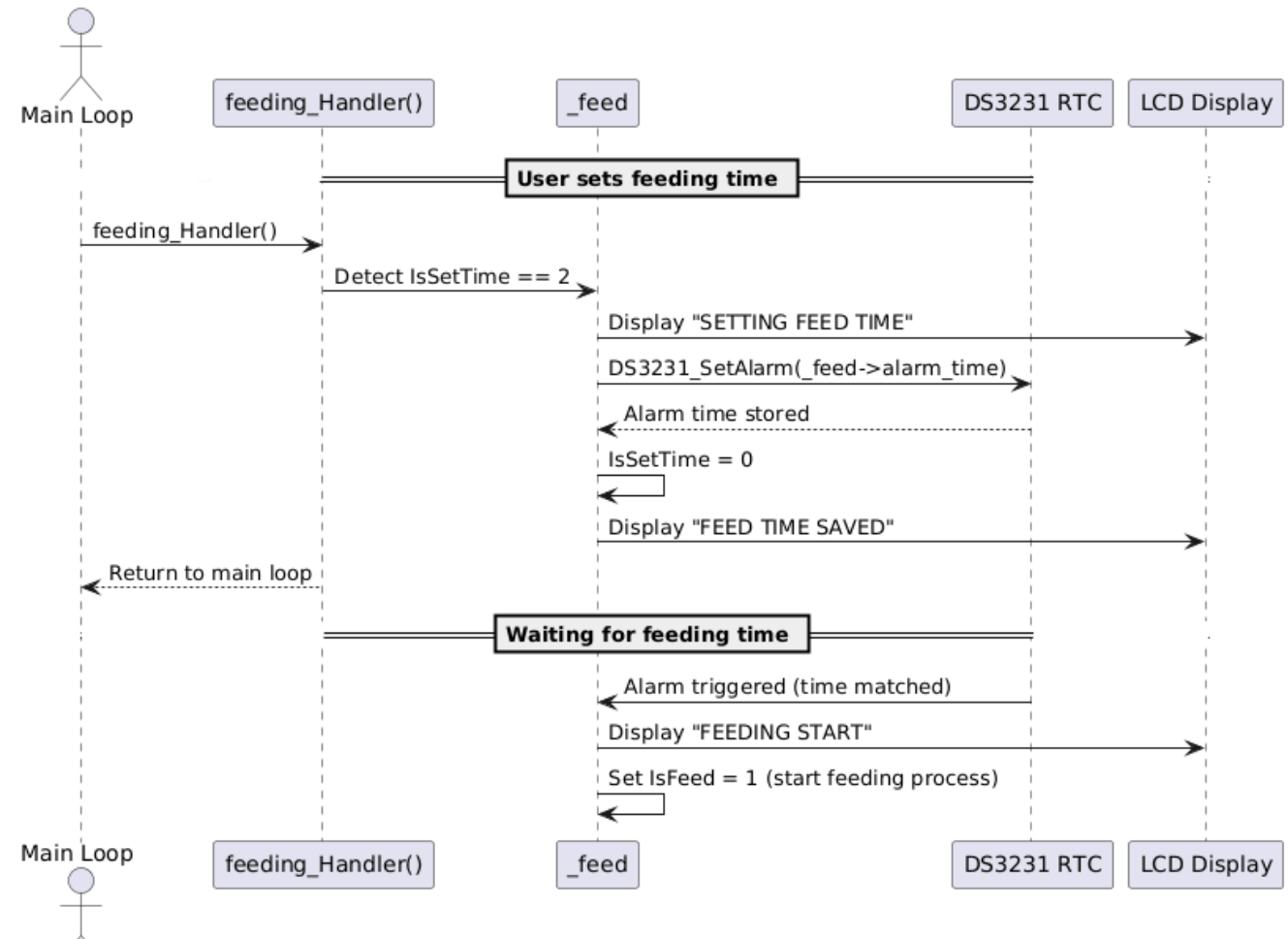
    uint8_t ctrl;
    HAL_I2C_Mem_Read(_i2c, DS3231_ADDRESS, 0x0E, 1, &ctrl, 1, 1000);
    ctrl |= (1<<2) | (1<<0);
    while(HAL_I2C_Mem_Write(_i2c, DS3231_ADDRESS, 0x0E, 1, &ctrl, 1, 1000) != HAL_OK);

    uint8_t status;
    HAL_I2C_Mem_Read(_i2c, DS3231_ADDRESS, 0x0F, 1, &status, 1, 1000);
    status &= ~(1<<0);
    while(HAL_I2C_Mem_Write(_i2c, DS3231_ADDRESS, 0x0F, 1, &status, 1, 1000) != HAL_OK);

}
```

3.6.2. STM32

c) Module thời gian thực DS3231





3.6. Thiết kế nhúng

3.6.2. STM32

d) Module siêu âm (đo lường thức ăn)

- Mục tiêu: Xác định lượng thức ăn còn lại trong hộp chứa.
- Thiết bị: Sử dụng cảm biến siêu âm HC-SR04.
- Nguyên tắc: Đo khoảng cách từ cảm biến (đặt trên nắp) xuống bề mặt thức ăn, sau đó quy đổi ra phần trăm (%).

- Bước 1 (Phát): STM32 gửi một xung $10\mu\text{s}$ (micro giây) đến chân TRIG của cảm biến.
- Bước 2 (Chờ): Cảm biến phát sóng siêu âm. Sóng chạm bề mặt thức ăn và phản xạ lại.
- Bước 3 (Nhận): Chân ECHO nhận sóng phản xạ và giữ mức CAO (HIGH) trong suốt thời gian sóng di chuyển (cả đi lẫn về).
- Bước 4 (Đo): STM32 dùng Timer (TIM2) để đo chính xác khoảng thời gian chân ECHO ở mức CAO.



3.6. Thiết kế nhúng

3.6.2. STM32

```
void HCSR04_Read(void)
{
    HAL_GPIO_WritePin(TRIG_HCSR04_GPIO_Port, TRIG_HCSR04_Pin, GPIO_PIN_SET);
    __HAL_TIM_SET_COUNTER(&htim2, 0);
    while (__HAL_TIM_GET_COUNTER(&htim2) < 10); // tạo xung 10us
    HAL_GPIO_WritePin(TRIG_HCSR04_GPIO_Port, TRIG_HCSR04_Pin, GPIO_PIN_RESET);
}
```

- Bước 1 (Phát): STM32 gửi một xung 10 μ s (micro giây) đến chân TRIG của cảm biến.
- Bước 2 (Chờ): Cảm biến phát sóng siêu âm. Sóng chạm bề mặt thức ăn và phản xạ lại.
- Bước 3 (Nhận): Chân ECHO nhận sóng phản xạ và giữ mức CAO (HIGH) trong suốt thời gian sóng di chuyển (cả đi lẫn về).
- Bước 4 (Đo): STM32 dùng Timer (TIM2) để đo chính xác khoảng thời gian chân ECHO ở mức CAO.



3.6. Thiết kế nhúng

3.6.2. STM32

```
pMillis = HAL_GetTick();  
while  
(!(HAL_GPIO_ReadPin(ECHO_GPIO_Port,  
ECHO_Pin)) && (HAL_GetTick() - pMillis) <  
20);  
Value1 = __HAL_TIM_GET_COUNTER(&htim2);  
  
pMillis = HAL_GetTick();  
while ((HAL_GPIO_ReadPin(ECHO_GPIO_Port,  
ECHO_Pin)) && (HAL_GetTick() - pMillis) <  
50);  
Value2 = __HAL_TIM_GET_COUNTER(&htim2);  
  
Distance = (float)(Value2 - Value1) *  
0.034 / 2;  
}
```

- Bước 1 (Phát): STM32 gửi một xung 10 μ s (micro giây) đến chân TRIG của cảm biến.
- Bước 2 (Chờ): Cảm biến phát sóng siêu âm. Sóng chạm bề mặt thức ăn và phản xạ lại.
- Bước 3 (Nhận): Chân ECHO nhận sóng phản xạ và giữ mức CAO (HIGH) trong suốt thời gian sóng di chuyển (cả đi lẫn về).
- Bước 4 (Đo): STM32 dùng Timer (TIM2) để đo chính xác khoảng thời gian chân ECHO ở mức CAO.



3.6. Thiết kế nhúng

3.6.2. STM32

d) Module siêu âm (đo lường thức ăn)

- Dựa trên thời gian đo được từ Timer (biến Value2 - Value1).
- Vi điều khiển tính khoảng cách đến vật cản theo công thức:

$$\text{Distance} = \frac{\text{Time} \times 0.034}{2}$$

- Chuyển đổi từ cm sang % dựa trên 2 mốc hiệu chuẩn thủ công:
- D_empty = 12.0f; (Khoảng cách (cm) khi hộp trống)
- D_full = 7.0f; (Khoảng cách (cm) khi hộp đầy)
- Công thức: percent = ((D_empty - Distance) / (D_empty - D_full)) * 100.0f;
- Kết quả luôn được giới hạn trong khoảng [0, 100%].



3.6. Thiết kế nhúng



3.6.2. STM32

e) Module hiển thị LCD

Trạng thái hiển thị	Nội dung hiển thị LCD	Mục đích
LCD_STATE_NORMAL	Nhiệt độ và độ đục nước	Hiển thị thông số môi trường cơ bản
LCD_STATE_FEEDING	“FEEDING...”	Báo đang tiến hành cho ăn
LCD_STATE_TIMESSET	Thời gian hẹn giờ	Hiển thị khi người dùng cài đặt lịch
LCD_STATE_INFO	Thời gian hiện tại và báo thức	Hiển thị thông tin thời gian thực
LCD_STATE_DISTANCE	Khoảng cách và phần trăm thức ăn	Đo và báo mức thức ăn còn lại



3.6. Thiết kế nhúng

3.6.2. STM32

e) Module hiển thị LCD

Trạng thái hiển thị	Nội dung hiển thị LCD	Mục đích
LCD_STATE_WARNING	Cảnh báo “TURBIDITY HIGH”	Hiển thị cảnh báo độ đục vượt ngưỡng
LCD_STATE_SET	Thông báo “Time's been set”	Hiển thị thông báo hẹn giờ đã được cài
LCD_STATE_SET_OFF	Thông báo “Alarm's deleted ”	Hiển thị thông báo hẹn giờ đã được xóa
LCD_STATE_LIGHT_ON	Thông báo “Light was on”	Hiển thị thông báo đèn đã được bật
LCD_STATE_LIGHT_OFF	Thông báo “Light was off”	Hiển thị thông báo đèn đã được tắt

3.6. Thiết kế nhúng

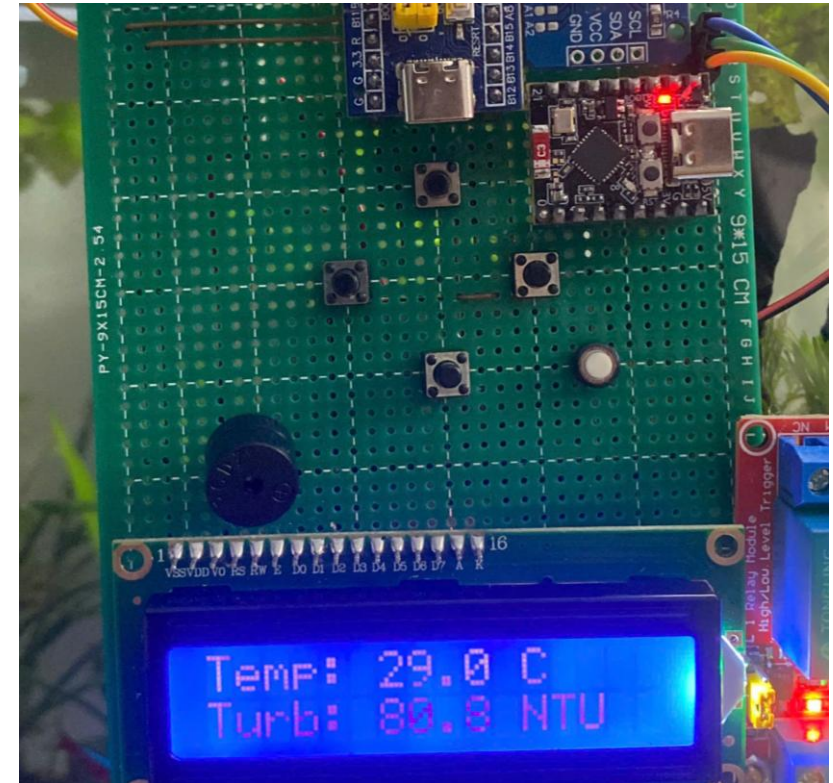
3.6.2. STM32

f) Module Button nút nhấn điều khiển

- Có hệ thống chống dôi phím (debounce)
- BUTTON: nút điều khiển chính (cho ăn/xác nhận hẹn giờ)
- BUTTON_UP: điều chỉnh giờ hẹn lên
- BUTTON_DOWN: điều chỉnh giờ hẹn lên
- BUTTON_CONTROL: Đổi màn hình sang chế độ khác

NORMAL → INFO → DISTANCE → NORMAL

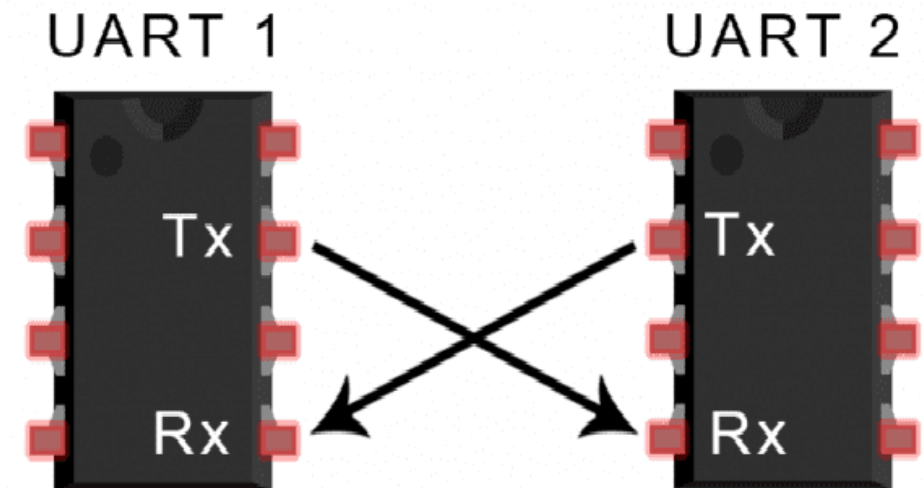
- BUTTON_OFF: xóa hẹn giờ
- ALARM: chuông báo cho cá ăn



3.6.2. STM32

g) Module giao tiếp UART với ESP32

1. Gửi dữ liệu (Truyền - TX): Định kỳ 10 phút một lần, nó sẽ tự động gửi một chuỗi dữ liệu chứa thông tin cảm biến (nhiệt độ, độ ẩm, % thức ăn) và thời gian hiện tại qua cổng UART.
2. Nhận dữ liệu (Nhận - RX): Nó luôn luôn lắng nghe dữ liệu đến. Khi nhận được một ký tự cụ thể (như 'L', 'l', hoặc 'F'), thực hiện một hành động ngay lập tức (như bật/tắt đèn, cho cá ăn).



3.6.2. STM32

g) Module giao tiếp UART với ESP32

- Kiểm tra xem đã đến thời điểm gửi dữ liệu hay chưa.
- Hàm `snprintf(...)` sẽ định dạng dữ liệu cảm biến.
- `HAL_UART_Transmit(...)` sẽ gửi toàn bộ chuỗi này qua UART.
- Sau khi gửi xong, dòng `last_uart_send_time = HAL_GetTick();` được dùng để cập nhật lại thời điểm gửi gần nhất
- Câu lệnh `HAL_UART_Receive_IT(...)` được gọi ngay trong vòng lặp `while(1)` (và cả bên trong hàm callback) nhằm đảm bảo UART luôn trong trạng thái sẵn sàng nhận dữ liệu.



3.6.2. STM32

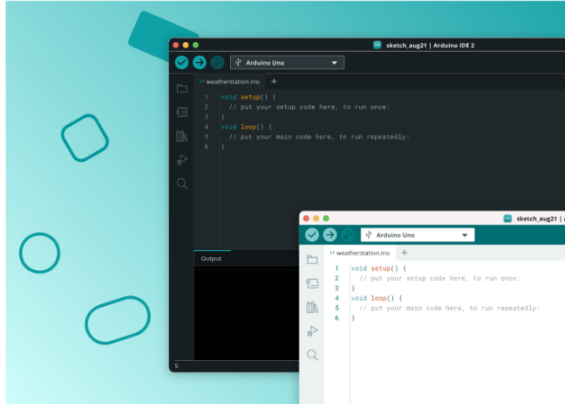
g) Module giao tiếp UART với ESP32

- Dòng if (huart->Instance == USART1) dùng để kiểm tra xem ngắt đến từ UART nào.
- Lệnh switch (rec) sẽ xác định ký tự vừa nhận được là gì.
- Các case 'L', case 'l', case 'F' tương ứng với từng lệnh điều. Đây chính là phần logic điều khiển từ xa của hệ thống.
- Cuối cùng, dòng HAL_UART_Receive_IT(...) là bắt buộc phải có: nó báo cho UART rằng “tôi đã xử lý xong dữ liệu, bây giờ hãy tiếp tục nghe byte tiếp theo.”



3.6.1 Cài đặt phần mềm và cấu hình ESP32

- Truy cập trang web chính thức của Arduino tại: <https://www.arduino.cc/en/software>.
- Tải xuống phiên bản Arduino IDE 2.3.6 (hoặc phiên bản mới nhất) phù hợp với hệ điều hành đang sử dụng (Windows, macOS hoặc Linux).
- Thực hiện cài đặt theo hướng dẫn của trình cài đặt.



Arduino IDE 2.3.6
Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit) **DOWNLOAD**

Nightly Builds
Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

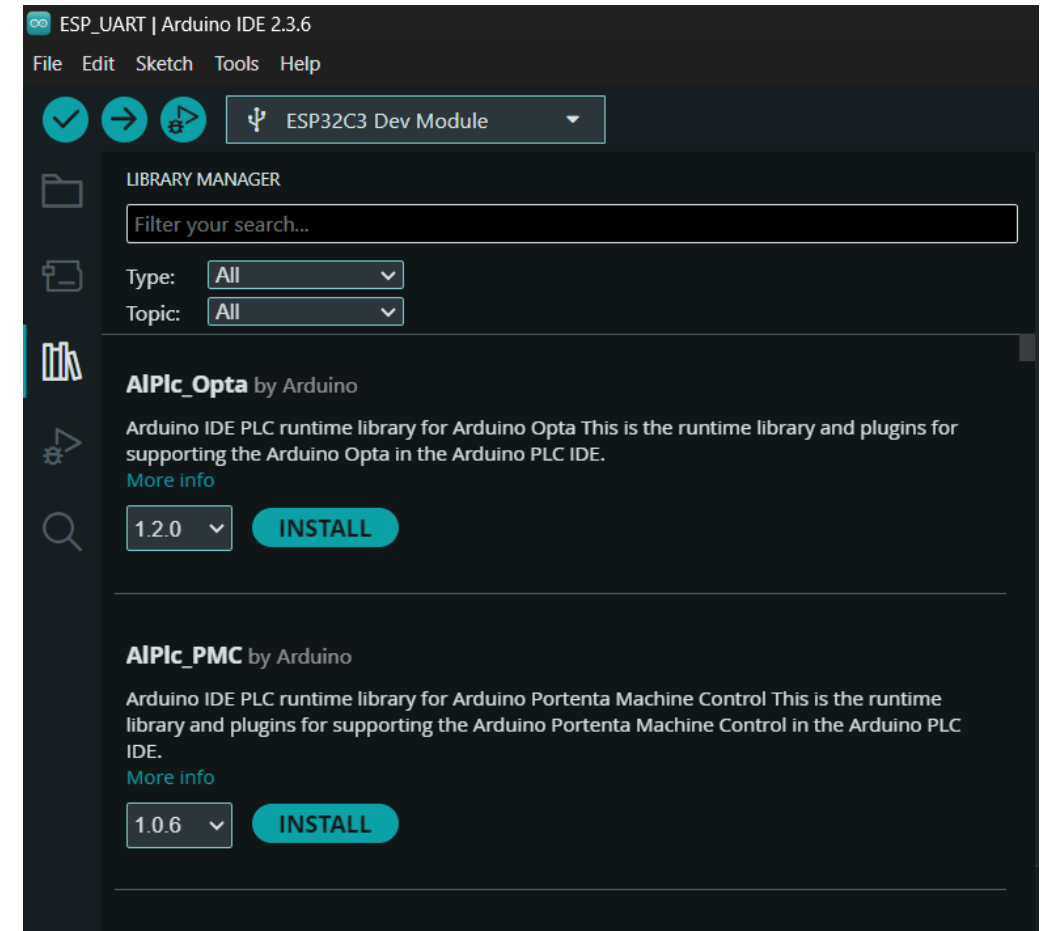


3.6.1 Cài đặt phần mềm và cấu hình ESP32



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

Cài đặt gói hỗ trợ (Board Manager) cho ESP32: để có thể biên dịch và nạp mã nguồn cho họ vi điều khiển ESP32, người dùng cần cài đặt gói hỗ trợ bo mạch của Espressif Systems thông qua Boards Manager của Arduino IDE.





3.6.1 Cài đặt phần mềm và cấu hình ESP32



Cài đặt thư viện phụ trợ: để mở rộng chức năng cho ESP32 và tích hợp các giao thức truyền thông, một số thư viện bên thứ ba được sử dụng và cài đặt Library Manager.

Một số thư viện chính được sử dụng trong dự án được liệt kê qua bảng sau:

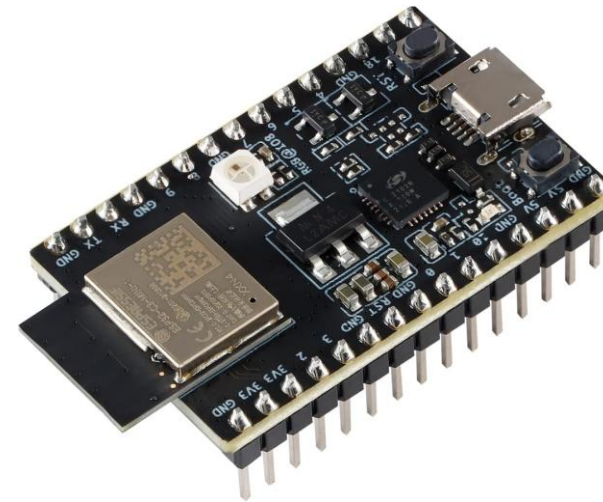
Tên thư viện	Tác giả	Chức năng chính
ArduinoJson	Benoît Blanchon	Tuần tự hóa dữ liệu cảm biến sang định dạng JSON và giải tuần tự gói tin JSON nhận từ máy chủ MQTT.
PubSubClient	Nick O'Leary	Cung cấp API gọn nhẹ cho giao thức MQTT, hỗ trợ kết nối, đăng ký (subscribe) và xuất bản (publish) dữ liệu lên broker.
NTPClient	Fabrice Weinberg	Truy vấn máy chủ NTP để đồng bộ hóa thời gian hệ thống theo giao thức UDP, đảm bảo độ chính xác thời gian.

Chức năng: ESP32-C3 Mini đóng vai trò IoT Gateway.

- Giao tiếp STM32: Nhận dữ liệu cảm biến & truyền lệnh điều khiển qua UART.
- Kết nối Cloud: Thiết lập Wi-Fi và truyền thông MQTT/TLS an toàn với HiveMQ Broker.

Cấu trúc code bao gồm:

- + Giai đoạn khởi tạo hệ thống `setup()`
- + Vòng lặp chính `loop()`
- + Xử lý mất kết nối `reconnect()`
- + Xử lý lệnh điều khiển `callback()`



HIVEMQ



3.6.3. ESP32



a) Gửi và nhận dữ liệu trên ESP32 và HiveMQ MQTT

Giai đoạn khởi tạo hệ thống `setup()`:

- Cấu hình kết nối Wifi
- Cấu hình MQTT Broker
- Cấu hình đồng bộ thời gian (NTP)
- Cấu hình tần suất gửi dữ liệu

- Cấu hình kết nối WiFi.

```
const char* ssid = "Wifi";  
const char* password = "11111111";
```

- Cấu hình MQTT Broker.

```
const char* mqtt_server =  
"fdca86b156bf4f86973c13d0a7fe4e2c.s1.eu.h  
ivemq.cloud";  
const int mqtt_port = 8883;  
const char* mqtt_user = "aquanova_user";  
const char* mqtt_pass = "Aquanova123";
```

- Cấu hình đồng bộ thời gian (NTP)

```
WiFiUDP ntpUDP;  
NTPClient timeClient(ntpUDP, "pool.ntp.org",  
25200); // 25200 = 7 * 60 * 60 (UTC+7)
```

- Cấu hình tần suất gửi dữ liệu

```
const unsigned long SEND_INTERVAL =  
60000;
```



3.6.3. ESP32



a) Gửi và nhận dữ liệu trên ESP32 và
HiveMQ MQTT

Hình ảnh minh chứng cho thấy ESP32 đã
khởi tạo và kết nối thành công với mạng
Wi-Fi 2.4 GHz.

Sau khi kết nối, module tiến hành đồng bộ
thời gian thực đảm bảo tính chính xác của
các gói tin.

```
17:06:04.528 -> [MQTT] Subscribed to:
aquanova/control
17:06:04.528 -> From STM32: 702.7,29.0,0,14:39:48
17:40:40.173 -> === AquaNova ESP32-C3 MQTT ===
17:40:40.173 -> [UART] Serial1 started for STM32 on
RX:20, TX:21
17:40:40.173 -> [WiFi] Connecting to Zun
17:40:40.346 -> ....
17:40:41.840 -> [WiFi] Connected!
17:40:41.840 -> [WiFi] IP: 10.77.235.196
17:40:41.840 -> [NTP] Syncing time...
17:40:41.992 -> [NTP] Time synced!
17:40:41.992 -> [SYSTEM] Setup complete!
17:40:41.992 -> [MQTT] Attempting connection...
connected!
17:40:44.264 -> [MQTT] Subscribed to:
aquanova/control
```



3.6.3. ESP32



a) Gửi và nhận dữ liệu trên ESP32 và HiveMQ MQTT

Vòng lặp chính `loop()`

- Duy trì kết nối MQTT
- Xử lý nền MQTT
- Gửi dữ liệu định kỳ

- Duy trì kết nối MQTT.

```
if (!client.connected()) {  
    reconnect();}
```

- Xử lý nền MQTT.

```
client.loop();
```

- Gửi dữ liệu định kỳ:

```
unsigned long now = millis();  
if (now - lastMsg > SEND_INTERVAL) {  
    lastMsg = now;  
    if(client.connected()) {  
        timeClient.update();  
        publishSensorData();}}
```

a) Gửi và nhận dữ liệu trên ESP32 và HiveMQ MQTT

Xử lý mất kết nối `reconnect()`:

Hàm `reconnect()` chịu trách nhiệm tái thiết lập phiên MQTT khi bị ngắt. ESP32 tạo một client ID duy nhất bằng cách kết hợp chuỗi cố định và địa chỉ MAC.

```
void reconnect() {  
    while (!client.connected()) {  
        Serial.print("[MQTT] Attempting  
connection...");  
  
        String clientId = "AquaNova_ESP32_C3_";  
        clientId += WiFi.macAddress();  
        clientId.replace(":", "");  
  
        if (client.connect(clientId.c_str(),  
mqtt_user, mqtt_pass)) {  
            Serial.println(" connected!");  
            if (client.subscribe(subscribe_topic)) {  
                Serial.print("[MQTT] Subscribed to:  
");  
                Serial.println(subscribe_topic);  
            } else {  
                Serial.println("[ERROR] Subscribe  
failed!");  
            }  
        } else {  
            Serial.print(" failed, rc=");  
            Serial.print(client.state());  
            Serial.println(" - trying again in 5  
seconds");  
            delay(5000);  
        }  
    }  
}
```




3.6.3. ESP32



a) Gửi và nhận dữ liệu trên ESP32 và HiveMQ MQTT

Xử lý lệnh điều khiển `callback()`

Hàm `callback()` được kích hoạt mỗi khi có bản tin đến từ topic điều khiển. Payload (chuỗi JSON) được phân tích bằng thư viện `ArduinoJson`.

```
void callback(char* topic, byte* payload,
unsigned int length) {
    Serial.print("\n[MQTT] Message arrived on
topic: ");
    Serial.println(topic);
    char message[length + 1];
    memcpy(message, payload, length);
    message[length] = '\0';
    Serial.print("[MQTT] Payload: ");
    Serial.println(message);
    DynamicJsonDocument doc(128);
    DeserializationError error =
deserializeJson(doc, message);

    if (error) {
        Serial.print("[ERROR] JSON parse failed:
");
        Serial.println(error.c_str());
        return;
    }
}
```

a) Gửi và nhận dữ liệu trên ESP32 và HiveMQ MQTT

Xử lý lệnh điều khiển `callback()`

Xác định lệnh tương ứng (“light”, “feeding”). ESP32 gửi ký tự điều khiển (‘L’, ‘F’) tới STM32 thông qua UART (`Serial1.write()`), để thực hiện hành động tương ứng.

```
if (doc.containsKey("light")) {
    int light_status = doc["light"];
    if (light_status == 1) {
        Serial1.write('L');
        Serial.println("[CONTROL] Sent 'L' (Light
ON) to STM32");
    } else {
        Serial1.write('l');
        Serial.println("[CONTROL] Sent 'l' (Light
OFF) to STM32");
    }
} else if (doc.containsKey("feeding")) {
    int feeding_status = doc["feeding"];
    if (feeding_status != 0) {
        Serial1.write('F');
        Serial.println("[CONTROL] Sent 'F' (Start
Feeding) to STM32");
    } else {
        Serial1.write('f');
        Serial.println("[CONTROL] Sent 'f' (Stop
Feeding) to STM32");
    }
} else Serial1.write('B');
}
```



3.6.3. ESP32



b) Truyền nhận dữ liệu trên ESP32 và STM32

Giao tiếp giữa hai vi điều khiển được thiết lập qua Serial1 của ESP32.

Giao tiếp UART cho phép hai vi điều khiển truyền dữ liệu song công (full-duplex) theo dạng chuỗi ký tự ASCII, giúp giảm độ trễ và tiết kiệm tài nguyên phần cứng so với giao thức phức tạp hơn như SPI hoặc I²C.

```
const int STM32_RX_PIN = 20; // Chân RX  
của ESP32 (kết nối với TX của STM32)  
const int STM32_TX_PIN = 21; // Chân TX  
của ESP32 (kết nối với RX của STM32)=
```

- STM32_RX_PIN (GPIO20) là chân đầu vào nhận dữ liệu (RX) trên ESP32, được nối với chân TX (truyền) của STM32.
- STM32_TX_PIN (GPIO21) là chân đầu ra phát dữ liệu (TX) trên ESP32, được nối với chân RX (nhận) của STM32.

```
Serial1.begin(9600, SERIAL_8N1,  
STM32_RX_PIN, STM32_TX_PIN);  
Serial.printf("[UART] Serial1 started for  
STM32 on RX:%d, TX:%d\n", STM32_RX_PIN,  
STM32_TX_PIN);
```



3.6.3. ESP32



b) Truyền nhận dữ liệu trên ESP32 và STM32

ESP32 trong mô hình đóng vai trò là bộ truyền thông WiFi, chịu trách nhiệm đóng gói dữ liệu và gửi lên máy chủ MQTT hoặc các dịch vụ đám mây khác. Dữ liệu được STM32 truyền sang ESP32 ở dạng payload, tức là chuỗi dữ liệu thô bao gồm các giá trị đo được từ cảm biến.

Ví dụ, chuỗi payload "340,25.6\n" sau khi xử lý sẽ được

ESP32 chuyển thành chuỗi JSON như sau:

```
{"turbidity":340, "temperature":25.6}
```

Cấu trúc chuỗi JSON trên gồm hai thành phần chính:

- + "turbidity": 340 – biểu thị giá trị độ đục của nước được cảm biến đo được (đơn vị NTU).
- + "temperature": 25.6 – thể hiện nhiệt độ môi trường hoặc của nước tại thời điểm đo (đơn vị °C).



3.6.3. ESP32



b) Truyền nhận dữ liệu trên ESP32 và STM32

Chuỗi JSON này sau đó sẽ được xuất bản (publish) lên broker MQTT để gửi đến các thiết bị hoặc dịch vụ đăng ký nhận dữ liệu. Giao tiếp UART giữa STM32 và ESP32 được thiết lập theo chuẩn truyền 8 bit dữ liệu, còn gọi là cấu hình 8N1 (gồm 1 bit start, 8 bit dữ liệu, không parity và 1 bit stop).

- Gửi dữ liệu thành công từ STM32 đến ESP32

```
17:06:04.528 -> [PUBLISH]
{"turbidity":702.7,"temperature":29,"feed":0,"t
hoi_gian":"14:39:48"}
Topic: aquanova/telemetry
QoS: 0
17:06:04.49 -> [PUBLISH]
{"turbidity":702.7,"temperature":29,"feed":30,"
thoi_gian":"14:39:49"}
Topic: aquanova/telemetry
QoS: 0
17:06:04.50 -> [PUBLISH]
{"turbidity":702.6,"temperature":29,"feed":35,"
thoi_gian":"14:39:50"}
Topic: aquanova/telemetry
QoS: 0
```



3.6.3. ESP32



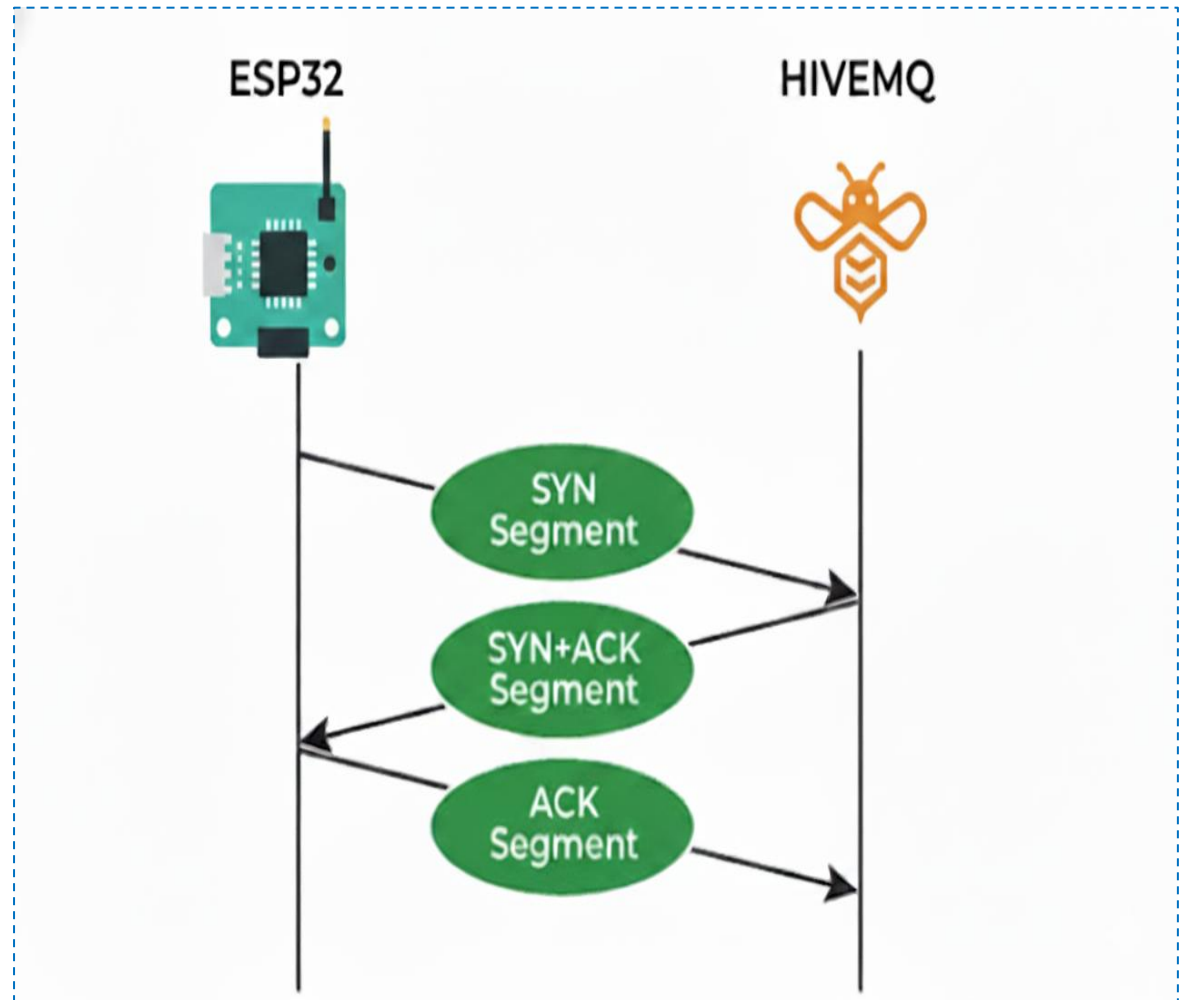
b) Truyền nhận dữ liệu trên ESP32 và STM32

Trong hàm publishSensorData(), ESP32 kiểm tra Serial1.available(). Khi có dữ liệu, hàm Serial1.readStringUntil('\n') đọc toàn bộ chuỗi dữ liệu cho đến ký tự xuống dòng, đảm bảo nhận trọn vẹn một bản tin.

```
void publishSensorData() {  
    if (Serial1.available() > 0) {  
        String input = Serial1.readStringUntil('\n');  
        input.trim(); if (input.length() > 0) {  
            String jsonPayload =  
                convertPayloadToJson(input);  
  
            Serial.print("From STM32: ");  
            Serial.println(input);  
            Serial.print("[PUBLISH] ");  
            Serial.println(jsonPayload);  
  
            if (!client.publish(publish_topic,  
                                jsonPayload.c_str())) {  
                Serial.println("[ERROR] Publish  
failed.");  
            }  
        }  
    }  
}
```


c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Khi ESP32 khởi tạo kết nối với HiveMQ Broker, thiết bị sẽ tiến hành quá trình bắt tay ba bước (three-way handshake) để thiết lập kênh truyền TCP ổn định.





3.6.3. ESP32



c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Quan sát thực tế bằng Wireshark cho thấy rõ chuỗi gói tin khởi tạo kết nối với các cờ TCP [SYN], [SYN, ACK] và [ACK]. Sau khi thiết lập thành công, tất cả bản tin MQTT (CONNECT, PUBLISH, SUBSCRIBE, PINGREQ, DISCONNECT) đều được truyền qua kênh TCP này.

Local Area Connection 10

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

frame.number == 23 or frame.number == 24 or frame.number == 25

No.	Time	Source	Destination	Protocol	Length	Info
23	21.630590	192.168.137.225	46.137.47.218	TCP	62	54460 → 8883 [SYN] Seq=0 Win=5760 Len=0 MSS=1436 SACK_PERM
24	21.847734	46.137.47.218	192.168.137.225	TCP	62	8883 → 54460 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM
25	21.966267	192.168.137.225	46.137.47.218	TCP	54	54460 → 8883 [ACK] Seq=1 Ack=1 Win=5760 Len=0

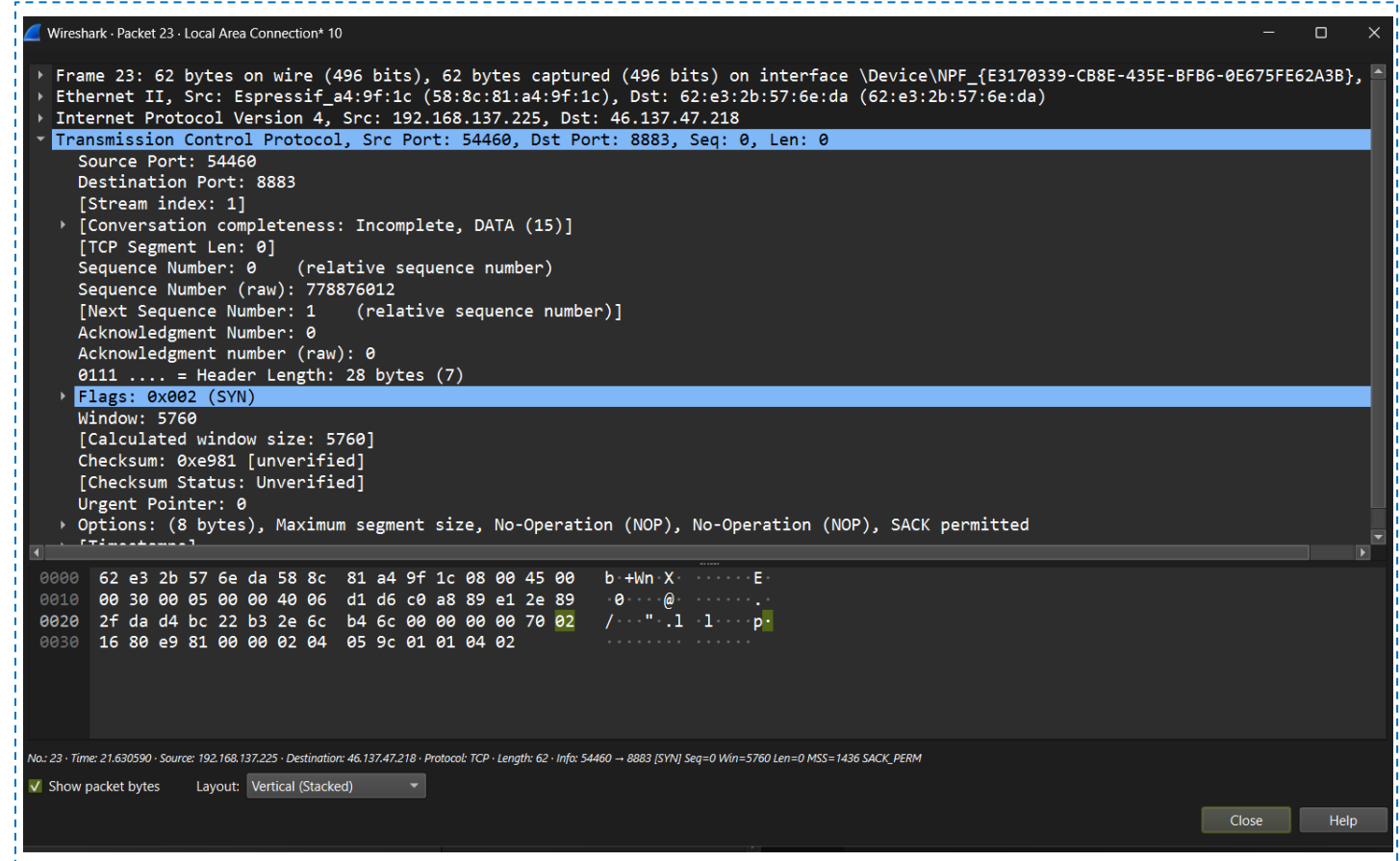


3.6.3. ESP32



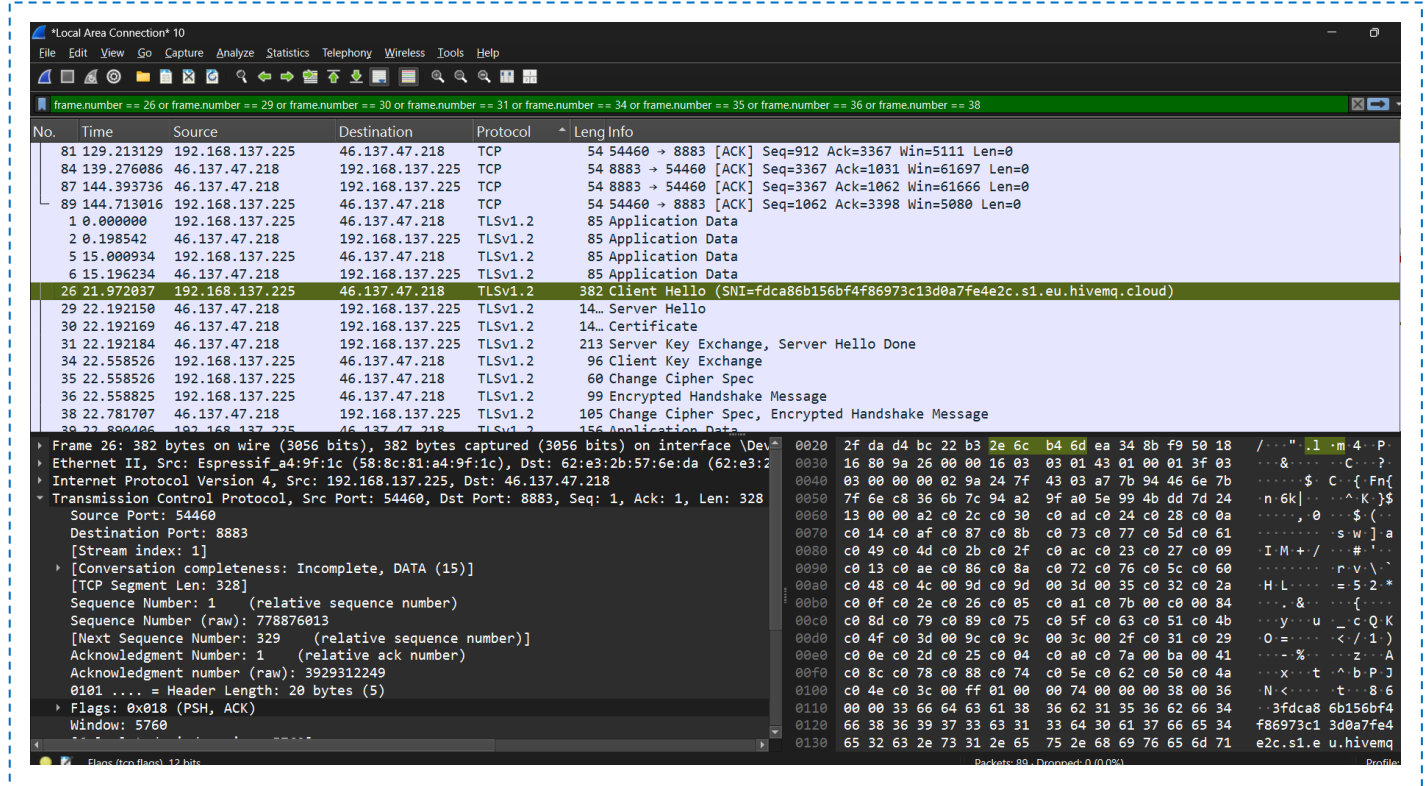
c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Sau khi ESP32 gửi gói SYN, MQTT Broker phản hồi bằng gói SYN-ACK, và cuối cùng ESP32 gửi lại ACK để hoàn tất quá trình three-way handshake.



c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Kênh giao tiếp TCP hiện tại đã được nâng cấp lên thành một kênh an toàn, mã hóa bằng TLS, sẵn sàng cho việc truyền tải dữ liệu ứng dụng MQTT



No.	Time	Source	Destination	Protocol	Length	Info
81	129.213129	192.168.137.225	46.137.47.218	TCP	54	54460 → 8883 [ACK] Seq=912 Ack=3367 Win=5111 Len=0
84	139.276086	46.137.47.218	192.168.137.225	TCP	54	8883 → 54460 [ACK] Seq=3367 Ack=1031 Win=61697 Len=0
87	144.393736	46.137.47.218	192.168.137.225	TCP	54	8883 → 54460 [ACK] Seq=3367 Ack=1062 Win=61666 Len=0
89	144.713016	192.168.137.225	46.137.47.218	TCP	54	54460 → 8883 [ACK] Seq=1062 Ack=3398 Win=5080 Len=0
10	0.000000	192.168.137.225	46.137.47.218	TLSv1.2	85	Application Data
20	0.198542	46.137.47.218	192.168.137.225	TLSv1.2	85	Application Data
5	15.000934	192.168.137.225	46.137.47.218	TLSv1.2	85	Application Data
6	15.196234	46.137.47.218	192.168.137.225	TLSv1.2	85	Application Data
26	21.972037	192.168.137.225	46.137.47.218	TLSv1.2	382	Client Hello (SNI=fdca86b156bf4f86973c13d0a7fe4e2c.s1.eu.hivemq.cloud)
29	22.192150	46.137.47.218	192.168.137.225	TLSv1.2	14...	Server Hello
30	22.192169	46.137.47.218	192.168.137.225	TLSv1.2	14...	Certificate
31	22.192184	46.137.47.218	192.168.137.225	TLSv1.2	213	Server Key Exchange, Server Hello Done
34	22.558526	192.168.137.225	46.137.47.218	TLSv1.2	96	Client Key Exchange
35	22.558526	192.168.137.225	46.137.47.218	TLSv1.2	60	Change Cipher Spec
36	22.558825	192.168.137.225	46.137.47.218	TLSv1.2	99	Encrypted Handshake Message
38	22.781707	46.137.47.218	192.168.137.225	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message
39	22.800000	192.168.137.225	46.137.47.218	TLSv1.2	156	Application Data

Frame 26: 382 bytes on wire (3056 bits), 382 bytes captured (3056 bits) on interface \Device\NPF{...} Ethernet II, Src: Espressif a4:9f:1c (58:8c:81:a4:9f:1c), Dst: 62:e3:2b:57:6e:da (62:e3:2b:57:6e:da) Internet Protocol Version 4, Src: 192.168.137.225, Dst: 46.137.47.218 Transmission Control Protocol, Src Port: 54460, Dst Port: 8883, Seq: 1, Ack: 1, Len: 328

Source Port: 54460
Destination Port: 8883
[Stream index: 1]
[Conversation completeness: Incomplete, DATA (15)]
[TCP Segment Len: 328]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 778876013
[Next Sequence Number: 329 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 3929312249
0101 = Header Length: 20 bytes (5)
Flags: 0x018 (PSH, ACK)
Window: 5760

0020 2f da d4 bc 22 b3 2e 6c b4 6d ea 34 8b f9 50 18 /...".l.m.4.P
0030 16 80 9a 26 00 00 16 03 03 01 43 01 00 01 3f 03 ...&...C...?
0040 03 00 00 00 02 9a 24 7f 43 03 a7 7b 94 46 6e 7b ...\$...C...{Fn
0050 7f 6e c8 36 6b 7c 94 a2 9f a0 5e 99 4b dd 7d 24 ...n 6k|...^K)\$
0060 13 00 00 a2 c0 2c c0 30 c0 ad c0 24 c0 28 c0 0a ...,...0...\$ (...
0070 c0 14 c0 af c0 87 c0 8b c0 73 c0 77 c0 5d c0 61 ...,...s w] a
0080 c0 49 c0 4d c0 2b c0 2f c0 ac c0 23 c0 27 c0 09 ...I M + / ...# ...
0090 c0 13 c0 ae c0 86 c0 8a c0 72 c0 76 c0 5c c0 60 ...,...n v \ ...
00a0 c0 48 c0 4c 00 9d c0 9d 00 3d 00 35 c0 32 c0 2a ...H L ... = 5 2 *
00b0 c0 0f c0 2e c0 26 c0 05 c0 a1 c0 7b 00 c0 00 84 ...&...{...Q
00c0 c0 8d c0 79 c0 89 c0 75 c0 5f c0 63 c0 51 c0 4b ...y ...u ...c Q K
00d0 c0 4f c0 3d 00 9c c0 9c 00 3c 00 2f c0 31 c0 29 ...O ...< / 1)
00e0 c0 0e c0 2d c0 25 c0 04 c0 a0 c0 7a 00 ba 00 41 ...%...z ...A
00f0 c0 8c c0 78 c0 88 c0 74 c0 5e c0 62 c0 50 c0 4a ...x ...t ^ b P J
0100 c0 4e c0 3c 00 ff 01 00 00 74 00 00 00 38 00 36 ...N < ...t ...8 6
0110 00 00 33 66 64 63 61 38 36 62 31 35 36 62 66 34 ...3fdca8 6b156bf4
0120 66 38 36 39 37 33 63 31 33 64 30 61 37 66 65 34 ...f86973c1 3d0a7fe4
0130 65 32 63 2e 73 31 2e 65 75 2e 68 69 76 65 6d 71 ...e2c.s1.e u.hivemq



3.6.3. ESP32



c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Phần payload chứa bản tin MQTT, bao gồm Fixed Header, Variable Header và Payload.

Ví dụ, khi ESP32 gửi một bản tin PUBLISH, phần dữ liệu TCP chứa nội dung:

“MQTT Publish Message, Topic

Name:aquanova/control, Payload:

“turbidity”:801.4, “temperature”:29, “feed”:0”,

và Wireshark hiển thị như sau:

```
Transmission Control Protocol
Source Port: 52231
Destination Port: 8883
Flags: PSH, ACK
```

c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Các gói tin hiển thị trên Wireshark cho thấy dữ liệu ứng dụng (Application Data) được truyền an toàn qua cổng 8883, cổng mặc định của MQTT over TLS. Ta thấy toàn bộ nội dung MQTT được đóng gói bên trong TCP Payload, và cờ PSH cho phép gửi dữ liệu tức thời mà không cần chờ đệm, trong khi cờ ACK xác nhận dữ liệu trước đó đã được nhận chính xác.

Local Area Connection 10

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

(ip.addr == 192.168.137.225 and tcp.port == 8883) and tls.record.content_type == 23

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
2	0.198542	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
5	15.000934	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
6	15.196234	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
39	22.890406	192.168.137.225	46.137.47.218	TLSv...	156	Application Data
43	23.128091	46.137.47.218	192.168.137.225	TLSv...	87	Application Data
46	23.198548	192.168.137.225	46.137.47.218	TLSv...	106	Application Data
47	23.416335	46.137.47.218	192.168.137.225	TLSv...	88	Application Data
50	38.204031	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
51	38.420230	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
54	53.199227	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
55	53.417674	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
57	68.207163	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
58	68.423404	46.137.47.218	192.168.137.225	TLSv...	85	Application Data
60	79.001022	192.168.137.225	46.137.47.218	TLSv...	173	Application Data
63	83.660596	192.168.137.225	46.137.47.218	TLSv...	85	Application Data
65	83.877783	46.137.47.218	192.168.137.225	TLSv...	85	Application Data

Transmission Control Protocol, Src Port: 49917, Dst Port: 8883, Seq: 1, Ack: 1, Len: 31

Source Port: 49917
Destination Port: 8883
[Stream index: 0]
[Conversation completeness: Incomplete (60)]
[TCP Segment Len: 31]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 407059153
[Next Sequence Number: 32 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 1527416820
0101 = Header Length: 20 bytes (5)
Flags: 0x018 (PSH, ACK)
Window: 5049
[Calculated window size: 5049]
[Window size scaling factor: -1 (unknown)]
Checksum: 0x71be [unverified]

c) Phân tích gói tin TCP trong quá trình truyền thông giữa ESP32 và HiveMQ Broker

Về mặt định lượng, các thử nghiệm thực tế cho thấy độ trễ trung bình khi truyền bản tin MQTT từ ESP32 đến Broker HiveMQ là tương đối nhỏ trong mạng WiFi ổn định.

69	98.896070	46.137.47.218	192.168.137.225	TLSv1.2	85 Application Data
75	113.660581	192.168.137.225	46.137.47.218	TLSv1.2	85 Application Data
76	113.877030	46.137.47.218	192.168.137.225	TLSv1.2	85 Application Data
79	128.664214	192.168.137.225	46.137.47.218	TLSv1.2	85 Application Data
80	128.881771	46.137.47.218	192.168.137.225	TLSv1.2	85 Application Data
83	139.012745	192.168.137.225	46.137.47.218	TLSv1.2	173 Application Data
86	144.177713	192.168.137.225	46.137.47.218	TLSv1.2	85 Application Data
88	144.393824	46.137.47.218	192.168.137.225	TLSv1.2	85 Application Data


```

[Next Sequence Number: 912      (relative sequence number)]
Acknowledgment Number: 3336      (relative ack number)
Acknowledgment number (raw): 3929315584
0101 .... = Header Length: 20 bytes (5)
  Flags: 0x018 (PSH, ACK)
  Window: 5142
  [Calculated window size: 5142]
  [Window size scaling factor: -2 (no window scaling used)]
  Checksum: 0xfab8 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
    [Time since first frame in this TCP stream: 107.033624000 seconds]
    [Time since previous frame in this TCP stream: 14.450667000 seconds]
  [SEQ/ACK analysis]
    TCP payload (31 bytes)
  Transport Layer Security
  
```

79	128.664214	192.168.137.225	46.137.47.218	TLSv1.2	85 Application Data
80	128.881771	46.137.47.218	192.168.137.225	TLSv1.2	85 Application Data
83	139.012745	192.168.137.225	46.137.47.218	TLSv1.2	173 Application Data
86	144.177713	192.168.137.225	46.137.47.218	TLSv1.2	85 Application Data
88	144.393824	46.137.47.218	192.168.137.225	TLSv1.2	85 Application Data


```

[Next Sequence Number: 3367      (relative sequence number)]
Acknowledgment Number: 912      (relative ack number)
Acknowledgment number (raw): 778876924
0101 .... = Header Length: 20 bytes (5)
  Flags: 0x018 (PSH, ACK)
  Window: 61816
  [Calculated window size: 61816]
  [Window size scaling factor: -2 (no window scaling used)]
  Checksum: 0x6b86 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  [Timestamps]
    [Time since first frame in this TCP stream: 107.251181000 seconds]
    [Time since previous frame in this TCP stream: 0.217557000 seconds]
  [SEQ/ACK analysis]
    TCP payload (31 bytes)
  Transport Layer Security
  
```



3.7 Thiết kế website



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

3.7.1. Cài đặt phần mềm và cấu hình

Môi trường xây dựng web cho mô hình AquaNovo

```
Flask==3.0.0
firebase-admin==6.5.0
python-dotenv==1.0.1
Flask-Cors==4.0.0
google-cloud-firestore>=2.16
paho-mqtt>=1.6.1
APScheduler>=3.10.4
```

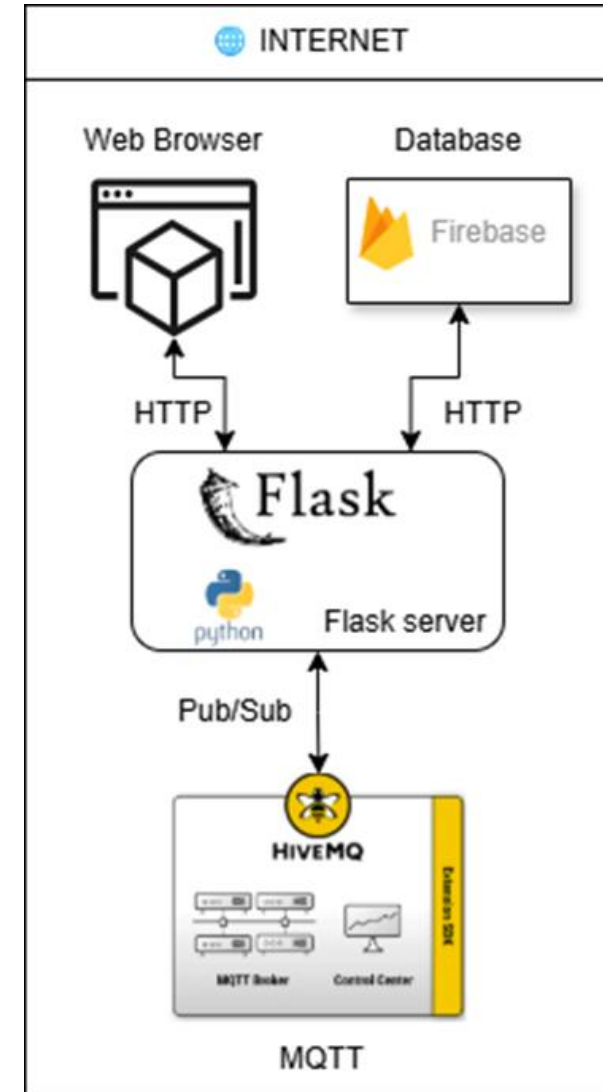
- Flask: Backend chính: tạo API, routing, render dashboard & feed timer
- Firebase-admin: Kết nối Firestore để đọc/ghi dữ liệu cảm biến & lịch cho ăn
- python-dotenv: Quản lý cấu hình bảo mật (MQTT, Firestore key, API token)
- Flask-CORS: Giúp JS frontend (Chart.js, fetch API) gọi Flask API
- google-cloud-firestore: Lưu dữ liệu đo, trạng thái thiết
- paho-mqtt: Publish/subscribe dữ liệu giữa Flask ↔ ESP32 qua HiveMQ Cloud
- APScheduler: Tự động gửi lệnh cho ăn theo giờ định sẵn

3.7.1. Cài đặt phần mềm và cấu hình

Môi trường xây dựng web cho mô hình AquaNovo

```
Flask==3.0.0  
firebase-admin==6.5.0  
python-dotenv==1.0.1  
Flask-Cors==4.0.0  
google-cloud-firestore>=2.16  
paho-mqtt>=1.6.1  
APScheduler>=3.10.4
```

Sơ đồ kết nối Backend, Frontend, Firebase và MQTT





3.7 Thiết kế website



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

3.7.2. Cấu trúc tổ chức chương trình Cấu trúc thư mục chính và thiết lập môi trường

```
# Database
FIREBASE_CREDENTIALS=C:\Users\ASUS\
Documents\GitHub\IoT_AquaNova\IoT_A
quaNova\serviceAccount.json
FIREBASE_DB=firestore
SECRET_KEY=AquaNovaSecret123

# HiveMQ Cloud MQTT
MQTT_HOST=d2b2655505394c9b8e88c06c8
d30d912.s1.eu.hivemq.cloud
MQTT_PORT=8883
MQTT_USER=aquanova_user
MQTT_PASS=12345678Aqua
MQTT_TOPIC=aquanova/telemetry
```

```
aquanova/
├── app.py          # Khởi tạo Flask app, load config, đăng ký blueprint
├── blueprints/
│   ├── dashboard/routes.py      # Cung cấp API dữ liệu (temperature,
│   │                             turbidity, feed)
│   ├── control/routes.py        # Điều khiển cho ăn, tạo/xóa lịch
│   ├── control/scheduler.py     # Lập lịch tự động = và gửi lệnh feed
│   ├── telemetry/routes.py      # Ghi nhận dữ liệu từ MQTT (telemetry)
│   └── admin/routes.py          # Giao diện quản trị
├── firebase_admin_init.py       # Kết nối Firestore
├── MCU                        # STM32, ESP32
├── mqtt/
│   ├── listener.py             # sub
│   └── publisher.py
├── static /
│   ├── css/styles.css          # Giao diện hiển thị
│   └── js/dashboard.js         # Xử lý cập nhật dữ liệu & biểu đồ
└── templates/
    ├── index.html              # Dashboard tổng quan
    ├── feedtimer.html          # Giao diện đặt lịch cho ăn
    ├── temperature.html        # Biểu đồ nhiệt độ
    ├── turbidity.html          # Biểu đồ độ đục
    └── admin.html               # Trang quản trị
```



3.7 Thiết kế website



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

3.7.3. Frontend

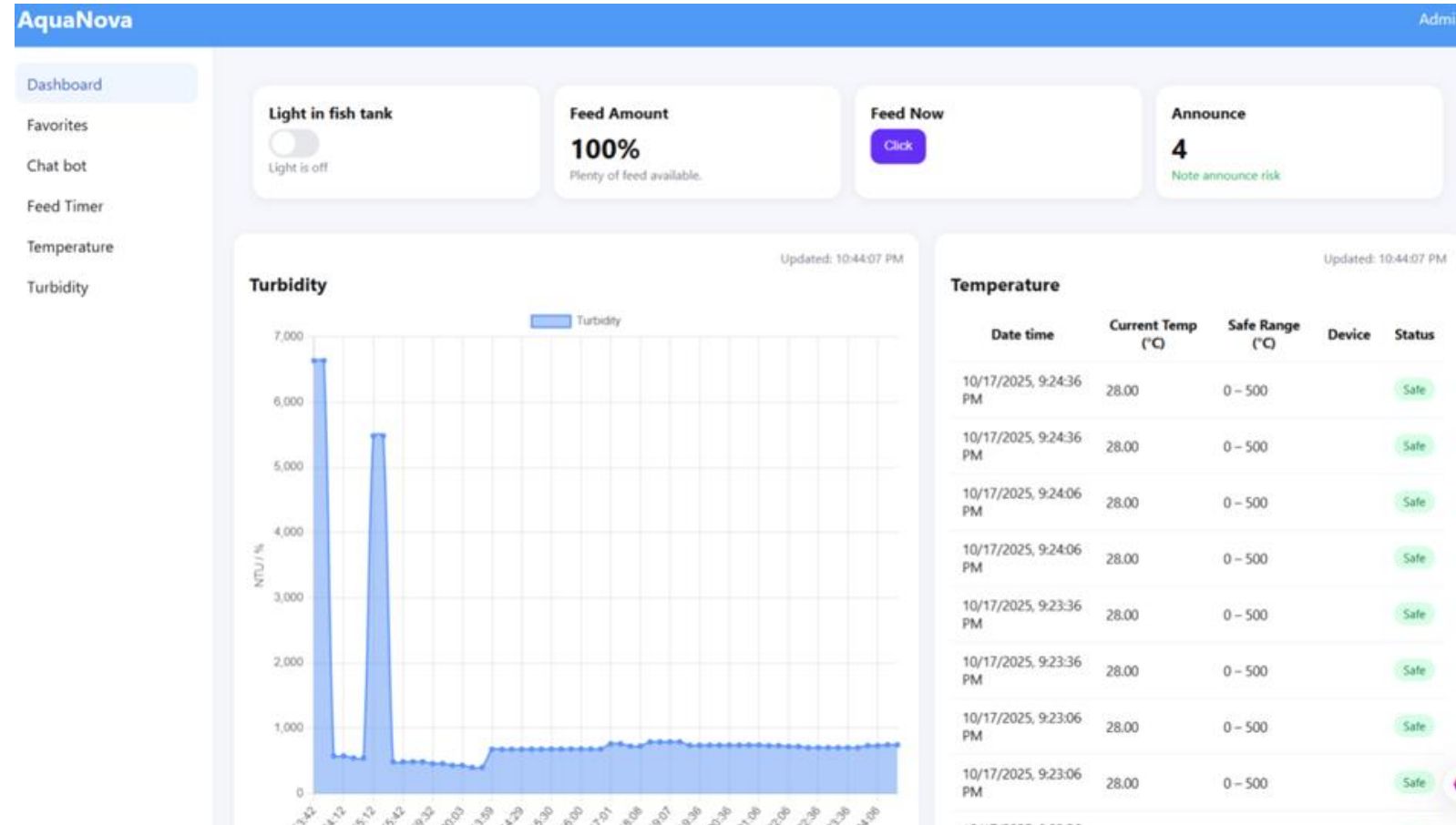
a) Tổng quan giao diện

Tất cả các trang web đều tuân theo bố cục hai phần chính:

- Sidebar (thanh điều hướng): chứa các liên kết đến các module chính
- Main Content (nội dung chính): hiển thị dữ liệu, biểu đồ và bảng tương ứng theo từng module

Khai báo HTML và thư viện

- CSS: định dạng bố cục (màu nền, lưới, thẻ card, sidebar).
- Chart.js: thư viện biểu đồ hiển thị dữ liệu cảm biến.



Bố cục giao diện của trang web người dùng chính



3.7 Thiết kế website



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

3.7.3. Frontend

a) Tổng quan giao diện

HTML cấu trúc tổng quát:

- aside: tạo thanh điều hướng bên trái (menu liên kết đến các module).
- main: vùng nội dung chính chứa thẻ thông tin (cards) và biểu đồ + bảng dữ liệu.
- card: hiển thị dữ liệu

```
:root {  
  --brand: #0ea5e9;    /* Xanh */  
  --muted: #6b7280;    /* Ghi nhạt */  
  --bg: #f6f7fb;       /* Nền sáng */  
}  
.card {  
  background: #fff;  
  border-radius: 14px;  
  box-shadow: 0 2px 12px rgba(0,0,0,.05);  
}
```

styles.css để thống nhất màu chủ đạo:

```
<div class="layout">  
  <aside>  
    <nav class="menu">  
      <a href="/dashboard" class="tab-link active">Dashboard</a>  
      <a href="/temperature" class="tab-link">Temperature</a>  
      <a href="/turbidity" class="tab-link">Turbidity</a>  
      <a href="/feedtimer" class="tab-link">Feed Timer</a>  
    </nav>  
  </aside>  
  <main>  
    <!-- Nội dung từng module -->  
  </main>  
</div>
```

```
<div class="layout">  
  <aside>...</aside>  
  <main>...</main>  
</div>  
<div class="cards">  
  <div class="card">Light in fish tank</div>  
  <div class="card">Feed Amount</div>  
  <div class="card">Feed Now</div>  
  <div class="card">Announce</div>  
</div>
```


3.7.3. Frontend

b) Phân tích mã nguồn giao diện Dashboard

Tập trung vào JavaScript (FE gọi BE)

- Điều khiển đèn hồ cá (Light Control): gửi yêu cầu POST đến API /control/light

```
lightSwitch.addEventListener('click', () => {  
  const isOn = lightSwitch.classList.toggle('on');  
  toggleLight(isOn);  
});
```

Gửi lệnh cho ăn ngay: hệ thống gửi yêu cầu POST đến Flask.

```
const res = await fetch('/control/feed-now', {  
  method: 'POST',  
  body: JSON.stringify({ amount: 20 })  
});
```

Biểu đồ độ đục (Turbidity Chart): Hàm ensureChart() khởi tạo biểu đồ đường. Dữ liệu được lấy từ API

/dashboard/latest?n=60, mỗi mẫu gồm:

- ts: thời gian đo
- turbidity: giá trị độ đục
- Nhãn trục X hiển thị theo thời gian (UTC+7).
- Biểu đồ được cập nhật tự động mỗi 10 phút (setInterval(loadAll, 600000)).

```
turbChart = new Chart(ctx, {  
  type: 'line',  
  data: { labels, datasets: [{ label: 'Turbidity',  
    data, borderWidth: 2, tension: .25, fill: true }] }  
});
```

3.7.3. Frontend

b) Phân tích mã nguồn giao diện Dashboard

Tập trung vào JavaScript (FE gọi BE)

- Điều khiển đèn hồ cá (Light Control): gửi yêu cầu POST đến API /control/light

```
lightSwitch.addEventListener('click', () => {  
    const isOn = lightSwitch.classList.toggle('on');  
    toggleLight(isOn);  
});
```

Gửi lệnh cho ăn ngay: hệ thống gửi yêu cầu POST đến Flask.

```
const res = await fetch('/control/feed-now', {  
    method: 'POST',  
    body: JSON.stringify({ amount: 20 })  
});
```

Bảng nhiệt độ (Temperature Table):

Hiển thị 10 mẫu dữ liệu gần nhất gồm thời gian, nhiệt độ, thiết bị đo, và trạng thái an toàn. Nếu giá trị nằm ngoài khoảng [SAFE_MIN, SAFE_MAX]

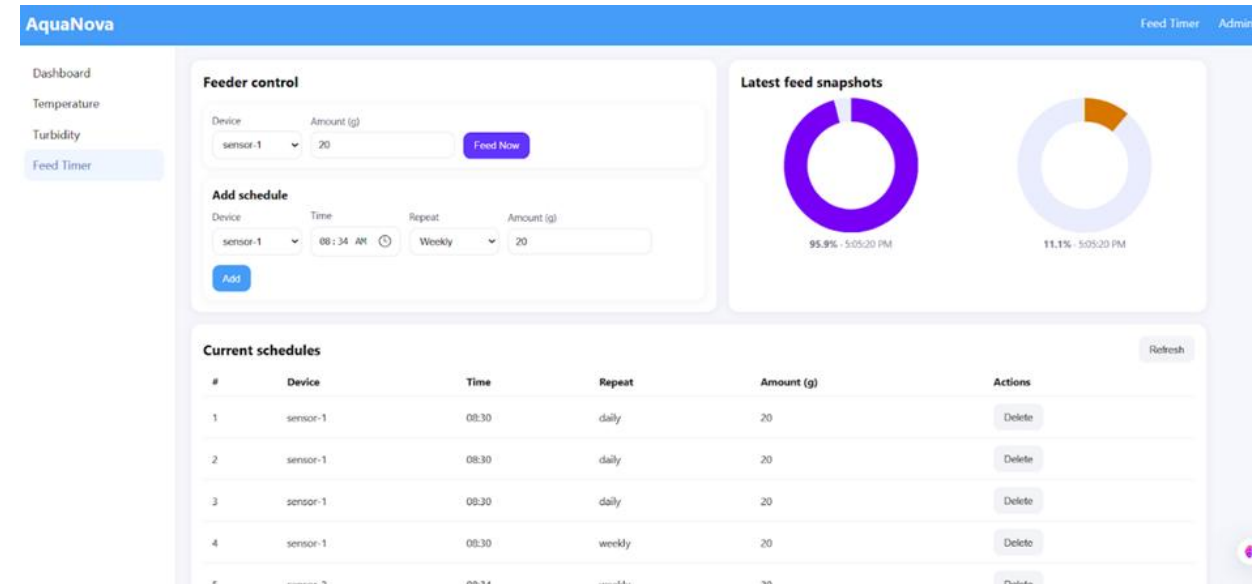
```
items.slice(0,10).forEach(r=>{  
    const t = Number(r.temperature);  
    const badge = `<span class="badge  
    ${ok?'safe':'alert'}">${ok?'Safe':'Alert'}</span>`;   
});
```

3.7.3. Frontend

c) Phân tích mã nguồn giao diện Feed Timer

- Trang feed_timer.html là thành phần điều khiển mô hình AquaNova:
- Điều khiển cho cá ăn ngay lập tức với lượng tùy chọn.
- Tạo và quản lý lịch cho ăn tự động theo ngày, theo tuần hoặc hàng ngày.
- Xem danh sách các lịch hiện tại và xóa khi cần.
- Theo dõi tình trạng thức ăn còn lại thông qua biểu đồ dạng Pie chart.
- Trang này hoạt động song song với Flask backend, giao tiếp qua các API:

/control/feed-now, /control/schedule, /control/schedules



Giao diện điều khiển cho ăn



3.7 Thiết kế website



3.7.3. Frontend

c) Phân tích mã nguồn giao diện Feed Timer

Bên trái (Feeder Control)

Phần Feeder Control gồm 2 nhóm:

- Feed Now: cho ăn ngay.
- Add Schedule: tạo lịch định sẵn.

```
<!-- Feed Now -->
<input type="number" id="fnAmount" value="20">
<button id="feedNowBtn">Feed Now</button>
<!-- Add Schedule -->
<input type="date" id="scDate">
<input type="time" id="scTime"><select
id="scRepeat">Once / Daily / Weekly</select>
<input type="number" id="scAmount" value="20">
<button id="addScheduleBtn">Add</button>
```

Bên phải (Feed Status – biểu đồ): Biểu đồ tròn (Pie Chart) thể hiện phần trăm thức ăn còn lại trong thùng.

```
<canvas id="feedPie1"></canvas>
<b id="feedPie1Num">--%</b>
<div id="feedPie1Warn" class="tag"
style="display:none">Low feed (&lt;10%)</div>
```



3.7 Thiết kế website



3.7.3. Frontend

c) Phân tích mã nguồn giao diện Feed Timer

Khai báo API

```
const FEED_NOW_API = '/control/feed-now';  
const SCHEDULE_API = '/control/schedule';  
const SCHEDULES_API = '/control/schedules';  
\
```

Xóa lịch đã đặt: Gửi request DELETE

/control/schedules/<id>

```
async function deleteSchedule(id){  
    const r = await fetch(`${SCHEDULES_API}/${id}`, {  
        method: 'DELETE' });  
}
```

Feed Now – Gửi lệnh cho ăn ngay

```
document.getElementById('feedNowBtn').onclick =  
async ()=>{  
    const amount =  
    Number(document.getElementById('fnAmount').value);  
    const msg =  
    document.getElementById('feedNowMsg');  
    msg.textContent = 'Sending...';  
    const r = await fetch(FEED_NOW_API, {  
        method: 'POST',  
        headers: { 'Content-Type': 'application/json' },  
        body: JSON.stringify({ amount })  
    });  
}
```

3.7.3. Frontend

c) Phân tích mã nguồn giao diện Feed Timer

Khai báo API

```
const FEED_NOW_API = '/control/feed-now';  
const SCHEDULE_API = '/control/schedule';  
const SCHEDULES_API = '/control/schedules';  
\
```

Xóa lịch đã đặt: Gửi request DELETE
/control/schedules/<id>

```
async function deleteSchedule(id){  
    const r = await fetch(`${SCHEDULES_API}/${id}`, {  
        method: 'DELETE' });  
}
```

Thêm lịch cho ăn mới

```
async function addSchedule(){  
    const date = document.getElementById('scDate').value;  
    const time = document.getElementById('scTime').value;  
    const repeat =  
document.getElementById('scRepeat').value;  
    const amount =  
Number(document.getElementById('scAmount').value || 0);  
    const msg = document.getElementById('addScheduleMsg');  
  
    const r = await fetch(SCHEDULE_API, {  
        method: 'POST',  
        headers: { 'Content-Type': 'application/json' },  
        body: JSON.stringify({ date, time, repeat, amount })  
    });  
}
```


3.7.3. Frontend

c) Phân tích mã nguồn giao diện Feed Timer

Khai báo API

```
const FEED_NOW_API = '/control/feed-now';  
const SCHEDULE_API = '/control/schedule';  
const SCHEDULES_API = '/control/schedules';  
\
```

Xóa lịch đã đặt: Gửi request DELETE

/control/schedules/<id>

```
async function deleteSchedule(id) {  
    const r = await fetch(`${SCHEDULES_API}/${id}`, {  
        method: 'DELETE' });  
}
```

Hiển thị danh sách lịch hiện tại

```
async function refreshSchedules(){  
    const r = await fetch(SCHEDULES_API);  
    const j = await r.json();  
    const list = j.items || [];  
    list.forEach((s,idx)=>{  
        const tr = document.createElement('tr');  
        tr.innerHTML = `  
            <td>${idx+1}</td>  
            <td>${s.date} || '-'</td>  
            <td>${s.time} || ''</td>  
            <td>${s.repeat}</td>  
            <td>${s.amount}</td>  
            <td><button  
onclick="deleteSchedule('${s.id}')">Delete</button></td>`;  
        });  
    }  
    {  
        "items": [  
            { "id": "abcd123", "date": "2025-10-23", "time": "08:30",  
            "repeat": "daily", "amount": 20 }  
        ]  
    }  
}
```

3.7.3. Frontend

d) Phân tích mã nguồn giao diện Temperature

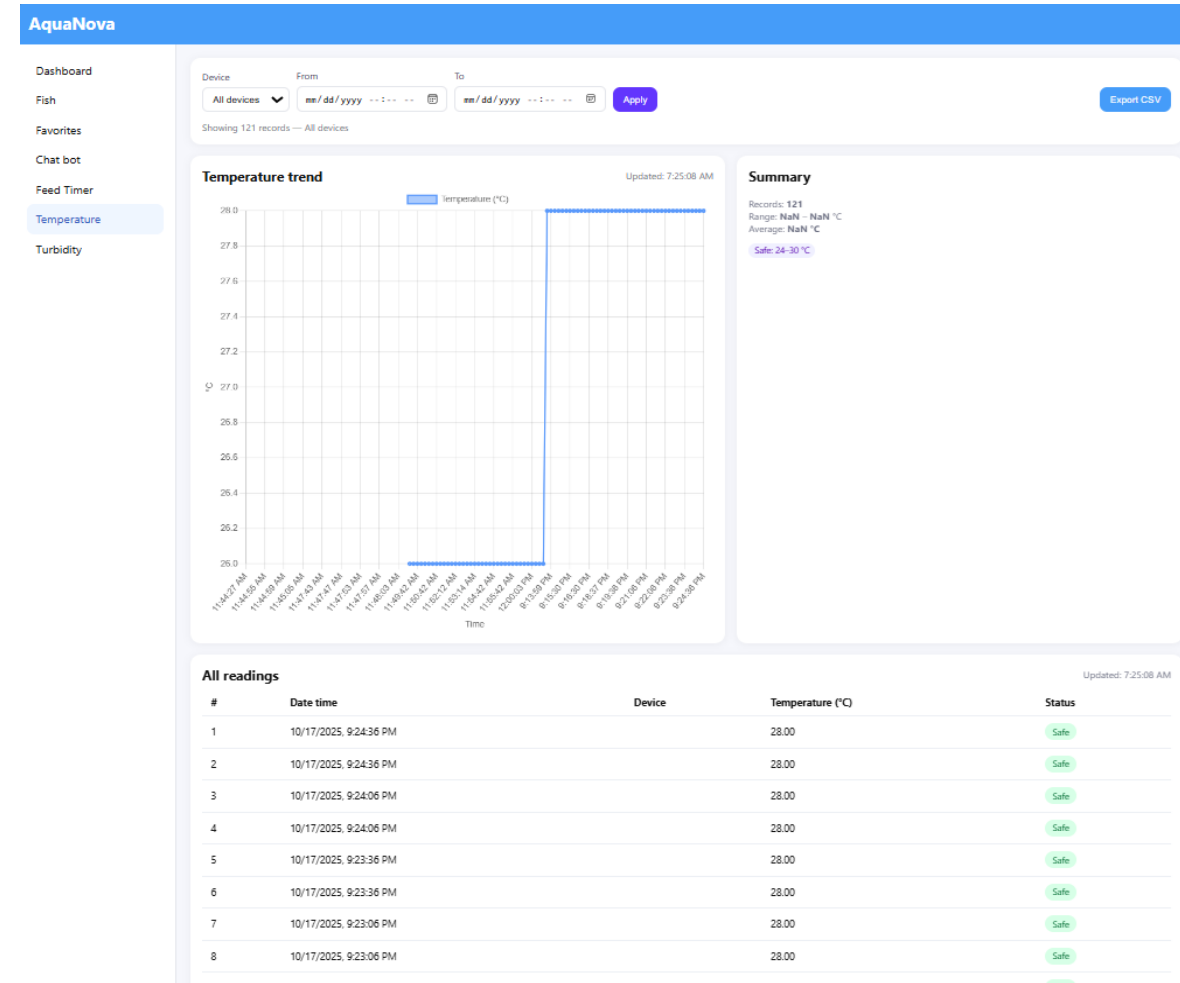
Lấy dữ liệu từ API:

- Gọi API /dashboard/latest (Flask backend) để nhận tối đa FETCH_N = 500 bản ghi gần nhất.

Mỗi bản ghi có dạng:

```
{"ts":"2025-10-23T10:00:00Z","temperature":27.5}
```

```
const res = await
fetch(`/dashboard/latest?n=${FETCH_N}`);
const json = await res.json();
allItems = json.items || [];
```



Giao diện hiển thị nhiệt độ



3.7 Thiết kế website



3.7.3. Frontend

d) Phân tích mã nguồn giao diện Temperature
Lấy dữ liệu từ API:

Xuất dữ liệu ra CSV

```
document.getElementById('exportBtn').onclick = ()  
=> {  
    const lines = [['timestamp', 'turbidity']];  
    filtered.forEach(r=> lines.push([r.ts,  
r.turbidity]));  
    ...  
    a.download = `temperature_${Date.now()}.csv`;  
};
```

Lọc dữ liệu và hiển thị: Gọi renderChart() để vẽ lại biểu đồ, và renderTable() để cập nhật bảng.

```
function applyFilter() {  
    const dev =  
document.getElementById('deviceSelect').value;  
    const from = new  
Date(document.getElementById('fromDt').value);  
    const to = new  
Date(document.getElementById('toDt').value);  
    ...  
    filtered = allItems.filter(...);  
    renderChart();  
    renderTable();  
}
```



3.7 Thiết kế website



3.7.3. Frontend

d) Phân tích mã nguồn giao diện Temperature

Lấy dữ liệu từ API:

- Gọi API /dashboard/latest (Flask backend) để nhận tối đa
FETCH_N = 500 bản ghi gần nhất.

Mỗi bản ghi có dạng:

```
{"ts":"2025-10-23T10:00:00Z","temperature":27.5}
```

```
const res = await  
fetch(`/dashboard/latest?n=${FETCH_N}`);  
const json = await res.json();  
allItems = json.items || [];
```

Xuất dữ liệu ra CSV

```
document.getElementById('exportBtn').onclick = ()  
=> {  
    const lines = [['timestamp', 'turbidity']];  
    filtered.forEach(r=> lines.push([r.ts,  
r.turbidity]));  
    ...  
    a.download = `temperature_${Date.now()}.csv`;  
};
```

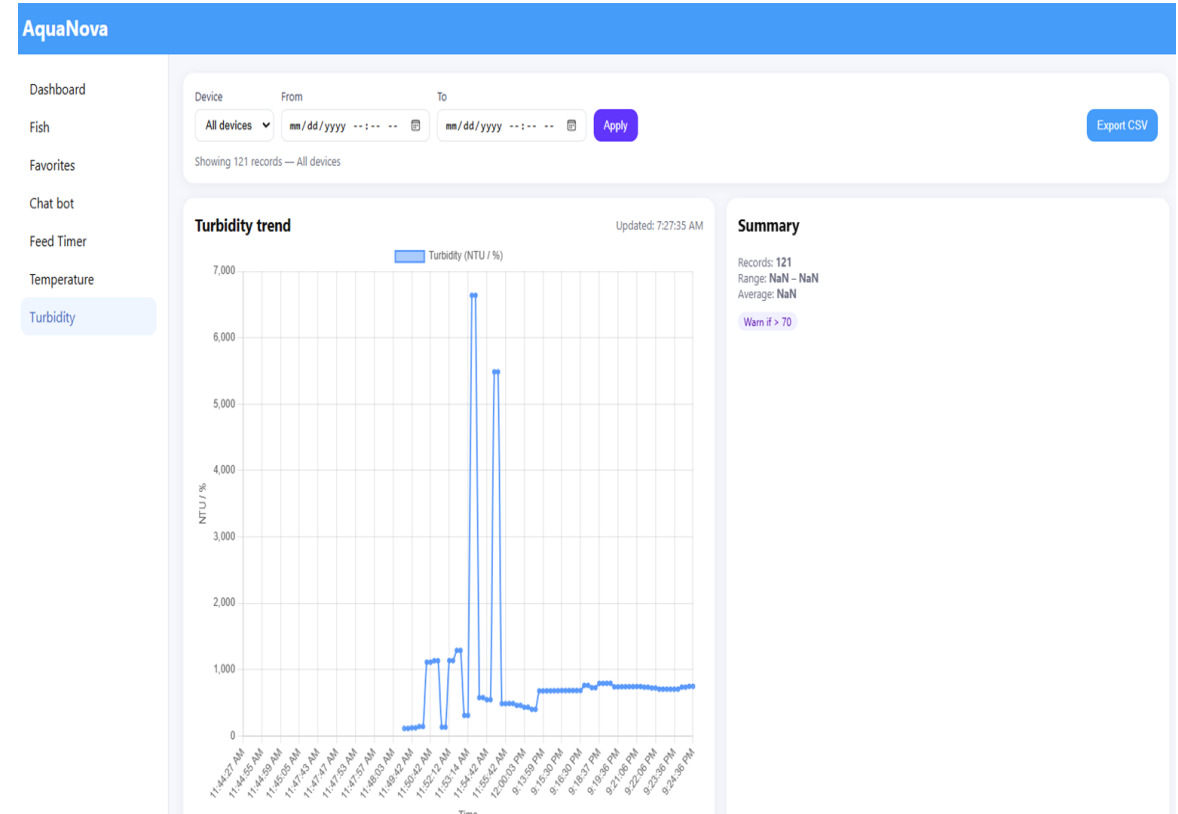
3.7.3. Frontend

d) Phân tích mã nguồn giao diện Turbidity

Lấy dữ liệu từ API:

- Thống kê tổng quan: Tính toán nhanh min, max, trung bình cho giai đoạn hiển thị.

```
const min = Math.min(...data), max =
Math.max(...data);
const avg = data.reduce((a,b)=>a+b,0)/data.length;
document.getElementById('summaryBox').innerHTML =
`Records: <b>${filtered.length}</b><br>
Range: <b>${min}</b> - <b>${max}</b><br>
Average: <b>${avg}</b>`;
```



Giao diện hiển thị độ đục

3.7.3. Frontend

d) Tương tác Front end với Flask API

Hoạt động	API Flask	Phương thức	Kết quả
Lấy dữ liệu cảm biến	/dashboard/latest	GET	Trả về danh sách các bản ghi mới nhất
Cho ăn ngay lập tức	/control/feed-now	POST	Gửi lệnh MQTT → thiết bị thực thi
Tạo lịch cho ăn	/control/schedule	POST	Lưu thông tin lịch vào Firestore
Lấy danh sách lịch	/control/schedules	GET	Hiển thị bảng “Current Schedules”
Xóa lịch	/control/schedules/<id>	DELETE	Xóa lịch tương ứng khỏi Firestore

3.7.4. Backend

- Cầu nối trung tâm giữa giao diện web, cơ sở dữ liệu Firebase, và thiết bị IoT (ESP32 qua MQTT).
- Tiếp nhận các yêu cầu từ người dùng (qua HTTP request).
- Gửi lệnh điều khiển (qua MQTT publish) hoặc truy vấn dữ liệu cảm biến (qua Firestore).
- Quản lý lịch tự động cho ăn (Feed Scheduler) sử dụng APScheduler.a) Cấu trúc Flask và Blueprints

Thành phần	Chức năng
Flask Core	Tổ chức web server, routing, template rendering
Blueprints	Tách chức năng logic (telemetry, control, dashboard, admin)
Firestore (Firebase)	Lưu trữ dữ liệu thời gian thực và lịch sử
MQTT Listener	Nhận dữ liệu cảm biến từ ESP32
MQTT Publisher	Gửi lệnh điều khiển Feed Now, bật/tắt đèn, v.v.
APScheduler	Lập lịch gửi lệnh định kỳ
CORS	Cho phép frontend gọi API từ domain khác
Templates & Static	Giao diện HTML/CSS/JS hiển thị cho người dùng

Bảng chi tiết vào
trò module
Backend



3.7 Thiết kế website



3.7.4. Backend

1. Cấu trúc tổng thể

```
from flask import Flask, render_template
from flask_cors import CORS
from config import Config
from firebase_admin_init import init_firebase
from blueprints.telemetry.routes import
telemetry_bp
from blueprints.control.routes import control_bp
from blueprints.control.scheduler import
start_scheduler
from blueprints.admin.routes import admin_bp
from blueprints.dashboard.routes import
dashboard_bp
from mqtt.listener import start_mqtt_background
```

2. Hàm create_app() – Khởi tạo ứng dụng Tạo Flask app và định nghĩa thư mục:

- static/ → chứa file CSS, JS, ảnh.
- templates/ → chứa các file HTML giao diện.
- Nạp cấu hình từ lớp Config.
- Kích hoạt CORS để cho phép truy cập từ web

client.

```
def create_app():
    app = Flask(__name__, static_folder="static",
template_folder="templates")
    app.config.from_object(Config)
    CORS(app) // ngăn truy cập trái phép
```



3.7 Thiết kế website



3.7.4. Backend

1. Cấu trúc tổng thể

```
from flask import Flask, render_template
from flask_cors import CORS
from config import Config
from firebase_admin_init import init_firebase
from blueprints.telemetry.routes import
telemetry_bp
from blueprints.control.routes import control_bp
from blueprints.control.scheduler import
start_scheduler
from blueprints.admin.routes import admin_bp
from blueprints.dashboard.routes import
dashboard_bp
from mqtt.listener import start_mqtt_background
```

2. Hàm create_app() – Khởi tạo ứng dụng Tạo Flask app và định nghĩa thư mục:

- static/ → chứa file CSS, JS, ảnh.
- templates/ → chứa các file HTML giao diện.
- Nạp cấu hình từ lớp Config.
- Kích hoạt CORS để cho phép truy cập từ web

client.

```
def create_app():
    app = Flask(__name__, static_folder="static",
template_folder="templates")
    app.config.from_object(Config)
    CORS(app) // ngăn truy cập trái phép
```



3.7 Thiết kế website



3.7.4. Backend

2. Hàm `create_app()` – Khởi tạo ứng dụng Tạo Flask app và định nghĩa thư mục:

- `static/` → chứa file CSS, JS, ảnh.
- `templates/` → chứa các file HTML giao diện.
- Nạp cấu hình từ lớp Config.
- Kích hoạt CORS để cho phép truy cập từ web

client.

```
def create_app():  
    app = Flask(__name__, static_folder="static",  
template_folder="templates")  
    app.config.from_object(Config)  
    CORS(app) // ngăn truy cập trái phép
```

```
import os  
from dotenv import load_dotenv  
load_dotenv()  
  
class Config:  
    SECRET_KEY = os.getenv("SECRET_KEY", "dev")  
  
    # Firebase  
    FIREBASE_CREDENTIALS =  
os.getenv("FIREBASE_CREDENTIALS")  
    FIREBASE_DB = os.getenv("FIREBASE_DB", "firestore")  
    # MQTT  
    MQTT_HOST = os.getenv("MQTT_HOST", "localhost")  
    MQTT_PORT = int(os.getenv("MQTT_PORT", "1883"))  
    MQTT_USER = os.getenv("MQTT_USER", "") #  
optional  
    MQTT_PASS = os.getenv("MQTT_PASS", "") #  
optional  
    MQTT_TOPIC = os.getenv("MQTT_TOPIC",  
"aquanova/devices/+/telemetry")
```



3.7 Thiết kế website



3.7.4. Backend

3. Khởi tạo firebase và

Đăng ký các Blueprint (Module API)

```
init_firebase()  
  
app.register_blueprint(telemetry_bp, url_prefix="/telemetry")  
app.register_blueprint(control_bp, url_prefix="/control")  
app.register_blueprint(admin_bp, url_prefix="/admin")  
app.register_blueprint/dashboard_bp, url_prefix="/dashboard")
```

4. Định nghĩa các route giao diện

```
@app.get("/")  
def home():  
    return render_template("index.html")  
  
@app.get("/admin")  
def admin_page():  
    return render_template("admin.html")  
  
@app.get("/temperature")  
def temperature_page():  
    return render_template("temperature.html")  
  
@app.get("/turbidity")  
def turbidity_page():  
    return render_template("turbidity.html")  
  
@app.get("/feedtimer")  
def feed_timer_page():  
    return render_template("feedtimer.html")
```



3.7 Thiết kế website



3.7.4. Backend

5. Khởi động MQTT Listener & APScheduler

a. start_mqtt_background(app)

Tạo thread chạy nền kết nối MQTT broker

Lắng nghe các topic:

- "aquanova/telemetry" → dữ liệu cảm biến gửi về.

Khi có dữ liệu, Flask tự động cập nhật lên Firestore.

b. start_scheduler()

- Kích hoạt bộ lập lịch APScheduler – cho phép tự động thực hiện lệnh cho ăn đúng giờ.
- Mỗi khi người dùng thêm lịch qua /control/schedule, hệ thống tạo job mới trong scheduler.

```
start_mqtt_background(app)
with app.app_context():
    start_scheduler()
Flask server chạy trên cổng mặc định (5000)
```

6. Khởi chạy ứng dụng

```
if __name__ == "__main__":
    app = create_app()
    app.run(debug=True, use_reloader=False)
```




3.7 Thiết kế website



3.7.4. Backend

5. Khởi động MQTT Listener & APScheduler

a. start_mqtt_background(app)

Tạo thread chạy nền kết nối MQTT broker

Lắng nghe các topic:

- "aquanova/telemetry" → dữ liệu cảm biến gửi về.

Khi có dữ liệu, Flask tự động cập nhật lên Firestore.

b. start_scheduler()

- Kích hoạt bộ lập lịch APScheduler – cho phép tự động thực hiện lệnh cho ăn đúng giờ.
- Mỗi khi người dùng thêm lịch qua /control/schedule, hệ thống tạo job mới trong scheduler.

```
start_mqtt_background(app)
with app.app_context():
    start_scheduler()
Flask server chạy trên cổng mặc định (5000)
```

6. Khởi chạy ứng dụng

```
if __name__ == "__main__":
    app = create_app()
    app.run(debug=True, use_reloader=False)
```

3.7.4. Backend

7. Các API thiết lập

a) Module Dashboard (giao tiếp với giao diện)

Các chức năng chính bao gồm:

- Truy xuất dữ liệu cảm biến mới nhất để hiển thị trên Dashboard.
- Cung cấp thông tin tổng quan của hệ thống
- Phát hiện và đếm số bản ghi bất thường để cảnh báo.
- Cung cấp bản ghi gần nhất phục vụ việc cập nhật tức thời trên web
- API /dashboard/latest – lấy danh sách bản ghi mới nhất

```
@dashboard_bp.get("/latest")
def latest():
    n = int(request.args.get("n", 100))
    db = firestore.client()
    docs = (
        db.collection("readings")
        .order_by("ts",
direction=firestore.Query.DESCEDENDING)
        .limit(n)
        .stream() )
    items = []
    for doc in docs:
        d = doc.to_dict()
        items.append({
            "ts": d.get("ts"),
            "temperature": d.get("temperature"),
            "turbidity": d.get("turbidity"),
            "feed": d.get("feed"),
            "thoi_gian": d.get("thoi_gian")
        })
    return jsonify({"items": items})
```

3.7.4. Backend

7. Các API thiết lập

a) Module Dashboard (giao tiếp với giao diện)

Các chức năng chính bao gồm:

- Truy xuất dữ liệu cảm biến mới nhất để hiển thị trên Dashboard.
- Cung cấp thông tin tổng quan của hệ thống
- Phát hiện và đếm số bản ghi bất thường để cảnh báo.
- Cung cấp bản ghi gần nhất phục vụ việc cập nhật tức thời trên web
- API /dashboard/latest – lấy danh sách bản ghi mới nhất

```
@dashboard_bp.get("/latest")
def latest():
    n = int(request.args.get("n", 100))
    db = firestore.client()
    docs = (
        db.collection("readings")
        .order_by("ts",
            direction=firestore.Query.DESCEDING)

{
    "items": [
        {"ts": "2025-10-23T10:35:00Z", "temperature": 28.3,
"turbidity": 56.2, "feed": 40}
    ]
}

        items.append({
            "ts": d.get("ts"),
            "temperature": d.get("temperature"),
            "turbidity": d.get("turbidity"),
            "feed": d.get("feed"),
            "thoi_gian": d.get("thoi_gian")
        })
    return jsonify({"items": items})
```



3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

a) Module Dashboard (giao tiếp với giao diện)

- API /dashboard/announce-count – phát hiện và đếm cảnh báo
- API /dashboard/summary – thống kê tổng quan

```
@dashboard_bp.get("/summary")
def summary():
    db = firestore.client()
    ponds = db.collection("readings").stream()
    devices = db.collection("feed_logs").stream()
    pond_count = len(list(ponds))
    device_count = len(list(devices))
    return jsonify({
        "ponds": pond_count,
        "devices": device_count
    })
```

```
@dashboard_bp.get("/announce-count")
def announce_count():
    db = firestore.client()
    q = (
        db.collection("readings")
        .order_by("ts",
direction=firestore.Query.DESCENDING)
        .limit(60)
    )
    cnt = 0
    for d in q.stream():
        r = d.to_dict()
        t = r.get("temperature")
        turb = r.get("turbidity")
        if t is not None and (t < 24 or t > 30):
            cnt += 1
        if turb is not None and turb > 200:
            cnt += 1
    return jsonify({"count": cnt})
```



3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

a) Module Control (điều khiển từ Web)

File control/routes.py chịu trách nhiệm nhận yêu cầu điều khiển từ web và gửi lệnh qua MQTT, cụ thể:

- Gửi lệnh điều khiển đến thiết bị IoT thông qua giao thức MQTT (bật/tắt đèn, cho cá ăn)
- Cung cấp các API RESTful để giao diện web (Feed Timer Page) có thể gửi lệnh điều khiển, thêm, xóa hoặc hiển thị danh sách lịch.

1. Cấu trúc tổng

```
from flask import Blueprint, request, jsonify,
current_app
import paho.mqtt.client as mqtt
import json, uuid, time
from firebase_admin import firestore

control_bp = Blueprint("control_bp", __name__)
_mqtt_pub_client = None
```



3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

a) Module Control (điều khiển từ Web)

File control/routes.py chịu trách nhiệm nhận yêu cầu điều khiển từ web và gửi lệnh qua MQTT, cụ thể:

- Gửi lệnh điều khiển đến thiết bị IoT thông qua giao thức MQTT (bật/tắt đèn, cho cá ăn)
- Cung cấp các API RESTful để giao diện web (Feed Timer Page) có thể gửi lệnh điều khiển, thêm, xóa hoặc hiển thị danh sách lịch.

2. Kết nối MQTT

```
def _get_pub():
    global _mqtt_pub_client
    if _mqtt_pub_client:
        return _mqtt_pub_client

    cfg = current_app.config
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
client_id="aquanova-control-pub")
    user = cfg.get("MQTT_USER")
    if user:
        client.username_pw_set(user, cfg.get("MQTT_PASS"))

    client.tls_set()
    client.connect(cfg.get("MQTT_HOST"),
int(cfg.get("MQTT_PORT", 8883)), keepalive=60)
    client.qos = 1
    _mqtt_pub_client = client
    return _mqtt_pub_client
```




3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

b) Module Control (điều khiển từ Web)

- API /control/light – Điều khiển đèn chiếu sáng

Chức năng: Gửi lệnh bật/tắt đèn đến thiết bị IoT qua topic aquanova/control. Dữ liệu gửi đi dạng JSON:

```
127.0.0.1 - - [16/Oct/2025 12:00:11] "POST /control/feed-now
HTTP/1.1" 200 - {'light': 1}
[Light] Publishing {'light': 1} → aquanova/control
{"light": 1} // Bật đèn
{"light": 0} // Tắt đèn
```

2. API /control/light – Điều khiển đèn chiếu sáng

```
@control_bp.post("/light")
def toggle_light():
    data = request.get_json(force=True) or {}
    light_val = data.get("light")
    if light_val not in (0, 1):
        return jsonify({"error": "light must be
0 or 1"}), 400
    topic = "aquanova/control"
    payload = {"light": light_val}
    _get_pub().publish(topic,
json.dumps(payload), qos=1)
```



3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

b) Module Control (điều khiển từ Web)

- API /control/feed-now – kích hoạt cho ăn ngay lập tức

Dữ liệu gửi đi dạng JSON: Gửi ít nhất 1 lần, có ACK xác nhận, nhưng có thể bị trùng

Mã thành công

```
127.0.0.1 - - [16/Oct/2025 12:00:11] "POST
/control/feed-now HTTP/1.1" 200 -
{'cmd': 'feed', 'amount': 10}
[FEED-NOW] Publishing {'feeding': 1} →
aquanova/control
```

2. API /control/feed-now – kích hoạt cho ăn ngay lập tức

```
@control_bp.post("/feed-now")
def feed_now():
    data = request.get_json(force=True)
    amount = data.get("amount", 20)
    topic = "aquanova/control"
    payload = {"feeding": 1}
    _get_pub().publish(topic,
        json.dumps(payload), qos=1)
    db.collection("feed_logs").add({
        "amount": amount,
        "timestamp": firestore.SERVER_TIMESTAMP
    })
    return jsonify({"ok": True})
```

3.7.4. Backend

7. Các API thiết lập

b) Module Control (điều khiển từ Web)

- API /control/schedule – Tạo lịch cho ăn tự động

Bảng tóm tắt chuỗi sự kiện cho ăn

Giai đoạn	Giao thức	Dữ liệu gửi đi	Mục tiêu
1	HTTP POST	{ "cmd": "feed", "amount": 10 }	Gửi từ trình duyệt đến Flask
2	MQTT Publish	{ "feeding": 1 }	Flask gửi đến ESP32
3	Firestore Log	{ "timestamp": ..., "source": "manual" }	Lưu nhật ký cho ăn

```
@control_bp.post("/schedule")
def add_schedule():
    data = request.get_json(force=True) or {}
    required = ("date", "time", "repeat")
    for key in required:
        if not data.get(key):
            return jsonify({"error": f"{key} required"}), 400

    sid = uuid.uuid4().hex[:12]
    item = {
        "id": sid,
        "date": data.get("date"),
        "time": data["time"],
        "repeat": data["repeat"],
        "created_at": int(time.time())
    }

    db = firestore.client()
    db.collection("schedules").document(sid).set(item)
```

Mã nhận thành công

```
127.0.0.1 - - [23/Oct/2025 08:30:15] "POST /control/schedule
HTTP/1.1" 200 -
{'cmd': 'add_schedule', 'date': '2025-10-23', 'time': '08:30',
'repeat': 'daily', 'amount': 20}
[SCHEDULE] Added: {'id': 'a4c19e3bdf21', 'date': '2025-10-23',
'time': '08:30', 'repeat': 'daily', 'created at': 1734947400}
```



3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

b) Module Control (điều khiển từ Web)

- API /control/schedules – Hiển thị danh sách lịch
- Truy xuất tất cả các lịch đã tạo từ collection schedules.
- Dữ liệu trả về để hiển thị trong bảng “Current Schedules” trên trang web.

```
@control_bp.get("/schedules")
def list_schedules():
    db = firestore.client()
    docs = db.collection("schedules").stream()
    items = [doc.to_dict() for doc in docs]
    return jsonify({"items": items})
```

API /control/schedules/<sid> – Xóa lịch cho ăn

```
@control_bp.delete("/schedules/<sid>")
def delete_schedule(sid):
    db = firestore.client()
    ref =
db.collection("schedules").document(sid)
    if not ref.get().exists:
        return jsonify({"error": "Schedule not
found"}), 404
    ref.delete()
    return jsonify({"ok": True})
```



3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

b) Bộ lập lịch tự động (Scheduler)

- APScheduler (Advanced Python Scheduler): thư viện giúp chạy các tác vụ định kỳ (cron / interval).

Cấu trúc tổng quan

```
from apscheduler.schedulers.background import  
BackgroundScheduler  
from datetime import datetime, date, timedelta  
from firebase_admin import firestore  
import json  
from .routes import _get_pub  
scheduler = BackgroundScheduler()
```

Hàm check_schedules() – Kiểm tra và thực thi lịch cho ăn

```
//1 Truy xuất dữ liệu lịch từ Firestore  
db = firestore.client()  
docs = db.collection("schedules").stream()  
//2 Kiểm tra thời gian thực tế so với lịch  
now = datetime.now()  
hh, mm = map(int, sched_time.split(":"))  
is_time_match = (  
    now.hour == hh and now.minute == mm and  
    abs(now.second) < 30  
)  
//3 Xác định loại lặp lại (repeat)  
if repeat == "none":           # chỉ thực hiện 1 lần, xóa  
    sau khi chạy  
elif repeat == "daily":        # thực hiện mỗi ngày cùng  
    giờ  
elif repeat == "weekly":       # thực hiện mỗi tuần cùng  
    ngày & giờ
```



3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

b) Bộ lập lịch tự động (Scheduler)

- APScheduler (Advanced Python Scheduler): thư viện giúp chạy các tác vụ định kỳ (cron / interval).

Cấu trúc tổng quan

```
from apscheduler.schedulers.background import  
BackgroundScheduler  
from datetime import datetime, date, timedelta  
from firebase_admin import firestore  
import json  
from .routes import _get_pub  
scheduler = BackgroundScheduler()
```

Hàm check_schedules() – Kiểm tra và thực thi lịch cho ăn

```
//4 Gửi lệnh cho ăn qua MQTT  
topic = "aquanova/control"  
payload = {"feeding": 1}  
_get_pub().publish(topic, json.dumps(payload),  
qos=1)  
//5 Ghi log sau khi thực thi  
db.collection("feed_logs").add({  
    "amount": amount,  
    "timestamp": firestore.SERVER_TIMESTAMP,  
})  
//6 Xóa lịch nếu chỉ chạy 1 lần  
if repeat == "none":  
  
db.collection("schedules").document(sched_id).delete()  
  
    print(f"[SCHEDULE] Deleted one-time schedule  
{sched_id}")
```




3.7 Thiết kế website



3.7.4. Backend

7. Các API thiết lập

b) Bộ lập lịch tự động (Scheduler)

- APScheduler (Advanced Python Scheduler): thư viện giúp chạy các tác vụ định kỳ (cron / interval).

Cấu trúc tổng quan

```
from apscheduler.schedulers.background import  
BackgroundScheduler  
from datetime import datetime, date, timedelta  
from firebase_admin import firestore  
import json  
from .routes import _get_pub  
scheduler = BackgroundScheduler()
```

Hàm check_schedules() – Kiểm tra và thực thi lịch cho ăn

```
//4 Gửi lệnh cho ăn qua MQTT  
topic = "aquanova/control"  
payload = {"feeding": 1}  
_get_pub().publish(topic, json.dumps(payload),  
qos=1)  
//5 Ghi log sau khi thực thi  
db.collection("feed_logs").add({  
    "amount": amount,  
    "timestamp": firestore.SERVER_TIMESTAMP,  
})  
//6 Xóa lịch nếu chỉ chạy 1 lần  
if repeat == "none":  
  
db.collection("schedules").document(sched_id).delete  
(  
    print(f"[SCHEDULE] Deleted one-time schedule  
{sched_id}")
```



3.7 Thiết kế website



3.7.5. Database

Dữ liệu được tổ chức theo các collection:

- readings/lưu dữ liệu cảm biến (temp, turbidity, feed, ts).
- schedules/ lưu các lịch cho ăn tự động (date, time, repeat, amount).
- feed_logs/ lưu lịch sử cho ăn thực tế.

(default)	schedules	4a4572d77b93
+ Start collection	+ Add document	+ Start collection
+ Add field		+ Add field
feed_logs	18d341b430d8	amount: 20
light_logs	4a4572d77b93	created_at: 1760715893
readings	505dc0b87e38	date: "2025-10-18"
schedules	58ac9d993d9c	id: "4a4572d77b93"
	df689052fe81	repeat: "none"
		time: "08:30"

. Khởi tạo trong file firebase_admin_init.py:

```
def init_firebase():
    global firebase_app, db
    cred_path = Config.FIREBASE_CREDENTIALS
    print(f"[Firebase Init] Using credentials:
{cred_path}")
    if not cred_path or not
os.path.isfile(cred_path):
        raise RuntimeError(f"Service account not
found: {cred_path}")
    if not firebase_admin._apps:
        cred = credentials.Certificate(cred_path)
        firebase_app =
firebase_admin.initialize_app(cred)
        db = firestore.client()
    return db
```



3.7 Thiết kế website



3.7.6. Giao thức IoT – MQTT

a. MQTT Listener (Subscriber)

Module MQTT Listener có nhiệm vụ duy trì kết nối giữa hệ thống Flask và các thiết bị IoT (ESP32/STM32) thông qua giao thức MQTT

1. Hàm `on_connect` đảm bảo hệ thống tự động bắt đầu nhận dữ liệu cảm biến ngay khi có kết nối mạng

```
def on_connect(client, userdata, flags, reason_code,
properties=None):

    write_log(f"[MQTT] CONNECTED with code={reason_code}")
    try:
        result, mid = client.subscribe(cfg["MQTT_TOPIC"], qos=1)
        write_log(f"[MQTT] SUBSCRIBE sent
topic={cfg['MQTT_TOPIC']} mid={mid} result={result}")
    except Exception as e:
        write_log(f"[MQTT] SUBSCRIBE ERROR: {e}")
```

2. Cơ chế xử lý ngắt kết nối (`on_disconnect`) : Tự động được gọi khi mất kết nối đến broker

```
def on_disconnect(client, userdata, disconnect_flags, reason_code,
properties=None):
    write_log(f"[MQTT] DISCONNECTED (reason_code={reason_code}) –
attempting reconnect...")

    while True:
        try:
            client.reconnect()
            write_log("[MQTT] RECONNECTED successfully!")
            break
        except Exception as e:
            write_log(f"[MQTT] Reconnect failed: {e}")
            time.sleep(5)
```

Ví dụ

```
2025-10-23T17:00:11 [MQTT] DISCONNECTED (reason_code=4) –
attempting reconnect...
2025-10-23T17:00:17 [MQTT] Reconnect failed: [Errno 110]
Connection timed out
2025-10-23T17:00:22 [MQTT] RECONNECTED successfully!
```

3.7.6. Giao thức IoT – MQTT

a. MQTT Listener (Subscriber)

3. Khởi động kết nối nền

- Chạy vô hạn trong thread nền, đảm bảo Flask có thể xử lý web song song.
- `loop_forever()` giữ kết nối liên tục và kích hoạt callback (`connect/disconnect/message`).

```
def start_mqtt_background(app):  
    client = mqtt.Client(  
        mqtt.CallbackAPIVersion.VERSION2,  
        client_id=f"aquanova-server-{uuid.uuid4().hex[:6]}"  
    )  
    client.username_pw_set(cfg["MQTT_USER"], cfg["MQTT_PASS"])  
    # tạo client với mã định danh duy nhất và đăng kí chủ đề  
    aquanova/telemetry  
    client.tls_set()  
    client.connect(cfg["MQTT_HOST"], int(cfg["MQTT_PORT"]),  
        keepalive=60)  
    client.on_message = on_message  
    client.subscribe(cfg["MQTT_TOPIC"], qos=1)  
    client.loop_forever()
```

2. Cơ chế xử lý ngắt kết nối (`on_disconnect`) : Tự động được gọi khi mất kết nối đến broker

```
def on_disconnect(client, userdata, disconnect_flags, reason_code,  
    properties=None):  
    write_log(f"[MQTT] DISCONNECTED (reason_code={reason_code}) –  
    attempting reconnect...")  
  
    while True:  
        try:  
            client.reconnect()  
            write_log("[MQTT] RECONNECTED successfully!")  
            break  
        except Exception as e:  
            write_log(f"[MQTT] Reconnect failed: {e}")  
            time.sleep(5)
```

Ví dụ

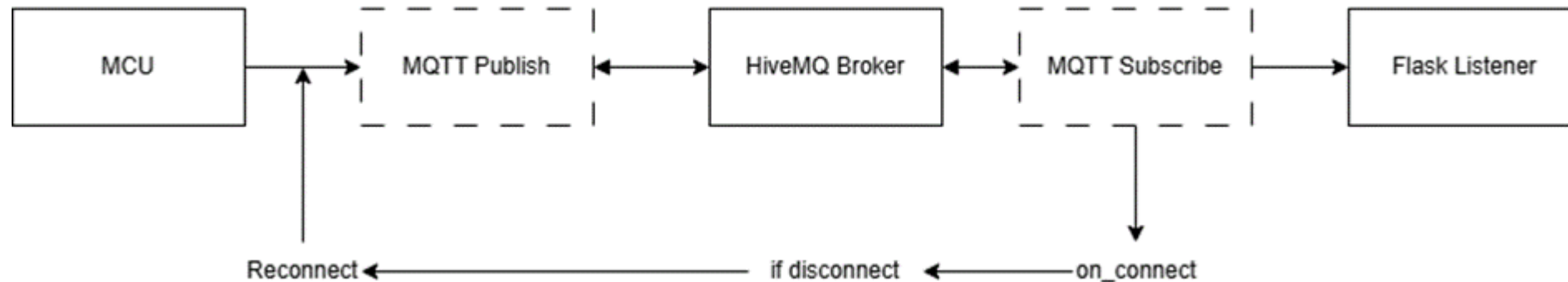
```
2025-10-23T17:00:11 [MQTT] DISCONNECTED (reason_code=4) –  
attempting reconnect...  
2025-10-23T17:00:17 [MQTT] Reconnect failed: [Errno 110]  
Connection timed out  
2025-10-23T17:00:22 [MQTT] RECONNECTED successfully!
```

3.7.6. Giao thức IoT – MQTT

a. MQTT Listener (Subscriber)

3. Khởi động kết nối nền

Dữ liệu được giải mã, thêm timestamp (ts) và lưu vào Firestore collection readings



Hình 60: Luồng kết nối MQTT



3.7 Thiết kế website



Department of
**COMPUTER & COMMUNICATIONS
ENGINEERING**

3.7.6. Giao thức IoT – MQTT

a. MQTT Listener (Subscriber)

3. Khởi động kết nối nền

Dữ liệu được giải mã, thêm timestamp (ts) và lưu vào Firestore collection readings

```
def on_message(client, userdata, msg):  
    payload = msg.payload.decode("utf-8")  
    data = json.loads(payload)  
    data.setdefault("ts", datetime.now(timezone.utc).isoformat())  
    db.collection("readings").add(data)
```

Ví dụ gói tin

```
2025-10-23T17:47:40.229846 [MQTT] MESSAGE topic=aquanova/telemetry  
payload={'turbidity': 176.8, 'temperature': 29, 'feed': 0,  
'thoi_gian': '17:46:39', 'ts': '2025-10-23T10:47:39.936766+00:00'}
```


3.7.6. Giao thức IoT – MQTT

a. MQTT Publisher (Flask gửi lệnh điều khiển)

- File `mqtt/publisher.py` định nghĩa lớp `MQTTPublisher`, chịu trách nhiệm publish lệnh đến thiết bị IoT.
- Khởi tạo và kết nối:

```
self.client = mqtt.Client(  
    mqtt.CallbackAPIVersion.VERSION2,  
    client_id=f"aquanova-publisher-{datetime.now().timestamp()}"  
)  
self.client.username_pw_set(user, pwd)  
self.client.tls_set()  
self.client.connect(cfg["MQTT_HOST"], int(cfg["MQTT_PORT"]),  
    keepalive=60)  
self.client.loop_start()
```

Gửi lệnh điều khiển:

```
def publish(self, topic: str, payload: dict, qos: int = 1):  
    msg = json.dumps(payload)  
    result = self.client.publish(topic, msg, qos=qos)  
    result.wait_for_publish()
```

Trong AquaNova, QoS = 1 được chọn cho cả subscribe và publish dữ liệu:

```
client.subscribe(cfg["MQTT_TOPIC"], qos=1)  
client.publish("aquanova/control", msg, qos=1)
```

Việc gọi `wait_for_publish()` đảm bảo gói tin được gửi hoàn tất trước khi tiếp tục xử lý, giúp hệ thống ổn định hơn.



3.8. Hiệu chuẩn và kiểm thử



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

3.8.1. Đánh giá độ chính xác dữ liệu của môi trường và thiết bị đo

3.8.2. Đánh giá đường truyền kết nối wifi

3.8.3. Độ trễ và tần suất cập nhật



3.8.2. Đánh giá đường truyền kết nối wifi



Mối quan hệ giữa công suất phát P_T , công suất nhận P_R và suy hao đường truyền L_P được mô tả đơn giản như sau:

$$P_R = P_T - L_P \text{ (dBm)}$$

Trong đó, L_P là tổng độ suy hao trên đường truyền, bao gồm suy hao không gian tự do (Free Space Path Loss - FSPL) và suy hao do vật cản.

FSPL được tính theo công thức:

$$FSPL = 20\log_{10}(d) + 20\log_{10}(f) + 20\log_{10}\left(\frac{4\pi}{c}\right) \text{ (dB)}$$

Với d là khoảng cách giữa thiết bị phát và nhận (mét), f là tần số sóng (Hz), và c là vận tốc ánh sáng (3×10^8 m/s).

3.8.2. Đánh giá đường truyền kết nối wifi

Tuy nhiên, khi thử nghiệm kết nối với mạng Wimesh-HCMUTE, module ESP32 đã không thể thiết lập kết nối MQTT thành công mặc dù đã kết nối được WiFi, điều này được xác định là do chính sách bảo mật mạng đã chặn các kết nối ra cổng MQTT.





3.8.3. Độ trễ và tần suất cập nhật



a) Độ trễ của STM32 gửi dữ liệu tới ESP32

Để kiểm chứng tính hợp lệ của kết quả này, tiến hành phân tích lý thuyết. Thời gian truyền tải một gói tin qua UART phụ thuộc vào tốc độ baud và tổng số bit của gói tin. Trong giao thức UART tiêu chuẩn (8N1), mỗi byte dữ liệu (8 bit) yêu cầu thêm 1 bit bắt đầu (start bit) và 1 bit kết thúc (stop bit), nâng tổng số bit cho mỗi byte lên 10 bit.

- Tổng số bit truyền tải: $10 \text{ byte} * 10 \text{ bit/byte} = 100 \text{ bit}$.
- Thời gian truyền 1 chiều (One-way):

$$T = \frac{\sum \text{bit}}{\sum \text{Baud}} = \frac{100 \text{ bit}}{9600 \text{ bit/s}} = 0.010416 \text{ s}$$

- Chuyển đổi sang mili-giây:

$$0.010416 * 1000 = 10.4 \text{ (ms)}$$



3.8.3. Độ trễ và tần suất cập nhật



b) Độ trễ của ESP32 gửi dữ liệu tới STM32

Khi ESP32 truyền dữ liệu đến STM32 thông qua giao thức UART (Universal Asynchronous Receiver and Transmitter), tổng độ trễ T_{total} có thể được biểu diễn như sau:

$$T_{total} = T_{tx} + T_{rx} + T_{proc}$$

Trong đó:

- + T_{tx} : thời gian truyền dữ liệu từ ESP32 qua đường UART.
- + T_{rx} : thời gian STM32 nhận toàn bộ khung dữ liệu.
- + T_{proc} : thời gian xử lý phần mềm (bao gồm ngắt, lưu đệm, và đọc dữ liệu).



3.8.3. Độ trễ và tần suất cập nhật



b) Độ trễ của ESP32 gửi dữ liệu tới STM32

Ví dụ, với tốc độ truyền $R_{baud} = 9600$ bps và gói dữ liệu gồm $N_{byte} = 20$, trong đó mỗi byte chứa 8 bit dữ liệu, 1 bit start và 1 bit stop, ta có:

$$N_{bit} = 20 \times (1 + 8 + 1) = 200 \text{ bit}$$

$$T_{tx} = \frac{200}{9600} \approx 2.08 \text{ ms}$$



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

CHƯƠNG 4: THỰC NGHIỆM, KIỂM CHỨNG VÀ ĐÁNH GIÁ



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

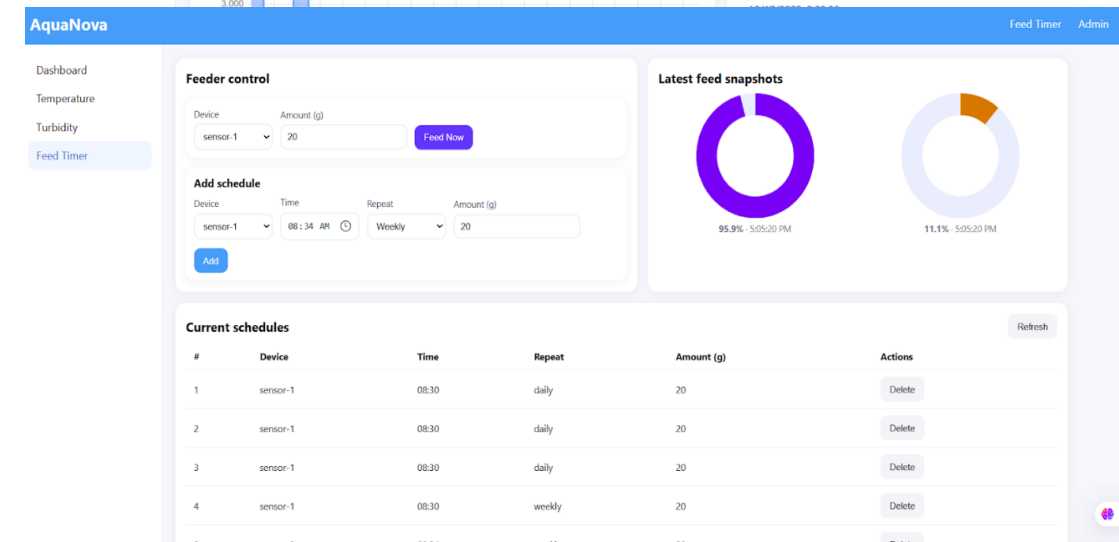
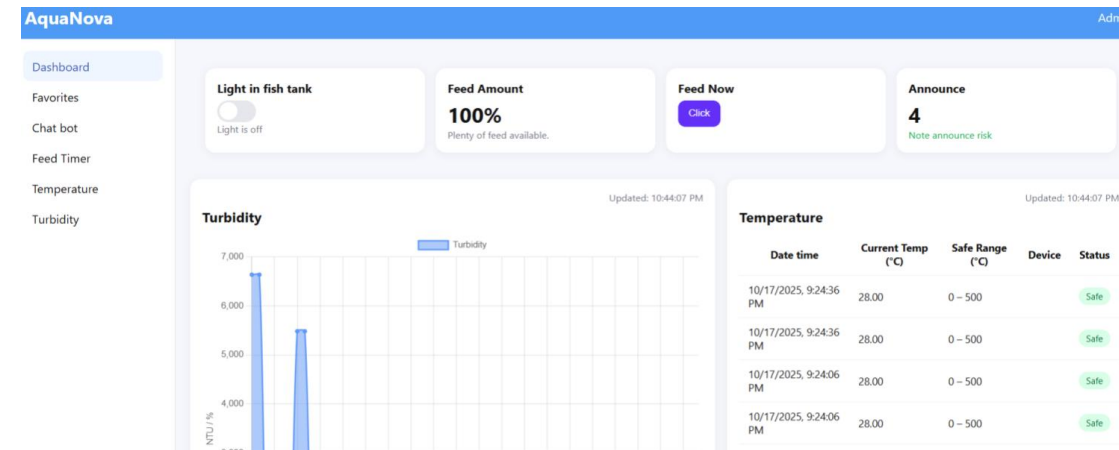
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN



Kết quả



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING





5.1. Kết luận

Ưu điểm

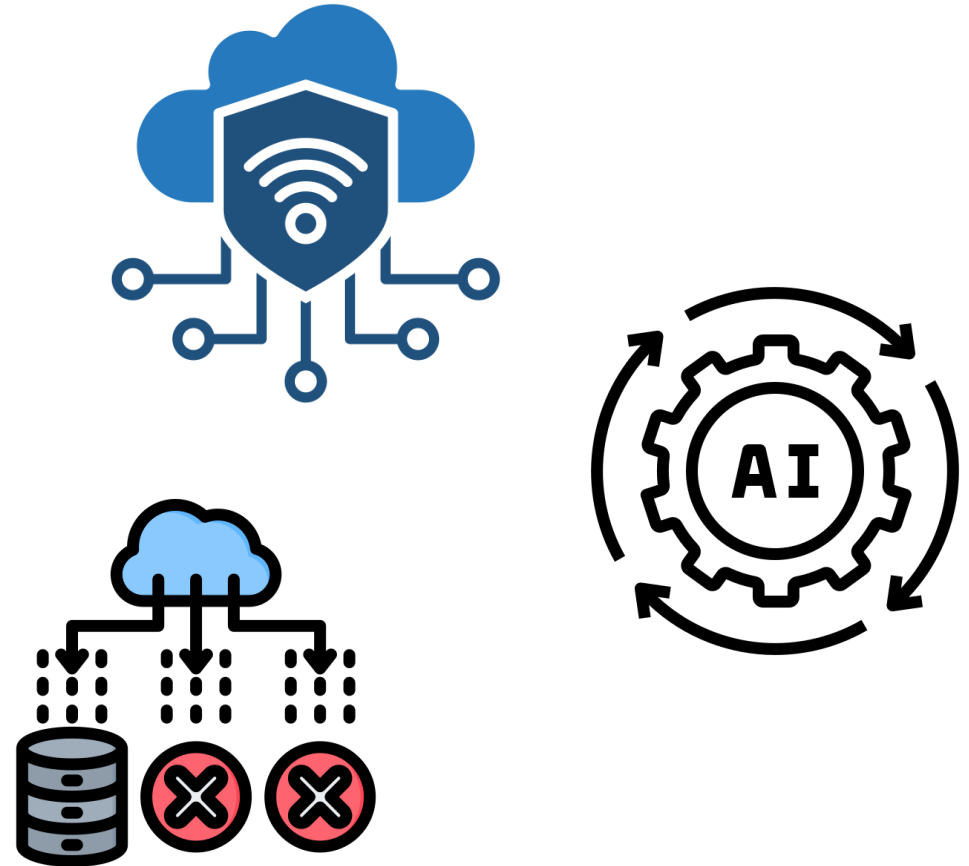
- Mô hình vận hành ổn định, đọc và xử lý dữ liệu độ đọc chính xác, phản hồi nhanh.
- Kết nối UART giữa STM32 và ESP32 đơn giản, truyền dữ liệu ổn định, ít lỗi.
- Giao tiếp dữ liệu qua MQTT giúp dễ dàng giám sát từ xa, tiện cho ứng dụng IoT.
- Cấu trúc mạch gọn gàng, chi phí thi công thấp hơn nhiều so với các hệ thống thương mại có cùng chức năng.
- Có khả năng mở rộng thêm nhiều cảm biến khác (pH, DO, mực nước) và điều khiển thiết bị như bơm, quạt, hoặc máy lọc nước.
- Tự động hóa được các tác vụ như cho ăn, lọc và thay nước theo độ đục hoặc thời gian định sẵn, giảm thao tác thủ công.

Nhược điểm

- Mô hình chưa có cơ chế mã hóa hoặc xác thực dữ liệu, dẫn đến tính bảo mật chưa cao.
- Cảm biến độ đục dễ bị ảnh hưởng bởi môi trường (ánh sáng ngoài, cặn, rêu bám), cần hiệu chuẩn định kỳ.
- Chưa có khả năng tự động kết nối lại WiFi nếu mất mạng, cần reset thủ công để khởi động lại ESP32.
- Giao diện giám sát còn đơn giản, chưa có cảnh báo thông minh hoặc lưu trữ dữ liệu dài hạn.
- Thuật toán xử lý còn cơ bản, chưa tối ưu về thời gian và khả năng phân tích dữ liệu môi trường.

5.2. Hướng phát triển

- a) Tăng cường bảo mật
- b) Tự động điều chỉnh lượng thức ăn dựa vào độ đục hoặc lịch định sẵn.
- c) Tích hợp chức năng gửi lịch hẹn giờ cho ăn (Timer Scheduling) và phân luồng gói tin MQTT





TÀI LIỆU THAM KHẢO



Department of
COMPUTER & COMMUNICATIONS
ENGINEERING

- [1] Google Firebase, “Cloud Firestore Overview,” Firebase Documentation, 2025. [Online]. Available: <https://firebase.google.com/docs/firestore>
- [2] Maxim Integrated, “DS18B20 Programmable Resolution 1-Wire Digital Thermometer,” Rev. 6, 2019. [Online]. Available: <https://datasheets.maximintegrated.com/en/ds/DS18B20.pdf>
- [3] DFRobot, “Gravity: Analog Turbidity Sensor/SEN0189,” 2019. [Online]. Available: https://wiki.dfrobot.com/Turbidity_sensor_SKU__SEN0189
- [4] R. Burhan, R. A. Rehman, B. Khan, and B. Kim, "IoT Elements, Layered Architectures, and Security Issues: A Comprehensive Survey," *Sensors*, vol. 18, no. 9, p. 2796, Aug. 2018. [Online]. Available: <https://doi.org/10.3390/s18092796>
- [5] “ISO 7027-1:2016 - Water quality — Determination of turbidity,” International Organization for Standardization, 2016, <https://www.iso.org/standard/62801.html>
- [6] J. Trevathan, T. Atchison, L. Read, P. Hitchens, and W. Read, “Towards the Development of an Affordable and Practical Turbidity Sensor,” *Sensors*, vol. 20, no. 7, p. 1993, 2020, <https://doi.org/10.3390/s20071993>



CẢM ƠN THẦY VÀ CÁC BẠN ĐÃ LẮNG NGHE

GVHD: ThS. Trịnh Quốc Thanh

SVTT: Nhóm 2

Môn: Kiến trúc và giao thức IoT